Grafana
Visualizando Dados

Apostila do aluno

Fábio Silva
Arquiteto de Soluções
Green Tecnologia

5 noites · 18h30 às 22h30
Fundamentos, interpretação e exercícios no ambiente local

Docker é o laboratório executável desta edição. A instalação nativa no Ubuntu possui roteiro e exemplos; os scripts serão preparados e testados pelo professor no Azure antes da liberação.

Sumário

Como usar esta apostila 3

Material e ambiente local 4

Noite 1 — Fundamentos e arquitetura 5

Noite 2 — Métricas e contexto de observabilidade 14

Noite 3 — Exploração de logs 26

Noite 4 — Dashboards e anotações 29

Noite 5 — Alertas e projeto final 32

Complemento de instalação Ubuntu 39

Objetivo e situação deste roteiro 39

Duas instalações e componentes separados 39

1 Conferir o sistema 39

2 Preparar APT e adicionar o repositório oficial 39

3 Instalar e iniciar o serviço 40

4 Validar localmente 40

Registro do teste Azure a preencher pelo professor 40

Preservar o trabalho entre as noites 41

Exercício opcional de alerta de erros 41

Referências 41

Referências dos fundamentos e integrações 41

Continuação: guia ilustrado dos exercícios 42

Green Tecnologia — Fábio Silva, Arquiteto de Soluções.

Como usar esta apostila

Revisão V11: teoria de métricas integrada ao contexto de observabilidade, logs, traces, exporters e arquitetura antes da execução no Grafana. O guia ilustrado aparece depois da parte conceitual e também está em alunos/Guia_Visual_Exercicios_V11.html.

O curso tem cinco noites, das 18h30 às 22h30, com intervalo de 20h20 a 20h45. São 20 horas de programação, incluindo 2h05 de intervalos; tempo de atividades: 17h55.

Noites 1, 3, 4 e 5

Bloco	Horário	Duração
Teoria	18h30–19h55	85 min
Demonstração	19h55–20h20	25 min
Intervalo	20h20–20h45	25 min
Exercícios comentados	20h45–21h00	15 min
Prática do aluno	21h00–22h15	75 min
Correção	22h15–22h30	15 min

Noite 2 — Métricas e PromQL

Bloco	Horário	Duração
Teoria de métricas	18h30–20h20	110 min
Intervalo	20h20–20h45	25 min
Exercício guiado: professor e alunos no Grafana	20h45–21h20	35 min
Consolidação autônoma dos mesmos exercícios	21h20–22h15	55 min
Correção	22h15–22h30	15 min

Na Noite 2, acompanhe a teoria e os exemplos de interpretação antes do intervalo. O ambiente Grafana será aberto para a execução conjunta às 20h45. Os 55 minutos seguintes servem para retomar os mesmos passos, comparar resultados e concluir as evidências. Não há demonstração separada de métricas antes do intervalo nem exercício obrigatório adicional.

As capturas deste material foram obtidas no laboratório local Grafana 12.0.1 em 19/10/2026. Elas ajudam a localizar os campos; não representam acesso atual à VM do professor. As quatro outras folhas

de exercícios mantêm seu conteúdo anterior. Os modelos para importar são referências opcionais em pasta separada; a atividade continua sendo manual.

Material e ambiente local

O professor poderá distribuir a versão da turma pelo GitHub ou pelo Google Drive. Os links finais serão informados antes da aula. Nesta entrega não foi criado repositório nem publicado arquivo no Drive.

No GitHub, clone o repositório informado pelo professor ou baixe seu ZIP. No Google Drive, baixe o ZIP completo e extraia antes de executar. Use uma única revisão do pacote para manter slides, folhas e laboratório consistentes.

O caminho Docker é o laboratório executável. O roteiro Ubuntu apresenta a segunda instalação, cujos scripts serão preparados e testados depois pelo professor em VM Azure.

Iniciar Docker

Entre na pasta lab/tutorial-environment e execute:

```
docker ps
docker compose -p green-fundamentals up -d --build
docker compose -p green-fundamentals ps
```

No Windows, pode usar iniciar-lab.ps1; no Linux, bash iniciar-lab.sh. Docker e Compose precisam estar instalados e ativos. O primeiro download pode levar alguns minutos.

Serviço	Navegador no ambiente local	Endereço entre containers
Grafana	http://localhost:3300	http://grafana:3000
Aplicação	http://localhost:38081	http://app:80
Prometheus	http://localhost:39090	http://prometheus:9090
Loki	http://localhost:33100	http://loki:3100
Receptor de notificações	http://localhost:38090	http://webhook:8080/

O pacote usa portas diferentes do repositório original para evitar conflitos conhecidos no ambiente de preparação. Se clonar diretamente https://github.com/grafana/tutorial-environment, confira os endereços: Grafana 3000 e aplicação 8081 no snapshot usado pelo curso.

O Grafana Docker do exemplo usa acesso anônimo administrativo para a demonstração. Os serviços publicados no pacote ficam vinculados a 127.0.0.1. Não transfira essa configuração automaticamente para produção.

Noite 1 — Fundamentos e arquitetura

Objetivo: compreender os sinais, o papel das ferramentas e os caminhos dos dados antes da prática.

O que é observabilidade?

É a capacidade de investigar o estado interno de um sistema a partir dos sinais que ele produz.

A investigação começa com uma pergunta e usa evidências para testar explicações.

Uma investigação pode começar pela lentidão do checkout e avançar até a etapa responsável pelo atraso. A capacidade de fazer perguntas úteis depende de coleta, contexto e correlação, além da ferramenta de visualização.

Monitoramento e investigação

Monitoramento acompanha condições conhecidas e avisa quando uma regra é atendida.

Observabilidade ajuda a investigar perguntas novas sobre o comportamento do sistema.

Um alerta de taxa de erros acompanha uma condição conhecida. Para descobrir em qual rota, versão ou serviço o problema se concentra, precisamos investigar os dados. Os dois usos se complementam.

Métricas: medidas ao longo do tempo

Métricas são valores numéricos associados a instantes e dimensões.

Permitem acompanhar tendências e comparar grupos com consultas agregadas.

Toda leitura precisa de unidade, período e identificação do que foi medido. Uma taxa de 12 req/s e um total de 12 requisições descrevem quantidades diferentes. As labels separam as séries por dimensões como rota e instância.

Logs: registros de eventos

Logs registram acontecimentos com horário e contexto.

Uma linha pode conter nível, serviço, mensagem, rota e identificador de trace.

Uma mensagem de erro é evidência registrada pela aplicação, mas pode não conter toda a causa do problema. Logs estruturados facilitam a extração de campos e a busca; volume e retenção também precisam ser planejados.

Traces: caminho de uma requisição

Um trace reúne o trabalho associado a uma requisição.

Cada span representa uma etapa, com início, duração e atributos.

Spans podem se sobrepor quando o trabalho acontece em paralelo. Por isso, somar as durações de todos eles nem sempre produz a duração total do trace. Instrumentação e propagação do contexto precisam existir na aplicação.

Métricas, logs e traces juntos

A métrica mostra a mudança. O log acrescenta contexto. O trace mostra as etapas.

A conclusão depende da combinação das evidências.

Um aumento de latência, um evento de timeout e um span demorado no pagamento sustentam uma hipótese de investigação. A proximidade de dois eventos no tempo, isoladamente, não prova uma relação de causa.

O que é o Grafana?

É uma plataforma para consultar, explorar e visualizar dados de diferentes fontes.

Permite criar dashboards e regras de alerta sobre os dados conectados.

Neste laboratório, a telemetria fica nos backends Prometheus, Loki e Tempo. O Grafana guarda configurações e dashboards e consulta as fontes conectadas. Uma fonte de dados define como conectar; a consulta define o que selecionar; o painel define como apresentar.

Grafana: vantagens e limites

Vantagens Reúne fontes, visualizações e alertas em uma interface.

Limites Depende da coleta, das fontes disponíveis e da qualidade das consultas.

A vantagem é reunir diferentes fontes em uma experiência de consulta e acompanhamento. A limitação é que um dashboard depende de coleta e consultas corretas. Recursos de autenticação e governança também variam entre edições e configurações.

O que é o Prometheus?

É um sistema de monitoramento e alertas que coleta e armazena métricas em séries temporais.

Neste laboratório, consulta endpoints HTTP de métricas periodicamente.

No modelo pull do laboratório, o Prometheus inicia um pedido HTTP a cada endpoint de métricas. A resposta contém amostras que são armazenadas por nome de métrica e combinação de labels. O Grafana consulta essas séries por meio da API do Prometheus.

Prometheus: vantagens e limites

Vantagens Modelo por labels, PromQL e ecossistema de exporters.

Limites Alta cardinalidade e retenção exigem planejamento. Não guarda logs nem traces.

Cada nova combinação de valores de labels cria uma série diferente. IDs de requisições ou usuários podem multiplicar esse número. A coleta periódica é apropriada para acompanhamento operacional, mas não substitui registros exatos de faturamento ou auditoria transacional.

Counter, gauge e distribuições

Counter acumula ocorrências e pode reiniciar. Gauge sobe e desce.

Histogram e summary descrevem distribuições de observações.

Um counter pode voltar a zero quando o processo reinicia. A função rate interpreta a evolução do contador e trata seus resets. Histogramas agrupam observações em buckets; summaries podem calcular quantis no cliente. Counter e gauge são o foco dos primeiros exercícios.

O que é o Loki?

É um backend para armazenar e consultar logs.

Organiza fluxos por labels e usa LogQL para selecionar eventos e obter medidas derivadas.

O modelo de indexação do Loki se apoia em labels e metadados, enquanto o corpo dos logs é compactado em chunks. Uma aplicação ou um agente precisa enviar os eventos; instalar somente o Loki não cria a coleta dos arquivos.

Loki: vantagens e limites

Vantagens Integração com o Grafana e correlação por contexto.

Limites Labels mal escolhidos e buscas amplas podem aumentar o custo das consultas.

Selecionar poucos fluxos e um período adequado reduz a quantidade de dados examinada. Labels de cardinalidade elevada podem fragmentar excessivamente os fluxos. Não existe uma economia percentual de armazenamento garantida para qualquer instalação.

O que é o Tempo?

É um backend para armazenar e consultar traces distribuídos.

Recebe spans de aplicações instrumentadas e permite investigar a duração das etapas.

A porta 4318 recebe OTLP HTTP no ambiente do professor. A porta 3200 atende a API de consulta do Tempo. Receber spans e consultar traces são funções distintas. A aplicação de demonstração usa o SDK OpenTelemetry para enviar spans diretamente.

Tempo: vantagens e limites

Vantagens Expõe etapas e durações da requisição; permite navegar a partir do trace_id.

Limites Depende de instrumentação, propagação de contexto e retenção.

A árvore de spans ajuda a localizar etapas demoradas. Sua completude depende da instrumentação, da propagação de contexto, da amostragem e da retenção. Um backend de traces instalado não produz automaticamente spans de todas as aplicações.

PromQL: consulta de métricas

PromQL seleciona, filtra e combina séries temporais.

rate calcula uma taxa a partir de um contador; sum pode agregar as séries.

A janela [5m] fornece as amostras para rate. Já Last 1 hour controla o período exibido na tela. Agregar por route soma as séries de cada rota. Esses passos serão praticados com as métricas tns_ do Grafana News na Noite 2.

```
sum(rate(tns_request_duration_seconds_count[5m])) by (route)
```

LogQL: consulta de logs

O seletor escolhe os fluxos. Os filtros escolhem linhas. Parsers extraem campos.

Também é possível calcular contagens e taxas a partir dos eventos.

No laboratório do professor, service identifica a aplicação. No Grafana News, os exercícios usam filename. O filtro |= seleciona texto nas linhas; o parser | json extrai campos de conteúdo JSON. Um envelope Docker pode exigir primeiro expor seu campo log.

```
{service="ecommerce-shop"} |= "ERROR"
```

TraceQL: consulta de traces

TraceQL seleciona traces a partir de condições sobre spans, recursos e duração.

Um TraceID conhecido também permite abrir uma requisição específica.

Uma expressão como { resource.service.name = "ecommerce-shop" } seleciona traces com spans do serviço. Já um TraceID conhecido abre uma requisição específica. Nem todo intervalo terá traces que correspondam à condição escolhida.

```
{ resource.service.name = "ecommerce-shop" }
```

Promtail: papel no laboratório

Promtail acompanha arquivos e logs de containers, acrescenta labels e envia os eventos ao Loki por HTTP.

Ele aparece aqui por fazer parte do ambiente legado do curso.

Promtail lê logs continuamente, acrescenta labels e envia os eventos para a API HTTP do Loki. Ele não é o responsável pelo envio de spans ao Tempo neste ambiente. Uma linha de log com trace_id e um span são registros diferentes ligados pelo contexto.

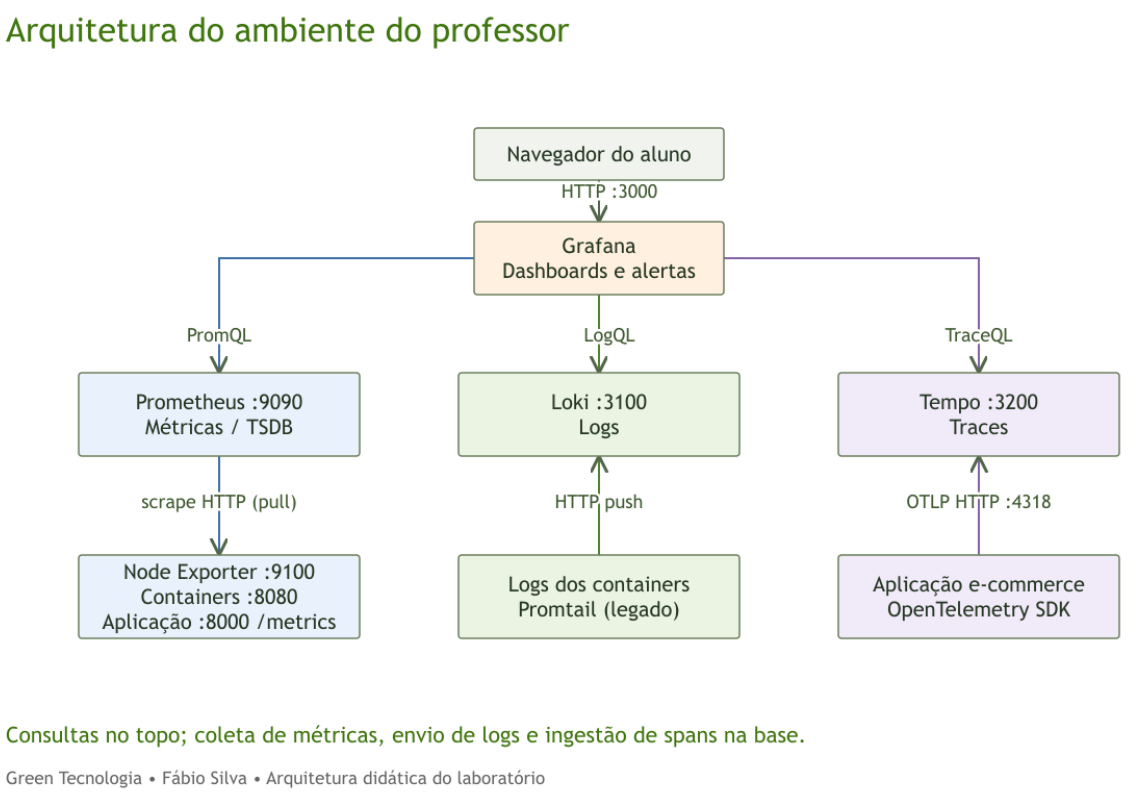
Promtail e a migração para Alloy

Promtail encerrou o suporte em 2 de março de 2026.

Para novas implantações, estude Grafana Alloy ou outro coletor mantido. O laboratório preserva o fluxo legado para comparação.

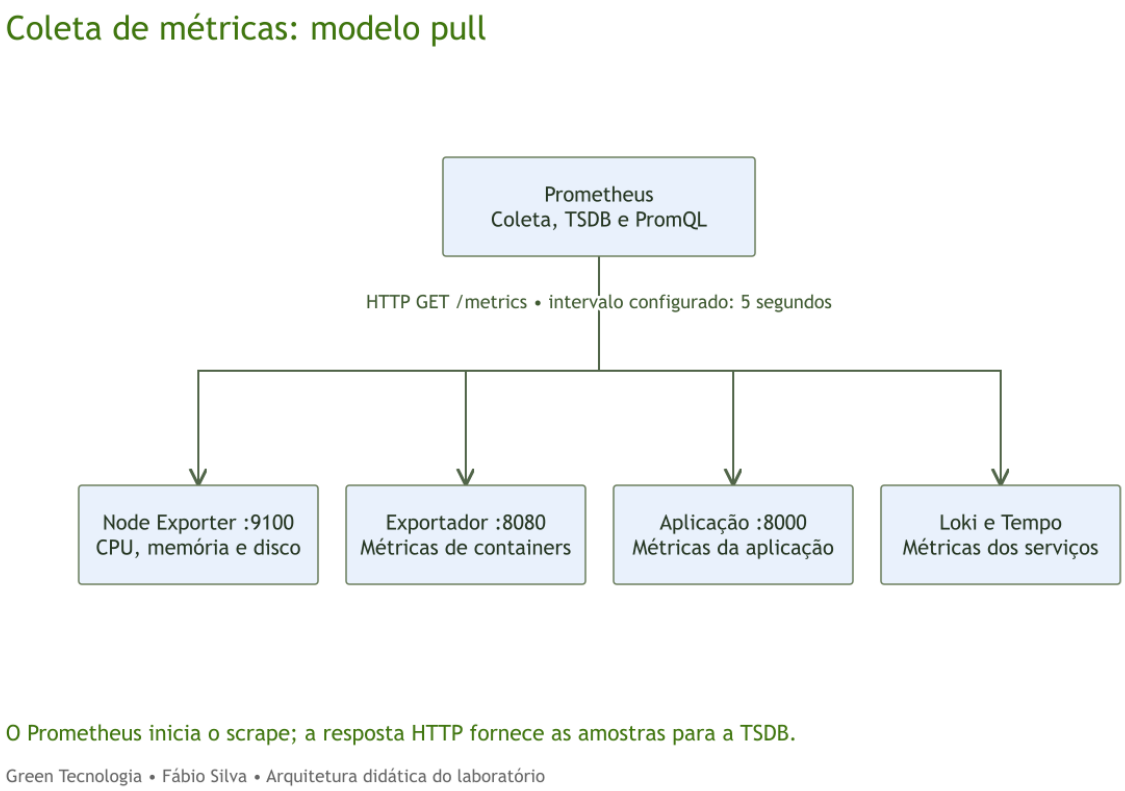
O suporte ao Promtail terminou em 02/03/2026. O curso mantém o fluxo legado para compreender o laboratório existente e apresenta a migração para Alloy como evolução. A migração exige conversão e validação da configuração; ela não foi executada na VM nesta entrega.

Arquitetura do ambiente do professor



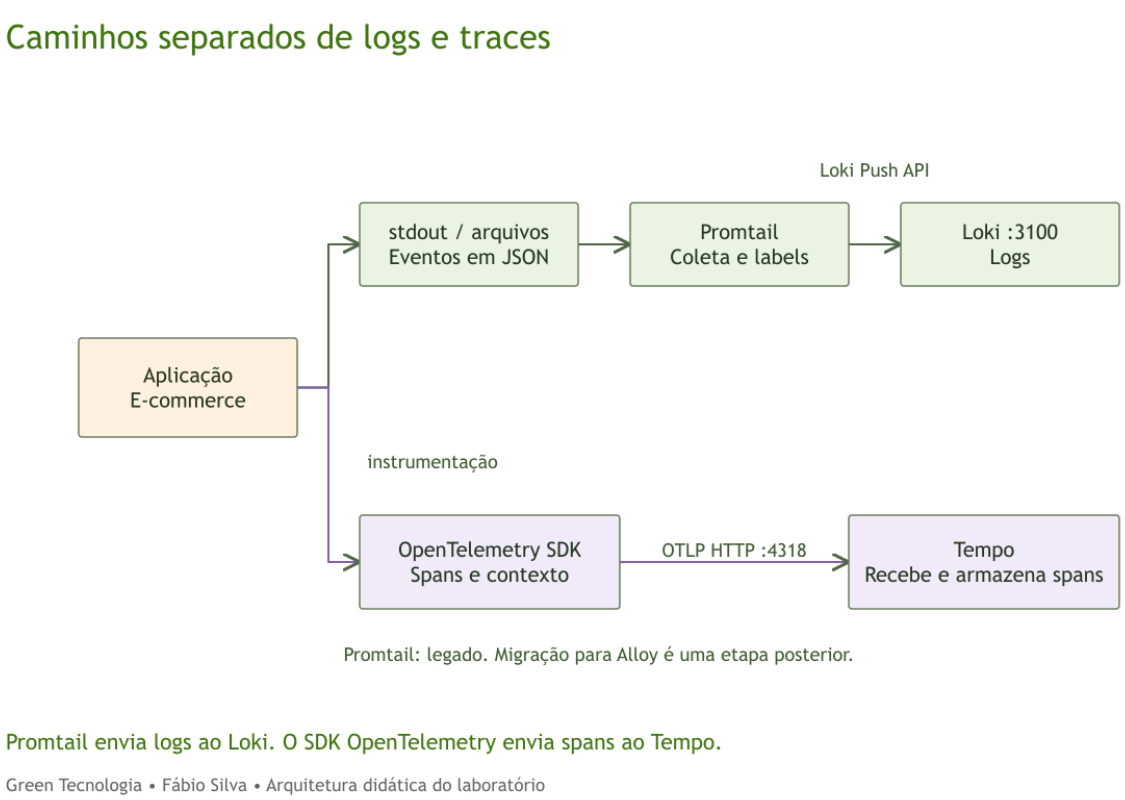
Grafana consulta Prometheus com PromQL, Loki com LogQL e Tempo com TraceQL. Prometheus inicia o scrape dos exporters. O coletor de logs envia ao Loki e a aplicação instrumentada envia spans ao Tempo.

Coleta de métricas: modelo pull



As setas representam o pedido HTTP de coleta. As amostras retornam na resposta e ficam na TSDB. O intervalo de 5 segundos é a configuração do arquivo local do professor; ele deve ser escolhido conforme o ambiente.

Caminhos separados de logs e traces



A aplicação gera eventos em stdout ou arquivos e também spans pelo SDK OpenTelemetry. Promtail coleta os eventos e envia ao Loki. Os spans seguem por OTLP HTTP até o Tempo, sem passar pelo Promtail.

Tutorial do aluno e demonstração

O aluno usa Grafana News no ambiente local.

O professor também mostra host, containers e e-commerce na VM Azure. Os conceitos são os mesmos; as métricas e labels mudam.

Os coletores de host e containers e a aplicação de e-commerce não fazem parte do pacote básico Grafana News. Antes de executar uma consulta, confirme se está usando a fonte e o ambiente correspondentes.

Preparação para a demonstração

Vamos observar a instalação Docker, a aplicação, as fontes e a navegação.

O roteiro Ubuntu explica a segunda instalação. Os scripts nativos serão validados antes da distribuição.

Docker e Compose precisam estar ativos. O professor demonstra antes do intervalo; os alunos recebem e executam as folhas a partir das 21h00. A instalação nativa de Grafana no Ubuntu é uma rota separada; ela não instala automaticamente toda a pilha de observabilidade.

Referência em vídeo

https://www.youtube.com/watch?v=TQur9GJHIIQ

Exercício 1.1 — Obter e iniciar o laboratório Docker

Tipo: Execução local. Tempo previsto: 25 minutos.

1. Baixe o pacote oficial da turma pelo GitHub ou Google Drive quando o professor disponibilizar o link.

2. Extraia ou clone a pasta completa e abra lab/tutorial-environment.

3. Confirme Docker ativo e inicie com o script de entrada ou Compose.

4. Abra a aplicação e o Grafana nos endereços locais do pacote.

Entrega: Ambiente local iniciado e dois endereços acessíveis.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 1.2 — Gerar atividade conhecida

Tipo: Navegação no navegador. Tempo previsto: 25 minutos.

1. Na aplicação Grafana News, publique um link com título e URL válidos.

2. Vote no link e atualize a página.

3. Registre o horário e descreva as ações feitas.

Entrega: Link publicado, voto registrado e anotação do horário.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 1.3 — Reconhecer os recursos do Grafana

Tipo: Navegação no Grafana. Tempo previsto: 25 minutos.

1. Localize Explore, Dashboards e Connections.

2. Abra a fonte Prometheus já existente.

3. Identifique a URL interna e teste a conexão quando disponível.

4. Escreva uma frase sobre o papel de cada recurso.

Entrega: Fonte identificada e explicação dos recursos de navegação.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Noite 2 — Métricas e contexto de observabilidade

Objetivo: escolher e interpretar medidas com unidade, população, dimensões e tempo; depois aplicar esses conceitos a contadores, taxas e agrupamentos no Grafana.

Na Noite 2, a teoria ocupa 18h30–20h20 (110 minutos), seguida do intervalo de 20h20–20h45. Às 20h45–21h20, professor e alunos executam juntos os três exercícios de métricas no Grafana. Às 21h20–22h15, retome os mesmos procedimentos no seu ambiente e conclua os registros sem acompanhar cada clique do professor. Correção: 22h15–22h30. A Noite 2 não possui um bloco separado de demonstração antes do intervalo.

Métricas e contexto de observabilidade

Métricas ajudam a responder perguntas sobre o comportamento de sistemas. Para interpretar um valor, identifique o que foi medido, a unidade, a população e o instante ou intervalo. A forma do gráfico só ganha significado depois dessas verificações. Ao longo desta noite, estudaremos estados atuais, contagens acumuladas, taxas e distribuições, sempre distinguindo uma observação de uma conclusão sobre sua causa.

A primeira parte da noite é dedicada a conceitos e casos didáticos, sem operação do Grafana. Depois do intervalo, todos executam juntos as consultas de Grafana News. Na sequência, cada aluno consolida os exercícios 2.1, 2.2 e 2.3. Os exemplos com números usados na teoria são inventados para ensino; não representam medições do ambiente da turma.

Referência: Documentação oficial

A evolução da observação de sistemas

Checks de host observam disponibilidade e recursos. Métricas de aplicação acrescentam volume, erros e duração das operações. Em serviços distribuídos, uma requisição pode percorrer processos e dependências externas. A investigação precisa preservar o contexto entre essas etapas.

CPU baixa não comprova que uma compra, autenticação ou consulta funciona. Uma aplicação pode aguardar banco, rede ou serviço externo. A observação deve relacionar recursos, comportamento do serviço e efeito percebido pelo usuário.

Referência: Documentação oficial

Monitoramento e observabilidade

Monitoramento acompanha sinais, tendências e condições de interesse. Observabilidade descreve a capacidade de compreender o comportamento interno a partir dos dados emitidos. Essas atividades se complementam. Ambas podem usar métricas, logs e traces.

Um alerta indica uma mudança que merece atenção. A investigação compara hipóteses com evidências. A instalação de uma interface não substitui instrumentação, contexto, retenção e qualidade das consultas.

Referência: Documentação oficial

Métricas, logs e traces

Métricas são medidas numéricas observadas ao longo do tempo. Logs descrevem eventos e seus campos. Traces relacionam operações de uma requisição por spans. Prometheus ou Mimir armazenam métricas, Loki armazena logs e Tempo armazena traces. Grafana consulta as fontes.

Os sinais respondem perguntas diferentes sobre o mesmo comportamento. Custo e utilidade dependem de volume, cardinalidade, retenção, consultas e amostragem. Perfis contínuos acrescentam evidências sobre consumo de recursos dentro do código.

Referência: Documentação oficial

Métrica: número com contexto

Uma medição interpretável combina grandeza, valor, unidade e contexto. Memória usada, duração de chamada e quantidade de requisições não podem ser comparadas como se fossem o mesmo objeto. O instante ou intervalo e as dimensões identificam a situação à qual o valor se refere.

Exemplo didático: um servidor utiliza 6 GiB. Em uma máquina de 8 GiB, isso corresponde a 75% da capacidade; em uma máquina de 64 GiB, a aproximadamente 9,4%. O mesmo absoluto leva a leituras diferentes. Também precisamos conferir se a definição inclui cache. Antes de comparar painéis, normalize unidades e confirme que ambos descrevem a mesma grandeza. Uma proporção de 0,05 pode representar 5%, mas 0,05 req/s representa frequência de eventos.

Referência: Documentação oficial • Documentação oficial

Infraestrutura, aplicação e negócio

Métricas de infraestrutura descrevem recursos como memória, processador, disco e filas. Métricas de aplicação descrevem a atividade do serviço, por exemplo volume, duração e respostas com falha. Métricas de negócio acompanham resultados como compras concluídas e pagamentos aprovados. Nenhuma camada substitui automaticamente as demais.

Exemplo didático: uma loja recebe mil tentativas de compra e duzentas falham enquanto a CPU permanece em 30%. Isso mostra que baixa utilização não comprova bom atendimento. Uma dependência externa é uma hipótese, mas os números não comprovam a causa. Para investigar, precisamos correlacionar sinais e entender a definição de cada medição. Total de chamadas HTTP também não equivale a total de pessoas: uma mesma pessoa ou automação pode fazer muitas chamadas.

Referência: Documentação oficial

Uma série temporal acompanha um contexto

Uma série temporal reúne observações de um mesmo contexto ao longo do tempo. Cada amostra tem um valor e um timestamp, que a posiciona na linha do tempo. No Prometheus, o nome da métrica e seu conjunto de labels identificam uma série. Se uma dimensão muda, podemos estar observando outra série.

Exemplo didático: o servidor A apresenta 6, 7 e 5 GiB em três instantes; o servidor B apresenta 3, 3 e 4 GiB nos mesmos instantes. Misturar todos os pontos em uma única linha, sem preservar a origem, produz uma história que nenhum servidor viveu. Leia a legenda e a seleção de ambiente antes de

interpretar tendência, mudança ou comparação. Um ponto isolado descreve uma observação, não uma tendência.

No formato Prometheus, nome e labels identificam a série e cada amostra tem valor e instante. HELP e TYPE descrevem a métrica. Labels de alvo podem ser acrescentados na coleta. Sem timestamp explícito, usa-se o horário de coleta conforme a configuração.

Referência: Documentação oficial

Eventos não são amostras

Evento e amostra são conceitos distintos. Uma aplicação pode atualizar um contador em cada requisição, enquanto o coletor registra periodicamente o acumulado. A coleta não precisa produzir um ponto separado para cada evento. A agregação permite observar volume, mas não preserva automaticamente todos os detalhes individuais.

Exemplo didático sem reset: o contador vale 100 em uma coleta e 140 na seguinte. O aumento observado é 40, mas os dois pontos não revelam se as requisições chegaram em rajada ou distribuídas, nem quem as fez. Logs e traces podem ajudar a investigar eventos individuais. Coleta periódica de métricas também não é a mesma decisão técnica que amostragem de traces; os mecanismos operam sobre sinais e objetivos diferentes.

Referência: Documentação oficial

Coleta: frequência, tempo e lacunas

A frequência de coleta determina quando novas observações podem ser registradas. A atualização da interface determina quando a tela solicita resultados novamente. Atualizar o navegador mais rápido não cria medições que o backend ainda não coletou. Maior frequência pode revelar detalhes adicionais, mas também gera mais trabalho e armazenamento.

Exemplo didático: com coletas nos segundos 0, 5 e 10, uma fila pode subir para 50 itens no segundo 2 e retornar a 2 itens no segundo 4. As duas leituras nas bordas podem não mostrar o pico. Um contador de ocorrências durante esse intervalo preservaria outra informação, sem reconstruir automaticamente a curva da fila. Escolha a medição e a frequência em função da pergunta, da duração do fenômeno e do custo.

Referência: Documentação oficial • Documentação oficial

Do sistema observado até o painel

O caminho de uma métrica inclui produção, transporte, armazenamento, consulta e interpretação. A aplicação pode produzir medições por instrumentação. Um exporter pode expor dados de componentes que não apresentam diretamente o formato esperado pelo coletor. Em pull, o coletor busca informações; em push, um produtor envia informações a um receptor.

No laboratório, Prometheus coleta e armazena as séries e Grafana consulta essa fonte para apresentar resultados. Outros pipelines podem utilizar OpenTelemetry e envio por OTLP. A existência de um painel não cria instrumentação automaticamente. Pushgateway tem finalidades específicas, como determinados trabalhos curtos em lote, e não é uma substituição universal do scraping. Se um alvo não responde, investigue também rede e configuração antes de concluir que o componente caiu.

Referência: Documentação oficial • Documentação oficial • Documentação oficial

Exporters e instrumentação

Uma biblioteca permite instrumentar operações no próprio código. Um exporter expõe dados de outro sistema em um formato coletável. Um collector recebe, transforma e encaminha telemetria. A escolha depende da pergunta e da origem do dado.

Node exporter atende métricas de host Linux; cAdvisor atende containers; blackbox exporter executa sondagens. Métricas de aplicação e negócio precisam de instrumentação adequada. Os dez exemplos de infraestrutura da folha de apoio exigem node_exporter e não substituem os exercícios obrigatórios do Grafana News.

Referência: Documentação oficial • Documentação oficial

Gauge: um estado que pode subir e descer

No vocabulário Prometheus, gauge representa um valor que pode subir ou descer. Temperatura, memória atual, conexões simultâneas e tamanho de fila são exemplos de estados. O valor precisa ser interpretado com sua unidade, capacidade e momento de leitura. Uma queda pode ser normal ou exigir investigação, conforme o contexto.

Exemplo didático: uma fila apresenta 12, depois 30 e depois 8 itens. A redução de 22 itens no saldo não comprova que exatamente 22 foram processados: novos itens podem ter entrado enquanto outros saíam. Para conhecer entradas e saídas separadamente, precisamos de medições próprias. Não aplique automaticamente rate de contador a memória ou temperatura. Essa função considera uma semântica de acumulado monotônico e pode interpretar quedas como resets.

Referência: Documentação oficial

Counter: acumulado de eventos

Um counter Prometheus representa uma contagem cumulativa que cresce entre resets. Um reinício pode fazer o valor recomeçar. É apropriado para total de requisições atendidas, tarefas concluídas ou erros ocorridos. O acumulado não informa, sozinho, a atividade atual: um processo antigo pode ter total grande e pouco tráfego agora.

Exemplo didático: as leituras são 100, 120 e 150. Sem reset, os aumentos observados são 20 e 30. Para obter taxa, falta conhecer o tempo. Somar 100 + 120 + 150 contaria várias vezes eventos já incluídos nos acumulados. O sufixo total costuma comunicar contagem, e count pode aparecer em componentes de distribuições; consulte sempre a definição. O count de uma duração representa quantidade de observações, não segundos.

Referência: Documentação oficial • Documentação oficial

O conceito é geral; os modelos variam

Os conceitos de medição existem em diversas plataformas, mas os modelos têm regras próprias. Gauge e Counter, nesta prática, seguem o vocabulário Prometheus. OpenTelemetry distingue instrumentos, incluindo Counter, UpDownCounter, Gauge e Histogram, e permite agregações com temporalidade delta ou cumulativa. Esses nomes não devem ser interpretados apenas por semelhança.

Exemplo didático: um produtor informa 10 no primeiro intervalo e 20 no segundo. Se são deltas, os intervalos somam 30 eventos. Se são acumulados do mesmo contador sem reset, o aumento entre as leituras é 10. A escolha da operação depende da semântica. No laboratório desta noite, trabalharemos com contagens cumulativas no padrão Prometheus; não será necessário configurar um pipeline OpenTelemetry.

Referência: Documentação oficial • Documentação oficial

Histograma: a distribuição importa

Uma distribuição mostra como muitas observações se espalham. Um histograma organiza frequências por faixas e permite examinar a cauda de valores maiores. No formato clássico Prometheus, os buckets são cumulativos: somá-los como se fossem faixas independentes duplicaria observações. Histograms nativos têm outra representação, preservando o objetivo de descrever a distribuição.

Exemplo didático: 90 chamadas duram 0,1 s e 10 duram 2 s. As durações somam 29 s e a média é 0,29 s. Nenhuma chamada desse exemplo durou exatamente a média, e dez chamadas tiveram experiência bem mais lenta. A média responde uma pergunta útil, mas não descreve sozinha o espalhamento. Os limites das faixas e a forma de cálculo influenciam a precisão das estimativas.

Referência: Documentação oficial • Documentação oficial

p95 não é média nem máximo

Média, percentil e máximo descrevem aspectos diferentes. A média divide a soma das observações por sua quantidade. O p95 é o valor até o qual ficam aproximadamente 95% das observações, conforme o método de cálculo. O máximo corresponde à maior observação. Em conjuntos pequenos e estimativas, convenções e aproximações importam.

Exemplo didático: p95 próximo de 0,8 s não significa que a média seja 0,8 s, que todas as chamadas demoraram esse tempo ou que nenhuma foi mais lenta. No Prometheus, histogramas permitem estimar quantis a partir das distribuições, enquanto summaries podem expor quantis calculados na origem. Não calcule o p95 global fazendo a média dos p95 de servidores: esses resumos não preservam a distribuição necessária. Compare populações e janelas compatíveis.

Referência: Documentação oficial

Labels: recortar a pergunta

Dimensões permitem recortar a pergunta. No Prometheus, labels podem distinguir serviço, ambiente, rota ou status HTTP. Filtrar seleciona as séries que participam; agrupar reorganiza o resultado, preservando determinadas dimensões. Alterar o agrupamento não altera automaticamente a unidade.

Exemplo didático: eventos estão separados em voto com status 200, voto com status 400, página inicial com 200 e página inicial com 302. Podemos somar por operação ou por código usando a mesma população. Uma taxa agrupada por rota continua em requisições por segundo se for agrupada por status. O código HTTP descreve a resposta; não é uma severidade de log. Prefira rotas lógicas, como uma operação de consulta, a URLs completas com identificadores que crescem sem controle.

Referência: Documentação oficial • Documentação oficial

Cardinalidade: quantas séries surgem?

Cardinalidade se relaciona à quantidade de combinações distintas de dimensões. Uma label útil permite perguntas operacionais, mas seus valores precisam de controle. Mais séries podem aumentar o custo de coleta, armazenamento e consulta. Não existe um único limite universal adequado a qualquer ambiente.

Exemplo didático: 3 serviços, 4 rotas e 5 status permitem até 60 combinações básicas. Nem todas precisam existir. Instâncias, ambientes e componentes de histograma podem multiplicar a quantidade. Acrescentar identificadores individuais, como e-mail, request_id ou URL completa com código de usuário, pode provocar crescimento contínuo. Para investigar uma chamada específica, logs e traces podem ser mais apropriados, com seus próprios cuidados. Escolha dimensões pela pergunta que respondem e pelo custo de mantê-las.

Referência: Documentação oficial • Documentação oficial

Taxa: quanto acontece por unidade de tempo

Taxa relaciona ocorrência e tempo. Para contadores Prometheus, rate calcula uma taxa média por segundo a partir das amostras de uma janela. A implementação também considera resets e extrapolação nas bordas. Uma divisão simples ajuda a aprender a unidade, mas não substitui todos os detalhes do cálculo real.

Exemplo didático simplificado: 120 requisições em 60 segundos correspondem a 2 req/s em média. Elas podem ter chegado de forma regular ou em rajadas. Outro exemplo, 3 requisições em um minuto, equivale a 0,05 req/s. Isso não representa uma fração de requisição executada, e sim frequência média de eventos discretos. A janela [5m] não muda a unidade para requisições por cinco minutos: o resultado de rate continua por segundo.

Referência: Documentação oficial

increase e o cuidado com resets

increase estima o aumento de um counter durante a janela; rate apresenta a taxa média por segundo. Para a mesma série e janela, essas leituras se relacionam pela duração do intervalo. Os ajustes do backend podem produzir estimativas fracionárias. Métricas de monitoramento não substituem necessariamente o registro transacional exigido para faturamento ou contabilidade exata.

Exemplo didático: 0,4 req/s por 300 s corresponde aproximadamente a 120 requisições. Em outro desenho, um contador aparece em 90, 100, 5 e 15; uma queda pode indicar reset. Subtrair apenas a primeira da última leitura produziria interpretação errada. Ao combinar instâncias, calcule a taxa por série antes de agregar, para que resets individuais possam ser identificados. Uma queda de counter não significa eventos negativos.

Referência: Documentação oficial

Agregar no espaço e no tempo

Agregar entre séries e agregar ao longo do tempo respondem perguntas diferentes. Em PromQL, sum combina séries no ponto de avaliação; funções de janela podem resumir observações de uma série no tempo. Ao agregar por rota, o resultado conserva esse recorte e combina outras dimensões conforme a expressão.

Exemplo didático: duas APIs com populações distintas recebem 2 req/s e 3 req/s no mesmo instante. O total é 5 req/s. Uma fila com leituras 2, 10 e 3 itens tem média amostral de 5 itens nesse conjunto; isso não significa 5 itens por segundo nem 15 itens processados. Antes de somar, confirme compatibilidade de definição e ausência de duplicação. Duas fontes que representam os mesmos eventos podem produzir total duplicado.

Referência: Documentação oficial • Documentação oficial

A média das porcentagens pode enganar

Uma razão global deve respeitar o peso das populações. Somar numeradores e denominadores compatíveis costuma responder à população total. A média simples de porcentagens só é adequada em situações específicas; não representa automaticamente o resultado geral de grupos com volumes distintos. Um total saudável também pode esconder um grupo com problema.

Exemplo didático: no mesmo intervalo, A tem 1 erro em 10 chamadas, ou 10%; B tem 0 em 990, ou 0%. O total é 1 erro em 1.000 chamadas, ou 0,1%, e não a média simples de 5%. A continua com 10% no próprio recorte. Não misture períodos ou definições diferentes. Se não houve chamadas, a razão requer tratamento explícito; ausência de denominador não comprova zero por cento de erro.

Referência: Documentação oficial • Documentação oficial

Quatro perguntas sobre saúde

Um conjunto inicial de perguntas ajuda a organizar o monitoramento. Tráfego descreve demanda; latência descreve tempo de atendimento; erros descrevem falhas; saturação indica trabalho esperando ou pressão sobre a capacidade. RED organiza taxa, erros e duração de serviços. USE organiza utilização, saturação e erros de recursos. Eles são guias de investigação, não uma lista universal de painéis obrigatórios.

Exemplo didático: 50 req/s, 2% de erros, p95 de 0,5 s e 20 tarefas em espera criam uma visão inicial. Se a fila cresce para 200 tarefas, surge uma pista, ainda sem causa comprovada. CPU elevada não comprova sozinha impacto no usuário, e CPU baixa não garante bom atendimento. Defina o que falha e o que espera em cada componente.

Referência: Documentação oficial • Documentação oficial • Documentação oficial

Zero, ausência e dado antigo

Zero e ausência são evidências diferentes. Uma série com resultado zero pode indicar que não houve novos eventos na janela. No data informa que a consulta não retornou séries para aquela seleção. Uma observação antiga também pode não representar o momento atual. A forma como o backend trata atualidade precisa ser considerada.

Exemplo didático: um laboratório iniciado às 21h não possui necessariamente dados das 20h. Uma rota nunca utilizada pode ainda não ter criado determinada combinação de labels. Em outro caso, a coleta pode ter falhado. Antes de concluir que o serviço está saudável, confira fonte, expressão, filtro, período, aplicação e coleta. Preencher toda ausência com zero pode esconder perda de observação; isso exige uma regra de significado explícita.

Referência: Documentação oficial • Documentação oficial

Período exibido não é janela de cálculo

Período visível, janela de cálculo e atualização têm papéis diferentes. Last 15 minutes escolhe o trecho de história mostrado. A expressão rate com [5m] utiliza uma janela de cinco minutos em cada ponto avaliado. A atualização controla quando a interface solicita resultados novamente. Nenhuma dessas opções deve ser confundida com a frequência original de coleta.

Imagine uma régua de quinze minutos e uma janela móvel de cinco minutos deslizando sobre ela. Cada ponto da taxa usa sua janela correspondente. Se o seletor passar a Last 5 minutes, a expressão com janela fixa [5m] permanece igual, a menos que seja editada. Por isso dois gráficos de períodos diferentes podem compartilhar a mesma janela de taxa e manter a mesma unidade por segundo.

Referência: Documentação oficial • Documentação oficial

Resolução: o gráfico também faz escolhas

A resolução depende do espaçamento dos resultados consultados ou apresentados e das opções do painel e da fonte. Um período longo pode ser mostrado com menos detalhe visual. Isso não permite concluir que a atividade foi constante. Também não permite recuperar observações que nunca foram coletadas. Frequência de coleta, janela e passo de consulta são escolhas distintas.

Exemplo didático: um pico de dez segundos fica mais fácil de perceber numa visão de cinco minutos do que num dia inteiro comprimido. Uma janela menor que o intervalo entre coletas pode não ter amostras suficientes para calcular taxa. Grafana oferece variáveis como $__rate_interval para ajudar em consultas usuais; nesta noite, [5m] fica explícito para facilitar o aprendizado. Diminuir janelas indefinidamente não garante maior precisão.

Referência: Documentação oficial • Documentação oficial

Baseline e limites precisam de contexto

Baseline é uma referência de comportamento observado. Compare horários, populações e condições semelhantes, mas lembre que o histórico pode incluir problemas. Um comportamento repetido não é automaticamente desejável. Capacidade, demanda e experiência do usuário ajudam a interpretar o que mudou. Correlação entre curvas é uma pista, não prova de causalidade.

Exemplo didático: 100 req/s podem ser usuais durante o expediente e incomuns às 3h. Uma campanha, um robô ou uma falha são hipóteses que exigem outras evidências. Limites precisam de definição e contexto; o valor usado para facilitar um exercício não deve ser copiado como política de produção. Os alertas serão aprofundados na Noite 5; aqui importa distinguir observação, referência e decisão.

Referência: Documentação oficial • Documentação oficial

Seis verificações antes de concluir

Antes de interpretar uma curva, formule a pergunta, confirme definição e unidade, identifique séries e dimensões, confira período e janela, classifique o resultado e avalie o limite da evidência. Uma resposta válida pode ser que falta outra medição. Essa disciplina também ajuda a comunicar resultados por capturas.

Exemplo didático: 0,05 na taxa da rota de votação significa 0,05 requisição por segundo em média na janela escolhida. Isso equivale aproximadamente a uma a cada vinte segundos, sem afirmar

regularidade. Não informa usuários únicos. Se o mesmo número representar uma proporção de erro, pode significar 5%, conforme a definição. Ao enviar um print, inclua contexto de fonte, consulta, unidade e período para que outra pessoa consiga compreender a conclusão.

Referência: Documentação oficial • Documentação oficial

Regras e notificações

Recording rules pré-calculam consultas e produzem séries derivadas. Alerting rules avaliam condições. O período for exige persistência antes de firing. Alertmanager administra agrupamento, roteamento, silêncios e inibição.

A janela de uma função rate e o período for cumprem papéis distintos. No Grafana Alerting, regras, políticas e contatos também precisam ser configurados. A existência de um alerta não configura automaticamente SMTP ou integrações externas. A prática de alertas ocorre na Noite 5.

Referência: Documentação oficial • Documentação oficial

Logs, Loki e LogQL

Logs registram eventos com timestamp e contexto. Loki organiza streams por labels e mantém o conteúdo dos logs. LogQL seleciona streams, filtra linhas e extrai campos. Seletor, grafia e parser devem corresponder aos dados existentes.

Um filtro literal por ERROR distingue maiúsculas e minúsculas. O parser JSON exige linhas JSON válidas. Identificadores muito variáveis, como trace_id, pedem cuidado de modelagem e não devem virar indiscriminadamente labels indexados. A prática detalhada de logs ocorre na Noite 3.

Referência: Documentação oficial

Traces, spans e Tempo

Traces relacionam spans de operações. Cada span registra tempo, duração, atributos e relações. Tempo armazena os traces e TraceQL permite selecioná-los. Instrumentação e propagação de contexto precisam estar configuradas nos componentes.

Spans aninhados ou paralelos impedem somar durações para obter o tempo total. Uma operação demorada orienta a hipótese de investigação, mas precisa ser interpretada com dependências, erros e outras evidências. Amostragem e instrumentação incompleta podem limitar o trace observado.

Referência: Documentação oficial • Documentação oficial

Correlação entre os sinais

Correlacionar sinais relaciona população, serviço, versão, ambiente e período. Exemplars podem associar uma observação de métrica a um trace. Derived fields podem extrair um identificador de log e criar um link. Outras integrações podem abrir logs ou métricas a partir do trace.

Essas relações dependem dos dados e da configuração das fontes. Uma correlação temporal orienta hipóteses, sem provar causalidade. Importar um dashboard não instala instrumentação nem habilita automaticamente os recursos entre as fontes.

Referência: Documentação oficial • Documentação oficial

Arquitetura de coleta e consulta

Uma arquitetura de observabilidade separa instrumentação, coleta, armazenamento e consulta. Prometheus coleta e consulta métricas. Loki atende logs, Tempo atende traces e Mimir pode compor uma arquitetura de métricas em escala. Alloy coleta, processa e encaminha telemetria conforme configuração. Grafana consulta as fontes.

A arquitetura é uma referência de papéis. Ela não significa que todos os componentes existam no laboratório local. Protocolos e caminhos variam: scrape e remote write são usados em métricas, enquanto OTLP atende sinais em caminhos compatíveis. Promtail permanece como componente legado e Alloy representa a evolução de coleta.

Referência: Documentação oficial • Documentação oficial

Configuração e fontes de dados

Compose descreve serviços, redes e volumes. Prometheus configura jobs, alvos e regras. Coletores possuem seus pipelines, e o Grafana pode receber conexões por provisioning. Versionamento e versões de imagens explícitas ajudam a reproduzir o ambiente. Persistência e backup têm finalidades diferentes.

O navegador e o servidor Grafana podem usar endereços diferentes. No laboratório local, o aluno acessa localhost:3300, enquanto o Grafana consulta Prometheus pelo nome do serviço na rede Docker. Uma conexão válida não garante a presença de toda métrica. O laboratório preparado para a turma continua sendo a referência operacional.

Referência: Documentação oficial • Documentação oficial

A escolha do painel

O painel deve corresponder à pergunta. Time series mostra evolução, Stat resume um valor, Table compara categorias, Heatmap apresenta distribuição e Logs mostra eventos. Gauge pode representar um percentual em relação a um limite conhecido.

Consulta, período, unidade e redução determinam o significado. Last e Mean não respondem à mesma pergunta. Variáveis filtram o escopo, sem criar novos dados. O dashboard JSON transporta configuração, mas depende das fontes e dados disponíveis no destino. A folha opcional com dez consultas de infraestrutura serve como referência de unidades e tipos.

Referência: Documentação oficial

Caso em sala: interprete antes do clique

Resolva o caso sem usar uma ferramenta. No mesmo intervalo, A tem 1 erro em 10 chamadas e B tem 0 em 990. Um counter passa de 100 para 160 em 60 segundos sem reset. Uma terceira consulta não retorna séries. Calcule a razão de erros global e a taxa média simplificada; depois explique o que a ausência permite concluir.

Gabarito: a razão global é 1/1.000 = 0,1%, mantendo A com 10% no próprio recorte. O aumento de 60 em 60 segundos corresponde a 1 req/s na conta simplificada. No data não comprova zero erros. A implementação real de rate considera mais detalhes do que essa conta de dois pontos. Em todas as respostas, registre unidade, população e limite da conclusão.

Referência: Documentação oficial • Documentação oficial

Síntese antes do intervalo

O bloco teórico termina às 20h20. Após o intervalo de 25 minutos, o exercício guiado começa às 20h45. As quatro perguntas de retomada são: o que o número mede, em qual unidade, para quais séries e em qual intervalo? Elas valem para infraestrutura, aplicações e negócio, independentemente da ferramenta escolhida.

Na execução guiada, a turma abrirá Explore, lerá o contador de Grafana News, calculará taxa e agrupará por rota e por status. Cada pessoa executa os passos no ambiente orientado. No pacote local, os endereços são localhost:3300 para Grafana e localhost:38081 para a aplicação; o professor pode orientar outros endereços. Depois da condução conjunta, os exercícios continuam com criação manual. Arquivos importáveis são apoio opcional.

Referência: Documentação oficial

Dez exemplos opcionais de infraestrutura

Consulte alunos/10_Exemplos_PromQL_Noite_2.html ou a versão Markdown. As consultas de CPU, RAM, disco, rede, load e uptime exigem node_exporter no Prometheus selecionado. O Grafana News não contém essas métricas por padrão. Os exemplos não substituem os exercícios 2.1–2.3.

Execução dos mesmos três exercícios

20h45–20h55: exercício 2.1, seleção da fonte e interpretação do contador; 20h55–21h08: exercício 2.2, período e taxa por rota; 21h08–21h14: exercício 2.3, agrupamento por status HTTP; 21h14–21h20: conferência conjunta dos mesmos resultados e passagem à consolidação.

21h20–21h35: retome 2.1 e finalize a interpretação do contador e dos labels; 21h35–21h55: retome 2.2, gere tráfego e registre rota, unidade e valor; 21h55–22h15: retome 2.3, compare as duas consultas e finalize as capturas e a explicação da janela. São os mesmos três exercícios, sem atividade obrigatória adicional.

Exercício 2.1 — Explorar a métrica da aplicação

Tipo: Consulta no navegador, durante o bloco guiado e a consolidação autônoma.

1. Abra Explore e selecione Prometheus.

2. Execute a métrica bruta.

3. Identifique dois labels disponíveis e registre o significado do valor.

Consulta a utilizar:

```
tns_request_duration_seconds_count
```

Entrega: Consulta executada e interpretação da contagem.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 2.2 — Consultar a taxa por rota

Tipo: Consulta no navegador, durante o bloco guiado e a consolidação autônoma.

1. Execute a consulta de taxa por rota.

2. Gere atividade na aplicação e aguarde a coleta.

3. Escolha Last 5 minutes e compare as séries.

4. Registre a unidade e a rota identificada.

Consulta a utilizar:

```
sum(rate(tns_request_duration_seconds_count[5m])) by (route)
```

Entrega: Taxa interpretada com unidade e agrupamento.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 2.3 — Comparar agrupamentos

Tipo: Consulta e navegação, durante o bloco guiado e a consolidação autônoma.

1. Troque o agrupamento por status_code.

2. Compare o resultado com o agrupamento por rota.

3. Mude o período mostrado e descreva o efeito.

4. Entregue uma frase explicando cada consulta.

Consulta a utilizar:

```
sum(rate(tns_request_duration_seconds_count[5m])) by (status_code)
```

Entrega: Duas consultas comparadas e uma conclusão sustentada.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Noite 3 — Exploração de logs

Objetivo: Conectar o Loki e localizar um erro provocado na aplicação.

Fundamentos para acompanhar a teoria

Métricas e eventos

Uma métrica ajuda a perceber uma mudança.

Um log pode trazer o contexto do evento que ocorreu naquele instante.

Estrutura de um evento de log

Um evento tem horário e conteúdo.

A mensagem e os campos disponíveis ajudam a relacionar o registro ao que a aplicação executou.

Labels e conteúdo no Loki

Os labels selecionam os fluxos de logs.

Os filtros de linha restringem o conteúdo retornado.

Leitura do seletor LogQL

As chaves delimitam o seletor.

O label filename identifica o arquivo usado neste ambiente de demonstração.

Filtro por texto

O operador |= mantém as linhas que contêm o texto indicado.

A seleção do fluxo continua valendo.

Leitura de uma linha

Observe o horário e o conteúdo do evento.

Expanda uma linha para inspecionar os campos disponíveis e relacioná-los à ação no navegador.

Erro, evento e diagnóstico

Um erro conhecido ajuda a validar o caminho de observação.

O registro encontrado precisa corresponder à ação e ao período investigados.

A janela da investigação

Use o horário do evento para reduzir o período.

Compare os logs próximos ao erro com as ações executadas na aplicação.

Live e histórico

Live acompanha os novos eventos.

Para analisar uma linha com calma, pare o acompanhamento e use uma consulta histórica.

Volume de logs e detalhes

A contagem por período mostra a concentração de eventos.

A linha expandida fornece o conteúdo necessário para investigar um evento específico.

Evidência e hipótese

A evidência descreve o que foi observado.

A hipótese propõe uma explicação que ainda precisa ser verificada.

Roteiro de investigação

Defina a pergunta, selecione a fonte e delimite o período.

Leia os eventos e registre a conclusão com a consulta usada.

Exercício 3.1 — Conectar e explorar Loki

Tipo: Configuração no navegador. Tempo previsto: 25 minutos.

1. Adicione a fonte Loki se ela ainda não existir no seu ambiente.

2. Use a URL interna http://loki:3100 no laboratório local.

3. Teste a conexão e execute o seletor do arquivo.

Consulta a utilizar:

```
{filename="/var/log/tns-app.log"}
```

Entrega: Fonte conectada e linhas da aplicação visíveis.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 3.2 — Gerar e localizar um erro

Tipo: Navegação e consulta. Tempo previsto: 25 minutos.

1. Envie um título sem URL na aplicação.

2. Anote o horário da ação.

3. No Explore, filtre as linhas de erro.

4. Abra o evento correspondente e registre a mensagem.

Consulta a utilizar:

```
{filename="/var/log/tns-app.log"} |= "error"
```

Entrega: Consulta, horário e mensagem relacionados à ação.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 3.3 — Explicar a investigação

Tipo: Navegação e interpretação. Tempo previsto: 25 minutos.

1. Compare a consulta do arquivo com a consulta filtrada.

2. Ajuste o período e observe a mudança no resultado.

3. Explique o que é evidência e o que seria uma hipótese.

4. Entregue os passos para um colega reproduzir a busca.

Entrega: Roteiro de investigação com conclusão e evidência.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Noite 4 — Dashboards e anotações

Objetivo: Construir um dashboard que relacione tráfego e erros.

Fundamentos para acompanhar a teoria

Uma pergunta por painel

Defina a pergunta antes de escolher o gráfico.

Nosso painel vai mostrar a taxa de requisições por rota.

Dashboard e painel

Um dashboard reúne painéis relacionados.

Cada painel combina uma consulta e uma forma de apresentar seu resultado.

Consulta e apresentação

A consulta determina os dados retornados.

A visualização deve representar esses dados sem alterar seu significado.

Escolha da visualização

Time series ajuda a observar a evolução no tempo.

Stat destaca um valor resumido. Table permite comparar campos e valores em linhas.

Título e unidade

Um título útil informa o que se mede.

Configure a unidade e use a legenda para identificar as rotas.

Eixos, legenda e período

O eixo temporal situa a observação.

A unidade dá significado ao valor, e a legenda identifica as séries.

Agregação e redução visual

Uma consulta pode retornar uma série temporal.

Uma visualização de valor único precisa escolher como resumir essa série.

Salvamento e organização

Salve o painel e o dashboard com nomes reconhecíveis.

Ajuste o tamanho para ler a legenda e comparar as séries.

Evento pontual e intervalo

Uma anotação pontual marca um instante.

Uma anotação de região representa um evento com início e fim.

Anotações manuais e por consulta

O registro manual descreve um evento informado pelo usuário.

Uma consulta ao Loki pode trazer eventos recorrentes como anotações no gráfico.

Correlação e causalidade

Eventos próximos no tempo podem orientar a investigação.

A proximidade, sozinha, não demonstra que um evento causou o outro.

Dashboard compreensível

Confira o que outra pessoa consegue entender sem a sua explicação.

Títulos, unidades e descrições devem permitir uma leitura consistente.

Revisão antes da demonstração

Um dashboard útil tem pergunta, consulta e apresentação coerentes.

As anotações acrescentam contexto para interpretar o período observado.

Exercício 4.1 — Criar e salvar um dashboard

Tipo: Configuração no navegador. Tempo previsto: 30 minutos.

1. Crie um dashboard da aplicação.

2. Adicione uma visualização Prometheus com a consulta por rota.

3. Configure Time series, título Tráfego por rota, unidade e legenda.

4. Salve com seu nome ou identificação da turma e reabra para conferir.

Consulta a utilizar:

```
sum(rate(tns_request_duration_seconds_count[5m])) by (route)
```

Entrega: Dashboard salvo, legível e com consulta coerente.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 4.2 — Registrar um evento manual

Tipo: Navegação e configuração. Tempo previsto: 20 minutos.

1. Execute uma ação conhecida na aplicação.

2. Adicione uma anotação manual com descrição e horário.

3. Se representar um teste com duração, use uma região.

4. Salve o dashboard.

Entrega: Anotação que descreve uma ação realmente executada.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 4.3 — Exibir erros como anotações

Tipo: Consulta e configuração. Tempo previsto: 25 minutos.

1. Em Settings e Annotations, adicione uma consulta chamada Erros da aplicação.

2. Selecione Loki e use o filtro de erro.

3. Ative a consulta, salve e provoque outro erro na aplicação.

4. Localize a anotação e explique a relação temporal com o gráfico.

Consulta a utilizar:

```
{filename="/var/log/tns-app.log"} |= "error"
```

Entrega: Consulta de anotações ativa e evento encontrado.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Noite 5 — Alertas e projeto final

Objetivo: Disparar um alerta e apresentar o dashboard final.

Fundamentos para acompanhar a teoria

O que dispara um alerta

Uma regra avalia dados e verifica uma condição.

Um ponto de contato define para onde enviar a notificação.

Regra, instância e mensagem

A regra define o cálculo e a condição.

A avaliação pode criar instâncias de alerta. A notificação comunica o estado ao contato configurado.

Consulta da regra

Use Prometheus e a consulta de tráfego por rota.

Confira o resultado antes de definir o limite.

A, B e C

A consulta produz os dados.

A redução gera o valor avaliado por série. A condição compara esse valor com o limite.

Escolha de um limite

O limite precisa fazer sentido para a unidade e para o comportamento observado.

Nesta revisão, o exercício local usa 0,01 req/s como limite didático e exclui route="metrics". Esse valor facilita a demonstração com tráfego manual e não é recomendação de produção.

Avaliação e espera

O intervalo define a frequência de avaliação.

O período pendente define por quanto tempo a condição precisa permanecer verdadeira.

Estados do alerta

Normal: condição não atendida.

Pending: condição atendida, aguardando o período definido.

Firing: condição atendida após a espera.

Sem dados e falha de avaliação

No Data indica ausência de dados utilizáveis.

Error indica uma falha ou timeout na avaliação. O comportamento desses casos precisa ser configurado.

Contato e entrega

O ponto de contato define a integração usada para notificar.

Uma mensagem de teste verifica a comunicação antes de testar o disparo real.

Duas opções de webhook

O tutorial usa uma URL exclusiva do Webhook.site.

O pacote local inclui um receptor com a mesma finalidade de teste.

Alerta e painel relacionado

O vínculo informa qual dashboard e painel ajudam na investigação.

O estado do alerta e suas anotações acrescentam contexto visual.

Referência em vídeo

How to Display Grafana Alerts to Your Dashboards. Use um trecho na aula conforme a orientação do professor; a referência completa serve ao estudo posterior.

Diagnóstico de alertas

Sem dados: confira consulta e fonte.

Não dispara: confira valor e condição.

Sem mensagem: confira contato e acesso ao receptor.

Síntese do ciclo completo

Precisamos confirmar dado, condição, estado e entrega.

O projeto final demonstra o caminho e explica as evidências usadas.

E-mail e integração com APIs externas

E-mail: servidor e destinatário

O servidor SMTP permite ao Grafana enviar mensagens.

O ponto de contato Email define quem recebe. São configurações diferentes.

SMTP: exemplo de configuração

Exemplo ilustrativo no arquivo grafana.ini.

Os valores devem corresponder ao servidor de e-mail disponível. O exemplo não foi aplicado à VM.

```
[smtp]
enabled = true
host = smtp.exemplo.invalid:587
from_address = grafana@exemplo.invalid
startTLS_policy = MandatoryStartTLS
```

No Ubuntu, o arquivo costuma ficar em /etc/grafana/grafana.ini. Em Docker, as variáveis GF_SMTP_* ficam no serviço Grafana. SMTP e destinatários não foram configurados nesta VM; são exemplos de localização dos campos.

Webhook: alerta para uma API externa

O Grafana envia uma requisição HTTP com detalhes do alerta ao endpoint receptor.

O software receptor pode abrir um chamado ou iniciar outro fluxo.

O receptor precisa aceitar a mensagem enviada e sua autenticação. Se o software de chamados exigir outro formato, um adaptador pode transformar a notificação. Uma resposta HTTP 2xx indica aceitação HTTP; confirme o resultado também no software de destino.

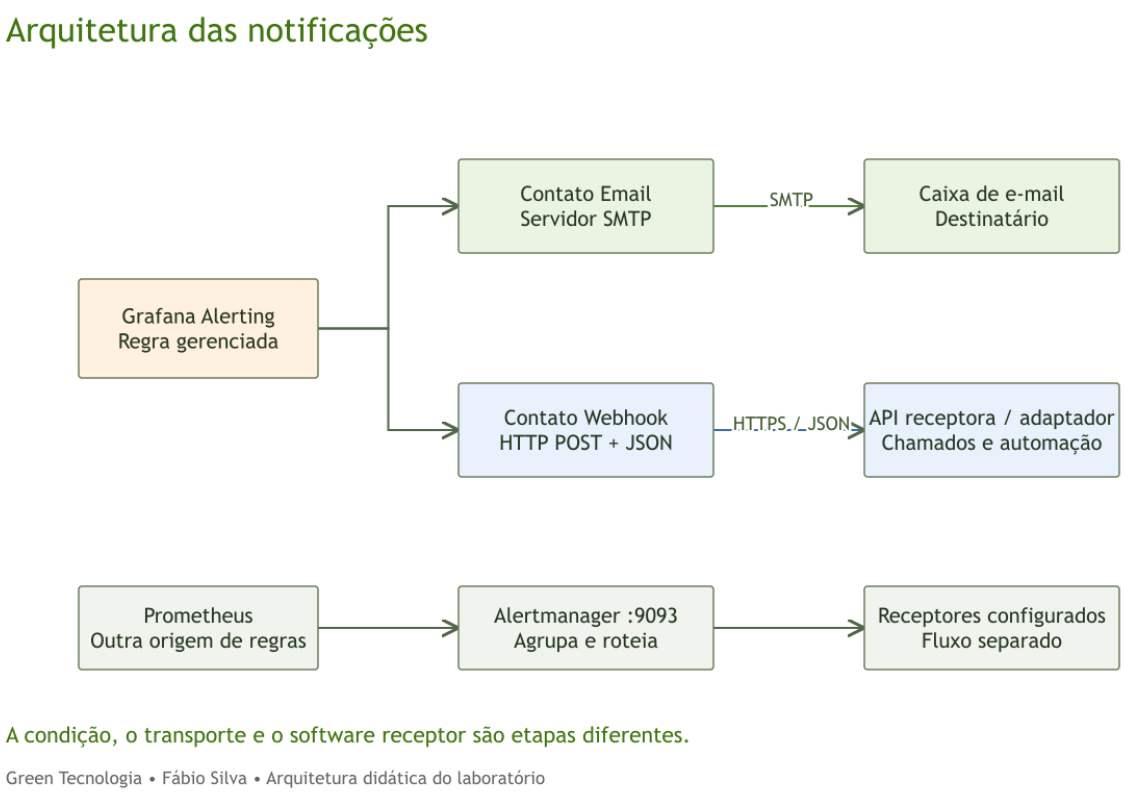
Webhook e API do Grafana

Webhook envia notificações para outro software.

A API do Grafana permite consultar e administrar regras e contatos, conforme permissões e versão.

Exemplos de leitura da API de provisionamento da versão do laboratório: GET /api/v1/provisioning/alert-rules e GET /api/v1/provisioning/contact-points. Exigem autenticação e permissões adequadas. Esses exemplos não foram executados.

Arquitetura das notificações



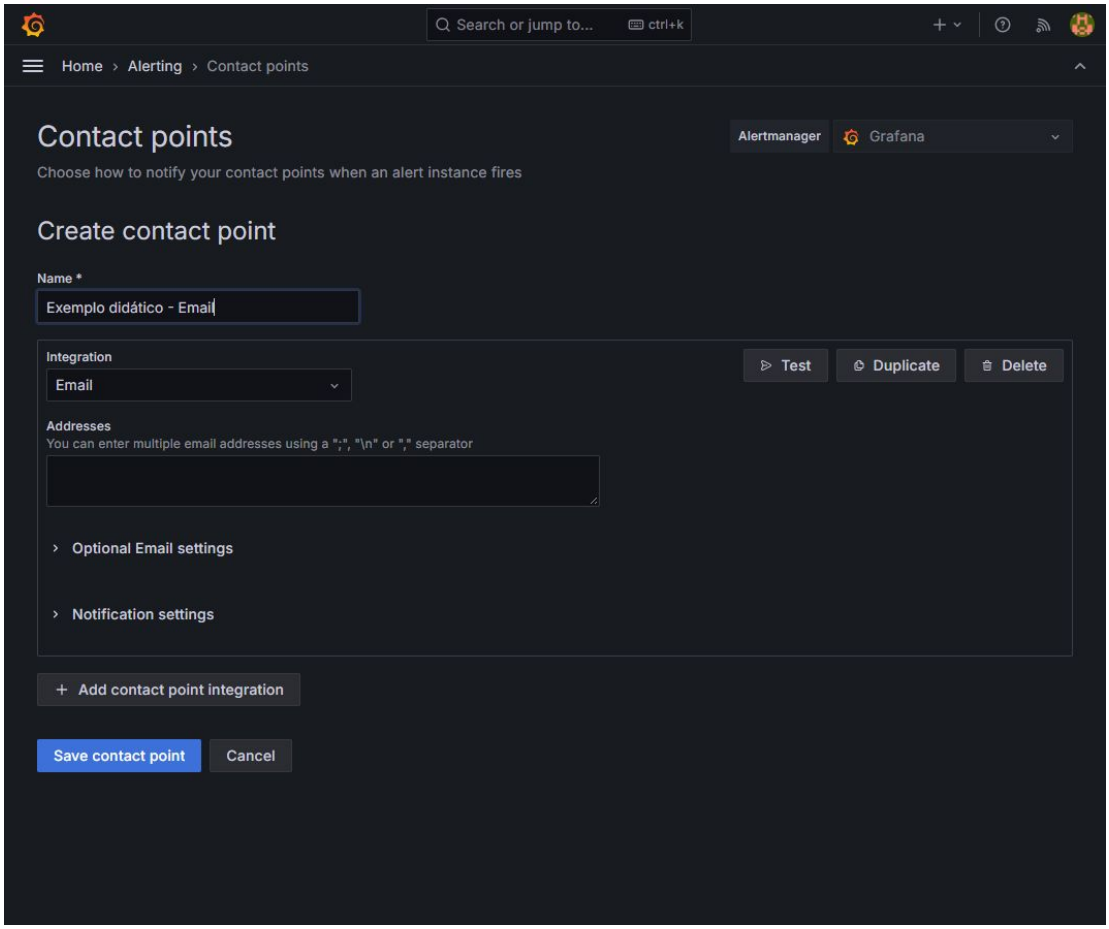
O fluxo Grafana-managed utiliza os contatos do Grafana. As regras do Prometheus podem usar o Alertmanager externo em outro caminho. A fonte Alertmanager no Grafana não configura automaticamente o envio de e-mails.

O que uma integração precisa validar

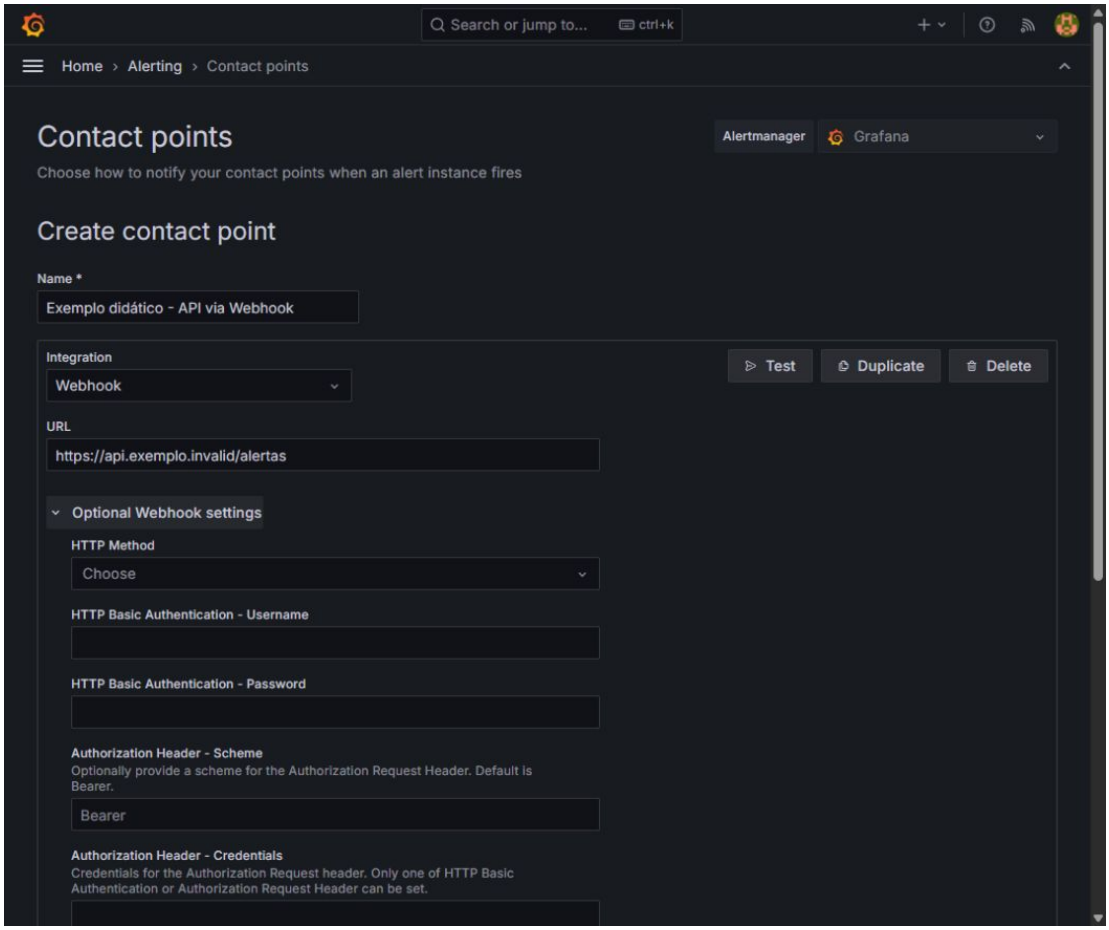
Confira o evento recebido, o estado, a identificação do alerta e o destino.

Teste também a resolução e evite criar chamados duplicados para a mesma ocorrência.

Localização dos campos na interface



Alerting > Contact points > Add contact point > Email > Addresses. O formulário foi apenas demonstrado, sem salvar destinatários ou enviar mensagens.



Selecione Webhook, informe a URL do receptor e confira as opções de método e autenticação. O domínio exemplo.invalid é ilustrativo e não deve ser usado como serviço real.

Exercício 5.1 — Validar o ponto de contato

Tipo: Configuração no navegador. Tempo previsto: 20 minutos.

1. Use o receptor local do pacote: http://localhost:38090 no navegador e http://webhook:8080/ no contato do Grafana.

2. Crie um ponto de contato Webhook no Grafana.

3. Envie uma mensagem de teste e confirme a chegada.

4. Salve o contato.

Entrega: Contato salvo e notificação de teste recebida.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 5.2 — Criar e disparar a regra

Tipo: Consulta e configuração. Tempo previsto: 30 minutos.

1. Crie uma regra Grafana-managed com identificação do aluno.

2. Use a consulta por rota em A, Reduce / Last em B e Threshold acima de 0.01 em C. Exclua a rota metrics na consulta de A.

3. Use avaliação de 10s, pendente de 0s e o contato testado.

4. Salve, gere tráfego e observe o valor e o estado.

5. Confirme uma notificação da regra no receptor.

Consulta a utilizar:

```
sum(rate(tns_request_duration_seconds_count{route!="metrics"}[5m])) by (route)
```

Entrega: Regra avaliada e mensagem de disparo recebida.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Exercício 5.3 — Vincular e apresentar o projeto

Tipo: Navegação e apresentação. Tempo previsto: 25 minutos.

1. Associe a regra ao seu dashboard e ao painel de tráfego.

2. Mostre a consulta, um erro nos logs e as anotações configuradas.

3. Apresente a evidência de recebimento do alerta.

4. Exporte ou registre seu trabalho antes de encerrar o ambiente.

Entrega: Dashboard e regra relacionados, com explicação do fluxo.

Seu registro:

- Consulta, comando ou caminho usado: _____

- Resultado observado: _____

- Interpretação e dúvida: _____

Complemento de instalação Ubuntu

Objetivo e situação deste roteiro

A segunda instalação apresenta o Grafana como serviço do Ubuntu. A referência mais recente consultada é Ubuntu 26.10. Ubuntu 26.04.1 LTS é a alternativa de suporte prolongado. O procedimento completo de automação será criado depois e testado por Fábio em uma VM Azure antes da distribuição. Nesta entrega não há scripts de instalação nativa aprovados.

Os comandos abaixo são exemplos comentados do fluxo documentado de instalação do Grafana OSS. Eles não representam um teste executado na VM Azure. O teste do professor deve registrar versão, arquitetura, comandos, resultados e ajustes. A versão de Grafana a fixar nos scripts será definida nessa validação.

Duas instalações e componentes separados

Docker executa o ambiente do tutorial, com Grafana, Prometheus, Loki e a aplicação. A instalação nativa abaixo instala apenas o Grafana. Ela não instala automaticamente as fontes e a aplicação.

Para comparar os dois Grafanas usando os mesmos dados, uma opção de laboratório é manter a stack de apoio Docker na mesma VM Ubuntu: Grafana Docker em 3300, Grafana nativo em 3000, Prometheus publicado em 39090 e Loki em 33100. No Grafana nativo, as fontes usam http://127.0.0.1:39090 e http://127.0.0.1:33100. Os nomes prometheus:9090 e loki:3100 servem à rede dos containers, não ao processo nativo do host. Esse arranjo também precisa ser validado pelo professor. Uma stack inteiramente nativa exigiria scripts adicionais e não está entregue nesta revisão.

1 Conferir o sistema

```
cat /etc/os-release
uname -m
```

Registre a versão e a arquitetura. A imagem escolhida na VM Azure deve corresponder ao alvo do roteiro que será liberado aos alunos.

2 Preparar APT e adicionar o repositório oficial

```
sudo apt-get update
sudo apt-get install -y apt-transport-https wget gnupg
sudo mkdir -p /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/grafana.asc https://apt.grafana.com/gpg-full.key
sudo chmod 644 /etc/apt/keyrings/grafana.asc
echo "deb [signed-by=/etc/apt/keyrings/grafana.asc] https://apt.grafana.com stable \
main" | sudo tee /etc/apt/sources.list.d/grafana.list
sudo apt-get update
```

O repositório define a origem dos pacotes e a chave permite verificar sua assinatura. O exemplo usa a linha estável. Não misture esse procedimento com a instalação por pacote avulso na mesma máquina de teste.

3 Instalar e iniciar o serviço

```
sudo apt-get install grafana
sudo systemctl daemon-reload
sudo systemctl enable --now grafana-server
systemctl status grafana-server --no-pager
```

O pacote é grafana e o serviço é grafana-server. O parâmetro enable configura inicialização com o sistema, enquanto --now também inicia o serviço imediatamente.

4 Validar localmente

```
wget -qO- http://127.0.0.1:3000/api/health
journalctl -u grafana-server -n 50 --no-pager
```

Na instalação nativa padrão, o primeiro acesso solicita autenticação. O login inicial padrão admin/admin deve ser alterado quando solicitado. Essa experiência é diferente do acesso anônimo do ambiente Docker do tutorial. Se o navegador estiver em outro computador, use o IP da VM e o acesso de rede definido pelo professor; localhost do notebook não acessa o servidor remoto.

Registro do teste Azure a preencher pelo professor

Item	Resultado
Data do teste	A preencher
Imagem Ubuntu e versão	A preencher
Arquitetura	A preencher
Versão Grafana instalada	A preencher
Serviço ativo e reinício validado	A preencher
API health e acesso no navegador	A preencher
Fontes de dados conectadas	A preencher
Resultado após reiniciar a VM	A preencher
Ajustes necessários	A preencher
Liberação dos scripts para a turma	Pendente

Referências: https://ubuntu.com/download/desktop e https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/ .

Preservar o trabalho entre as noites

O pacote não adiciona volume persistente para a base interna do Grafana. Para manter o container com suas configurações entre encontros, use stop e start. Antes de down, exporte os dashboards. Não use down -v como rotina, pois também remove os volumes.

```
docker compose -p green-fundamentals stop
docker compose -p green-fundamentals start
```

Exercício opcional de alerta de erros

Para quem concluir o fluxo principal: consultar sum(rate(tns_request_duration_seconds_count{status_code=~"4..|5.."}[5m])), verificar que há dados após provocar URL vazia e criar uma condição maior que zero. Este complemento não substitui o projeto principal.

Referências

- https://grafana.com/tutorials/grafana-fundamentals/

- https://github.com/grafana/tutorial-environment

- https://prometheus.io/docs/concepts/metric_types/

- https://prometheus.io/docs/prometheus/latest/querying/basics/

- https://grafana.com/docs/loki/latest/query/log_queries/

- https://grafana.com/docs/grafana/latest/visualizations/panels-visualizations/

- https://grafana.com/docs/grafana/latest/alerting/fundamentals/alert-rule-evaluation/

- https://ubuntu.com/download/desktop

- https://grafana.com/docs/grafana/latest/setup-grafana/installation/debian/

Referências dos fundamentos e integrações

https://grafana.com/tutorials/grafana-fundamentals/

https://prometheus.io/docs/introduction/overview/

https://prometheus.io/docs/concepts/metric_types/

https://grafana.com/docs/loki/latest/get-started/overview/

https://grafana.com/docs/loki/latest/query/

https://grafana.com/docs/tempo/latest/traceql/

https://grafana.com/docs/loki/latest/send-data/promtail/

https://grafana.com/docs/alloy/latest/set-up/migrate/from-promtail/

https://grafana.com/docs/grafana/latest/alerting/configure-notifications/manage-contact-points/integrations/configure-email/

https://grafana.com/docs/grafana/latest/alerting/configure-notifications/manage-contact-points/integrations/webhook-notifier/

https://grafana.com/docs/grafana/v11.3/developers/http_api/alerting_provisioning/

Continuação: guia ilustrado dos exercícios

As páginas seguintes detalham as atividades das cinco noites. Na Noite 2, elas serão iniciadas junto com o professor depois do intervalo e retomadas no período de consolidação autônoma. O arquivo Complemento_Visual_V7.pdf permanece separado como referência histórica opcional, sem integrar o cronograma atual de métricas.

Exercícios com passo a passo

Do primeiro clique à conferência do resultado

Fábio Silva • Arquiteto de Soluções • Green Tecnologia

Roteiro pedagógico atualizado em 20/10/2026. As capturas de referência foram obtidas em 19/10/2026 no laboratório local Grafana 12.0.1. Os valores e horários podem variar na sua instalação.

Como usar este guia

Siga os passos numerados e confira o resultado esperado. O bloco “Se não funcionar” ajuda a diagnosticar cada etapa. No HTML, clique na imagem para abrir o original e ampliar.

Durante a aula

Nas Noites 1, 3, 4 e 5: teoria 18h30–19h55; demonstração 19h55–20h20; intervalo 20h20–20h45; exercícios comentados 20h45–21h00; prática individual 21h00–22h15; correção 22h15–22h30.

Na Noite 2, a teoria ocupa 18h30–20h20 (110 minutos), seguida do intervalo de 20h20–20h45. Às 20h45–21h20, professor e alunos executam juntos os três exercícios de métricas no Grafana. Às 21h20–22h15, retome os mesmos procedimentos no seu ambiente e conclua os registros sem acompanhar cada clique do professor. Correção: 22h15–22h30. A Noite 2 não possui um bloco separado de demonstração antes do intervalo.

Noite 1: Preparação e primeiros passos

Noite 2: Métricas e PromQL

Noite 3: Logs e LogQL

Noite 4: Dashboards e anotações

Noite 5: Alertas e entrega do projeto

Pré-requisitos

Pacote completo extraído, Docker Desktop ativo no Windows ou Docker + Compose no Ubuntu. Use o Grafana News do pacote; Node Exporter, e-commerce e Tempo do ambiente do professor não são requisitos destas atividades.

Abas do laboratório: Grafana http://localhost:3300 • aplicação http://localhost:38081 • receptor http://localhost:38090.

Endereços que não podem ser confundidos

1. No navegador: Grafana http://localhost:3300 ; aplicação http://localhost:38081 ; receptor http://localhost:38090 ; Prometheus http://localhost:39090 ; Loki http://localhost:33100.
2. Dentro do Grafana: fonte Prometheus http://prometheus:9090 ; fonte Loki http://loki:3100 ; contato Webhook http://webhook:8080/. Esses nomes são resolvidos dentro da rede Docker.
3. localhost no navegador é o seu computador. localhost dentro de um container é aquele container. Os endereços internos não são URLs para abrir diretamente no navegador.
4. As capturas são do Grafana 12.0.1 do pacote. A VM de demonstração do professor usa outra versão. Se um menu estiver recolhido, abra o ícone Grafana; em telas pequenas, consulte Show more items.
5. Os exercícios são feitos manualmente. Os arquivos em exemplos-importaveis são modelos opcionais, independentes das entregas dos exercícios.

Diagnóstico rápido antes de pedir ajuda

1. Página não abre: confira a porta correta e se os containers estão em execução. No terminal da pasta lab/tutorial-environment, execute docker ps. O serviço do lab deve aparecer em execução.
2. Porta ocupada: não desligue projetos desconhecidos. Verifique se é uma cópia deste próprio laboratório; se for, reutilize a cópia ativa e a pasta correspondente. Alterar portas exige atualizar os endereços nos guias.
3. No data em Prometheus: confirme a fonte, gere atividade na aplicação, espere duas coletas de 5s, confira o relógio e use Last 15 minutes. Execute novamente. Zero é um resultado numérico; No data significa ausência de amostras na consulta.
4. No data em Loki: primeiro teste a fonte; depois confirme a consulta filename, gere uma ação na aplicação e aguarde a coleta. Remova temporariamente o filtro error para separar falha no filtro de ausência de logs.
5. Erro de sintaxe: copie somente a expressão e substitua aspas curvas por aspas retas. PromQL pertence ao Prometheus; LogQL pertence ao Loki.
6. Confirme cada Save: salvar painel, dashboard, fonte, contato e regra são ações distintas. Test no contato e Preview na regra não equivalem a salvar.
7. Registre fonte, consulta, período, horário e mensagem exata do erro. Isso permite explicar o problema e reproduzir sua tentativa.

Noite 1 — Guia de exercícios

Preparação e primeiros passos

Fábio Silva • Arquiteto de Soluções • Green Tecnologia

Roteiro pedagógico atualizado em 20/10/2026. As capturas de referência foram obtidas em 19/10/2026 no laboratório local Grafana 12.0.1. Os valores e horários podem variar na sua instalação.

Como usar este guia

Siga os passos numerados e confira o resultado esperado. O bloco “Se não funcionar” ajuda a diagnosticar cada etapa. No HTML, clique na imagem para abrir o original e ampliar.

Durante a aula

Exercícios comentados: 20h45–21h00. Prática individual: 21h00–22h15 (75 minutos). Correção: 22h15–22h30. Intervalo: 20h20–20h45.

Atividade	Objetivo	Min
1.1	Preparar e iniciar	25
1.2	Publicar e votar	25
1.3	Navegar e validar fonte	25

Pré-requisitos

Pacote completo extraído, Docker Desktop ativo no Windows ou Docker + Compose no Ubuntu. Use o Grafana News do pacote; Node Exporter, e-commerce e Tempo do ambiente do professor não são requisitos destas atividades.

Abas do laboratório: Grafana http://localhost:3300 • aplicação http://localhost:38081 • receptor http://localhost:38090.

NOITE 1 • EXERCÍCIO 1.1 • ETAPA 1

Preparar a pasta e iniciar o laboratório

1. Baixe o ZIP completo da turma e use Extrair tudo. Guarde em uma pasta simples, por exemplo C:/CursoGrafana. Não execute arquivos dentro do ZIP.
2. Abra a pasta que contém iniciar-lab.ps1, iniciar-lab.sh, lab e alunos. Abra o Docker Desktop e aguarde o mecanismo ficar em execução.
3. No Windows, clique com o botão direito dentro dessa pasta e escolha Abrir no Terminal. No PowerShell, execute o comando mostrado abaixo. A pasta atual deve ser a raiz extraída do pacote.
4. Aguarde o término do download e da construção da aplicação. O primeiro início pode levar vários minutos. Só prossiga se o script terminar sem erro e mostrar os três endereços.
5. No Ubuntu com Docker e Compose já instalados, use bash ./iniciar-lab.sh na mesma pasta. Esta é a rota Docker; os scripts de instalação nativa no Ubuntu continuam sendo uma etapa posterior do curso.
6. Mantenha os serviços em execução durante as cinco noites. Os passos seguintes são feitos no navegador, em três abas: Grafana, aplicação e receptor de notificações.

```
# Windows / PowerShell, na raiz extraída do pacote
powershell -NoProfile -ExecutionPolicy Bypass -File .\iniciar-lab.ps1

# Ubuntu com Docker e Compose já instalados
bash ./iniciar-lab.sh
```

Resultado esperado: O script mostra Grafana em localhost:3300, aplicação em localhost:38081 e receptor em localhost:38090.

Se não funcionar: Se docker não for reconhecido, reabra o terminal após instalar o Docker Desktop. Se houver erro de conexão com o daemon, abra o Docker Desktop. Se uma porta estiver ocupada, não encerre outros projetos: consulte a seção de diagnóstico deste guia.

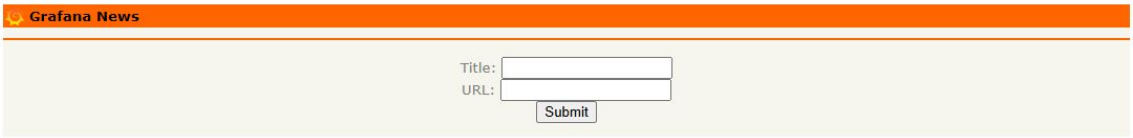
NOITE 1 • EXERCÍCIO 1.1 • ETAPA 2

Abrir a aplicação e o Grafana

1. Abra http://localhost:38081 em uma aba. Você deve ver Grafana News e os campos Title e URL.
2. Abra uma segunda aba e digite http://localhost:3300. O laboratório Docker do pacote usa acesso anônimo; não é necessário digitar usuário ou senha.
3. Abra uma terceira aba em http://localhost:38090. Ela exibirá o receptor local de mensagens, usado na Noite 5.
4. Adicione os três endereços aos favoritos e mantenha as abas abertas. Não use os endereços da VM do professor nos exercícios locais.
5. Anote a data, o horário e o fuso do seu computador. Eles ajudam a localizar suas ações nos gráficos e logs.

Resultado esperado: Aplicação, Grafana e receptor abrem no navegador.

Se não funcionar: Uma lista vazia no Grafana News é normal. Página de login no Grafana pode indicar outra instalação: confira a porta 3300. localhost refere-se ao computador onde o navegador está aberto.



Tela real • 01-news-vazio.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 1 • EXERCÍCIO 1.2 • ETAPA 3

Preencher o link a publicar

1. Volte à aba Grafana News. Em Title, digite Exercício Green - Noite 1 - seguido do seu nome.
2. Em URL, digite https://grafana.com/tutorials/grafana-fundamentals/. Inclua https://.
3. Confira os dois campos antes de continuar. Faça uma captura da tela preenchida, se desejar registrar o antes.
4. Clique uma vez em Submit. Aguarde a resposta; os campos ficam vazios e seu título aparece na lista.

Resultado esperado: O link publicado aparece abaixo do título Grafana News.

Se não funcionar: Se aparecer empty url, o campo URL estava vazio. Use Voltar do navegador, preencha-o e envie novamente. Evite clicar várias vezes em Submit.



Tela real • 02-news-preenchido.jpg • Abra a imagem original no guia HTML para ampliar.

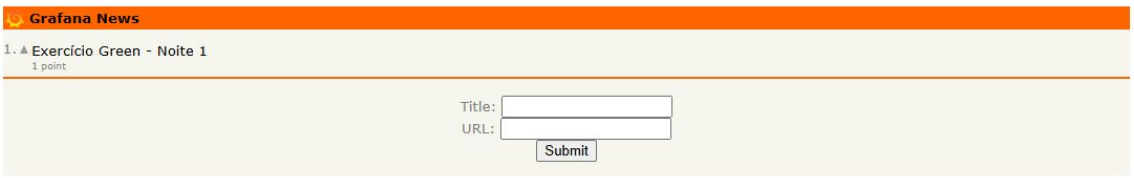
NOITE 1 • EXERCÍCIO 1.2 • ETAPA 4

Votar e gerar atividade conhecida

1. Na linha do seu link, clique no pequeno triângulo à esquerda do título. Não clique no próprio título: ele abre o site publicado.
2. Confira a contagem abaixo do link. Ela deve passar de 0 points para 1 point; o valor pode ser diferente se você já votou antes.
3. Atualize a página pelo botão Recarregar do navegador. Repita a atualização algumas vezes para produzir requisições GET.
4. Registre o horário de publicação e do voto. Na Noite 2 você verá métricas; na Noite 3 verá os registros dessa aplicação.

Resultado esperado: O título permanece na lista e a contagem de votos aumenta.

Se não funcionar: Se outra página abrir, volte à aplicação e use o triângulo. Um número diferente do print não é falha: compare a mudança após sua ação.



Tela real • 03-news-voto.jpg • Abra a imagem original no guia HTML para ampliar.

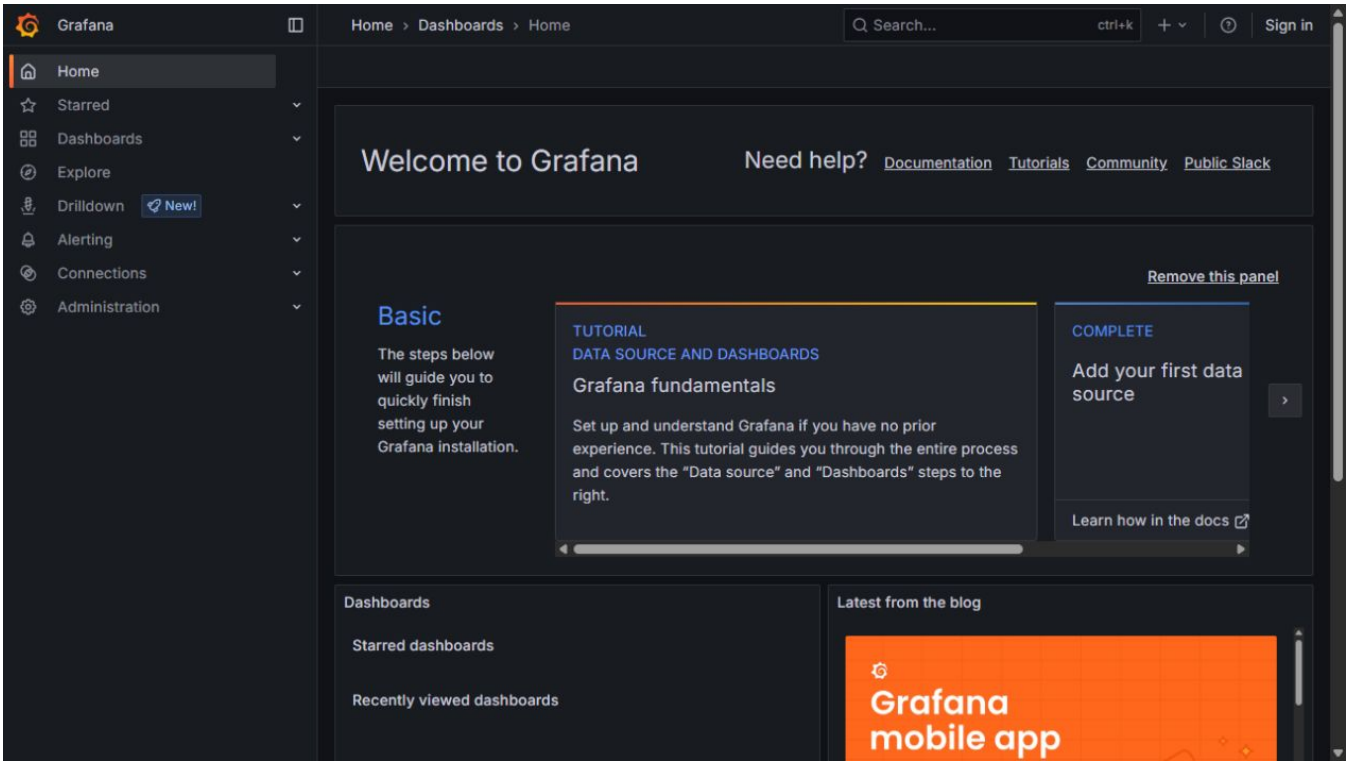
NOITE 1 • EXERCÍCIO 1.3 • ETAPA 5

Reconhecer a navegação do Grafana

1. Volte à aba http://localhost:3300. Clique no ícone do Grafana, no canto superior esquerdo, se o menu estiver fechado.
2. Localize Explore: é onde você executará consultas temporárias sem precisar salvar um painel.
3. Localize Dashboards: reúne painéis salvos para acompanhar os dados.
4. Localize Connections. Expanda pela seta e clique em Data sources para ver as fontes conectadas.
5. Se o menu aparecer como uma janela sobre a página, escolha o destino e feche-o depois para ganhar espaço. Em telas menores, botões extras podem estar em Show more items.

Resultado esperado: Você consegue encontrar Explore, Dashboards e Connections > Data sources.

Se não funcionar: A tela inicial e os avisos de novidades podem variar. Neste guia, use os nomes dos botões, não apenas a posição no print.



Tela real • 04-grafana-inicio.jpg • Abra a imagem original no guia HTML para ampliar.

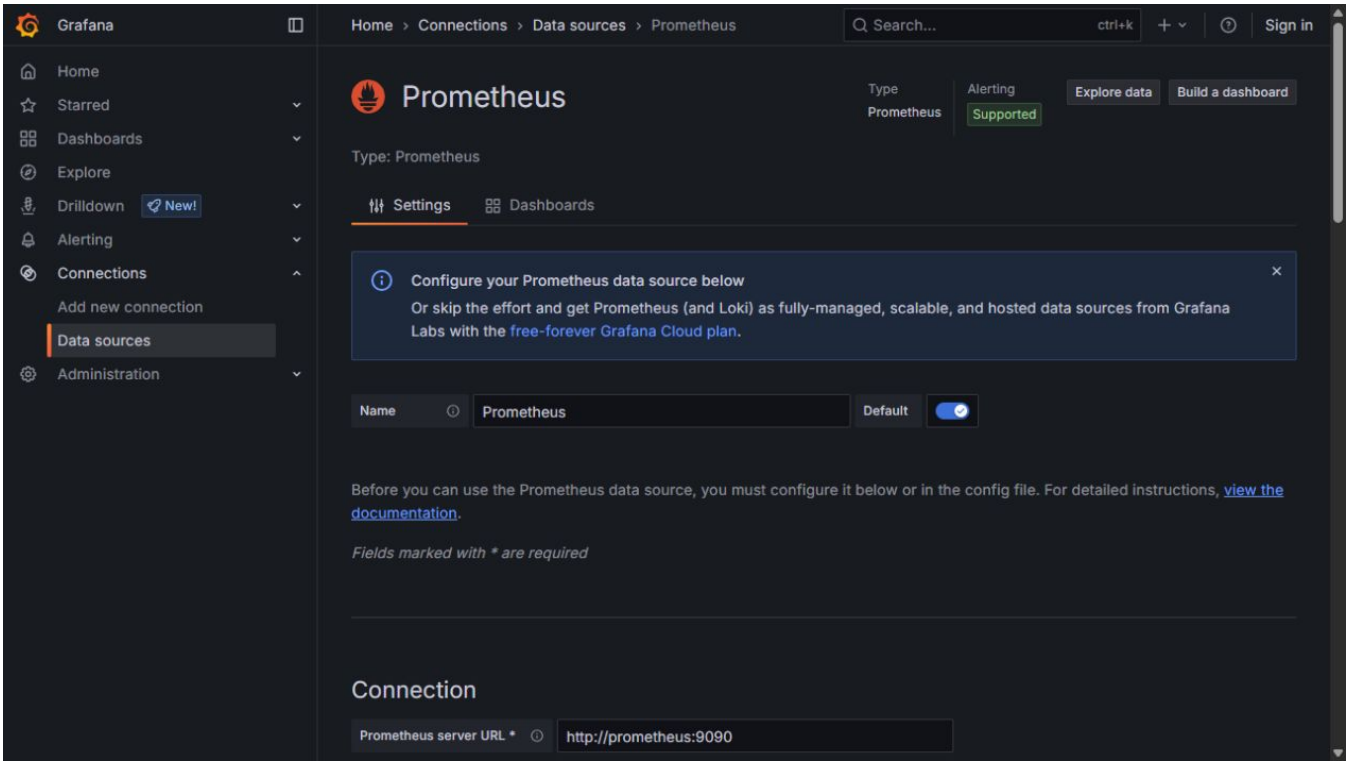
NOITE 1 • EXERCÍCIO 1.3 • ETAPA 6

Abrir Prometheus e conferir sua URL

1. Em Connections > Data sources, localize a fonte Prometheus. Ela já vem criada no pacote; não adicione outra.
2. Clique no nome Prometheus. Na aba Settings, confirme Name = Prometheus.
3. Na seção Connection, confira Prometheus server URL = http://prometheus:9090.
4. Deixe a URL interna como está. O Grafana faz essa conexão dentro da rede Docker; localhost:39090 é o endereço externo do Prometheus para o navegador.
5. Role até o fim da página e clique em Save & test. Aguarde a mensagem de confirmação.

Resultado esperado: A configuração usa http://prometheus:9090 e a fonte está marcada como padrão.

Se não funcionar: Se não existir Prometheus, confira se iniciou o pacote completo e a porta correta do Grafana. Não substitua a URL por localhost:9090: no container, localhost aponta para o próprio Grafana.



Tela real • 06-prometheus-url.jpg • Abra a imagem original no guia HTML para ampliar.

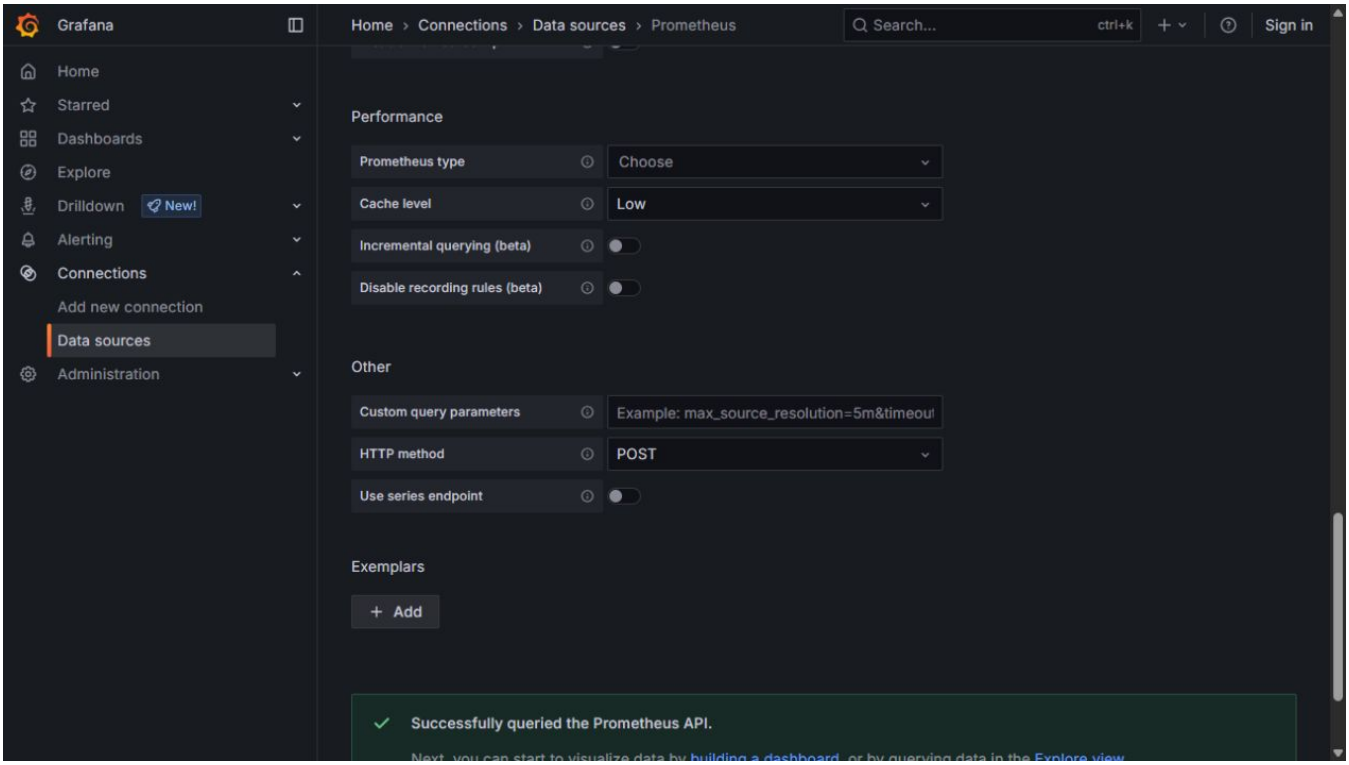
NOITE 1 • EXERCÍCIO 1.3 • ETAPA 7

Validar a fonte e registrar a entrega

1. Leia a mensagem Successfully queried the Prometheus API. Ela confirma a comunicação do Grafana com a API da fonte.
2. Faça uma captura com a confirmação e registre a URL interna da fonte.
3. Escreva uma frase sobre cada recurso: Explore consulta; Dashboard apresenta painéis salvos; Data source configura a conexão.
4. Entregue as evidências da noite: aplicação acessível, link com voto e teste da fonte.
5. Mantenha o ambiente iniciado para continuar na Noite 2. Não remova containers nem volumes.

Resultado esperado: Você tem três evidências e sabe explicar o papel dos recursos.

Se não funcionar: Save & test bem-sucedido não garante que qualquer métrica exista. A consulta à aplicação será validada na próxima noite.



Tela real • 07-prometheus-teste.jpg • Abra a imagem original no guia HTML para ampliar.

Noite 1 — conferência da entrega

1.1 — Preparar e iniciar (25 minutos)

Concluído: ____ Evidência salva em: _____

1.2 — Publicar e votar (25 minutos)

Concluído: ____ Evidência salva em: _____

1.3 — Navegar e validar fonte (25 minutos)

Concluído: ____ Evidência salva em: _____

Registre para a correção

Fonte / consulta / caminho utilizado: _____

Horário, período e resultado observado: _____

O que esse resultado significa: _____

Se houve falha: mensagem exata e verificações já feitas: _____

Noite 2 — Guia de exercícios

Métricas e PromQL

Fábio Silva • Arquiteto de Soluções • Green Tecnologia

Roteiro pedagógico atualizado em 20/10/2026. As capturas de referência foram obtidas em 19/10/2026 no laboratório local Grafana 12.0.1. Os valores e horários podem variar na sua instalação.

Como usar este guia

Siga os passos numerados e confira o resultado esperado. O bloco “Se não funcionar” ajuda a diagnosticar cada etapa. No HTML, clique na imagem para abrir o original e ampliar.

Durante a aula

Na Noite 2, a teoria ocupa 18h30–20h20 (110 minutos), seguida do intervalo de 20h20–20h45. Às 20h45–21h20, professor e alunos executam juntos os três exercícios de métricas no Grafana. Às 21h20–22h15, retome os mesmos procedimentos no seu ambiente e conclua os registros sem acompanhar cada clique do professor. Correção: 22h15–22h30. A Noite 2 não possui um bloco separado de demonstração antes do intervalo.

Exercício guiado — 35 minutos

20h45–20h55: exercício 2.1, seleção da fonte e interpretação do contador; 20h55–21h08: exercício 2.2, período e taxa por rota; 21h08–21h14: exercício 2.3, agrupamento por status HTTP; 21h14–21h20: conferência conjunta dos mesmos resultados e passagem à consolidação.

Consolidação autônoma — 55 minutos

21h20–21h35: retome 2.1 e finalize a interpretação do contador e dos labels; 21h35–21h55: retome 2.2, gere tráfego e registre rota, unidade e valor; 21h55–22h15: retome 2.3, compare as duas consultas e finalize as capturas e a explicação da janela. São os mesmos três exercícios, sem atividade obrigatória adicional.

Atividade	Objetivo	Min
2.1	Consolidação: contador e labels	15
2.2	Consolidação: taxa por rota	20
2.3	Consolidação: agrupamento e período	20

Pré-requisitos

Pacote completo extraído, Docker Desktop ativo no Windows ou Docker + Compose no Ubuntu. Use o Grafana News do pacote; Node Exporter, e-commerce e Tempo do ambiente do professor não são requisitos destas atividades.

Abas do laboratório: Grafana http://localhost:3300 • aplicação http://localhost:38081 • receptor http://localhost:38090.

Conclua a Noite 1 e mantenha a aplicação em execução. Se o professor utilizar seu próprio endereço, siga o endereço informado em sala. As capturas continuam sendo referências do laboratório local.

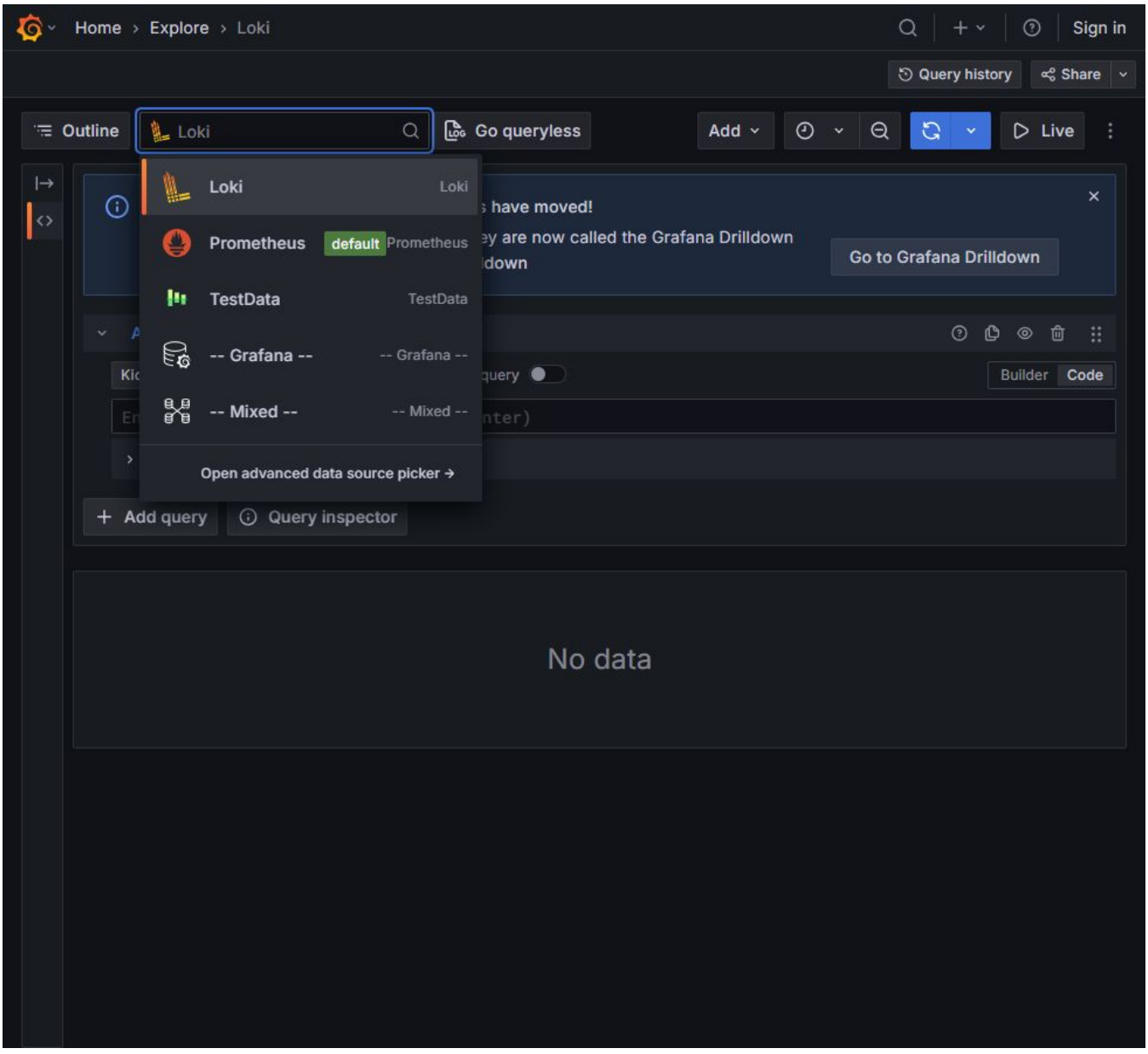
NOITE 2 • EXERCÍCIO 2.1 • ETAPA 1

Selecionar a fonte no Explore

1. Abra http://localhost:3300 e, no menu principal, clique em Explore.
2. Clique no seletor de fonte no topo da área de consulta. Ele pode mostrar apenas o ícone e uma seta em uma tela estreita.
3. Na lista, clique em Prometheus. Não selecione TestData nem Loki.
4. No editor da consulta A, marque Code à direita de Builder. Se houver uma consulta anterior, clique dentro do editor e use Ctrl+A para substituí-la.
5. Se um aviso azul sobre Drilldown ocupar a tela, feche-o pelo X do próprio aviso. O exercício continua no Explore.

Resultado esperado: A consulta A indica Prometheus e o modo Code está ativo.

Se não funcionar: Se o editor mencionar Loki query, troque a fonte para Prometheus. PromQL não é executado na fonte Loki.



Tela real • 49-explore-selecionar-fonte.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 2 • EXERCÍCIO 2.1 • ETAPA 2

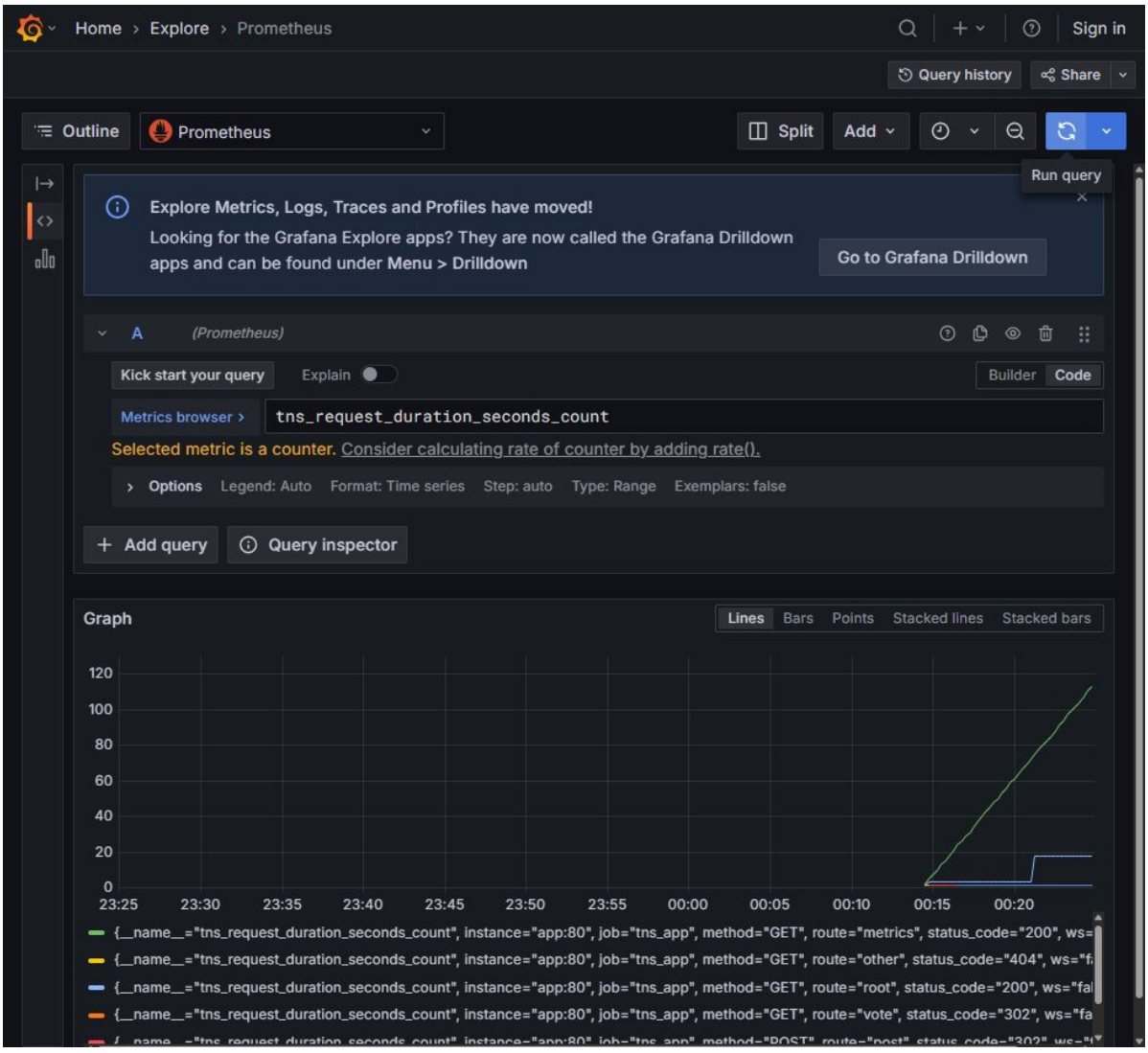
Executar e interpretar o contador

1. Cole a consulta abaixo no editor A. Use aspas retas e copie somente a consulta, sem as marcas de bloco de código.
2. Clique em Run query, no topo à direita, ou use Shift+Enter. Aguarde o gráfico carregar.
3. Abra o seletor de período e use Last 15 minutes. Isso é suficiente para um laboratório recém-iniciado.
4. Leia a legenda abaixo do gráfico. Identifique route e status_code em pelo menos duas séries.
5. Anote que o valor representa uma contagem de requisições por combinação de labels, não CPU, tempo em segundos ou req/s.
6. Atualize a aplicação em outra aba, aguarde aproximadamente 10 segundos e execute Run query novamente. O contador pode voltar a zero se o processo reiniciar.

```
tns_request_duration_seconds_count
```

Resultado esperado: O gráfico mostra séries do contador, com labels como route e status_code.

Se não funcionar: Em No data, confirme a fonte Prometheus, abra a aplicação, atualize-a, aguarde duas coletas de 5s e execute novamente. Um gráfico vazio à esquerda do horário de início do lab é esperado.



Tela real • 08-explore-contador.jpg • Abra a imagem original no guia HTML para ampliar.

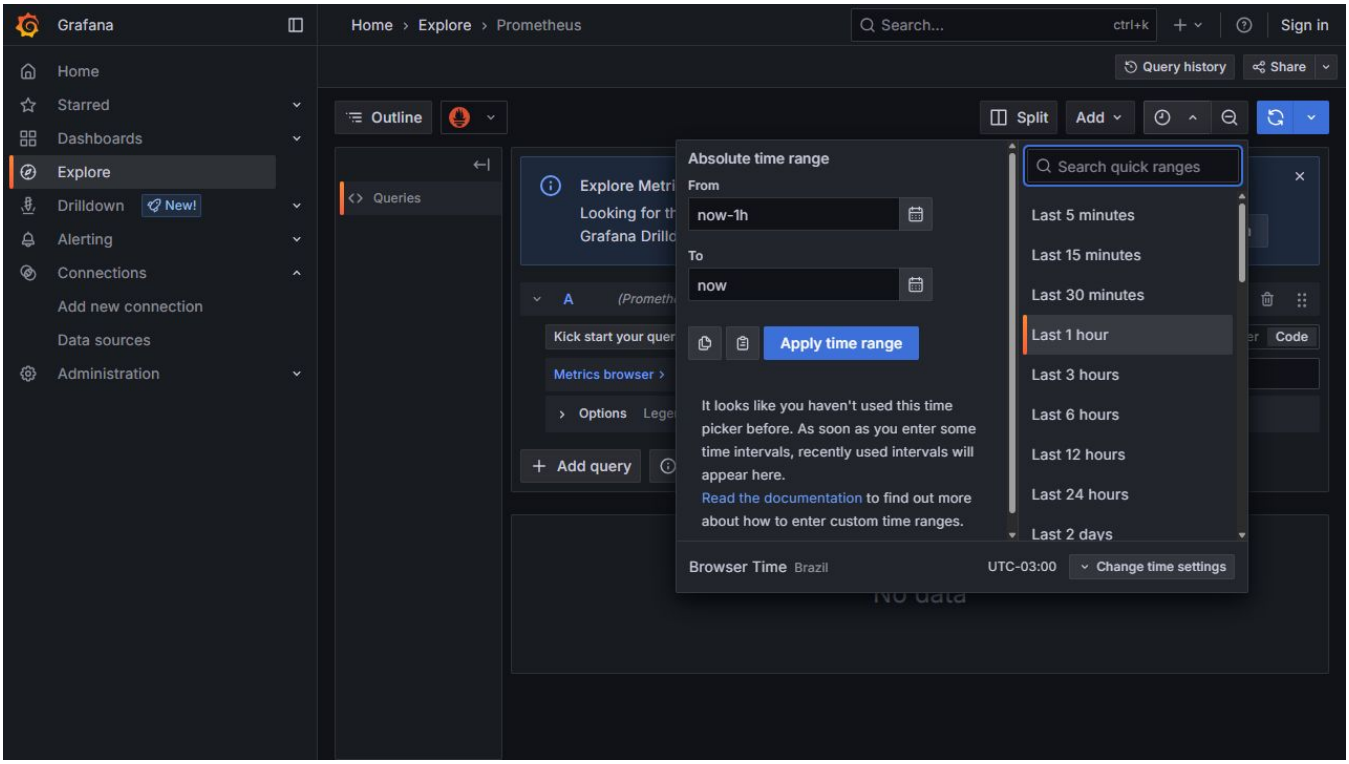
NOITE 2 • EXERCÍCIO 2.2 • ETAPA 3

Mudar o período pela tela

1. No Explore, clique no botão de período ao lado de Run query. Ele mostra algo como Last 1 hour ou Last 15 minutes.
2. Na lista de períodos relativos, clique no texto Last 5 minutes. O menu fecha ao aplicar a seleção.
3. Confirme que o botão passou a mostrar Last 5 minutes.
4. Se o período aparecer apenas como um relógio, expanda Show more items ou feche o menu lateral.
5. Não preencha datas absolutas neste exercício. Esse período acompanha o relógio do computador e mostra a janela mais recente.

Resultado esperado: O período visível é Last 5 minutes.

Se não funcionar: Se você clicou e nada mudou, clique no texto do intervalo, não na borda da caixa. Um período absoluto não avança sozinho.



Tela real • 09-periodo.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 2 • EXERCÍCIO 2.2 • ETAPA 4

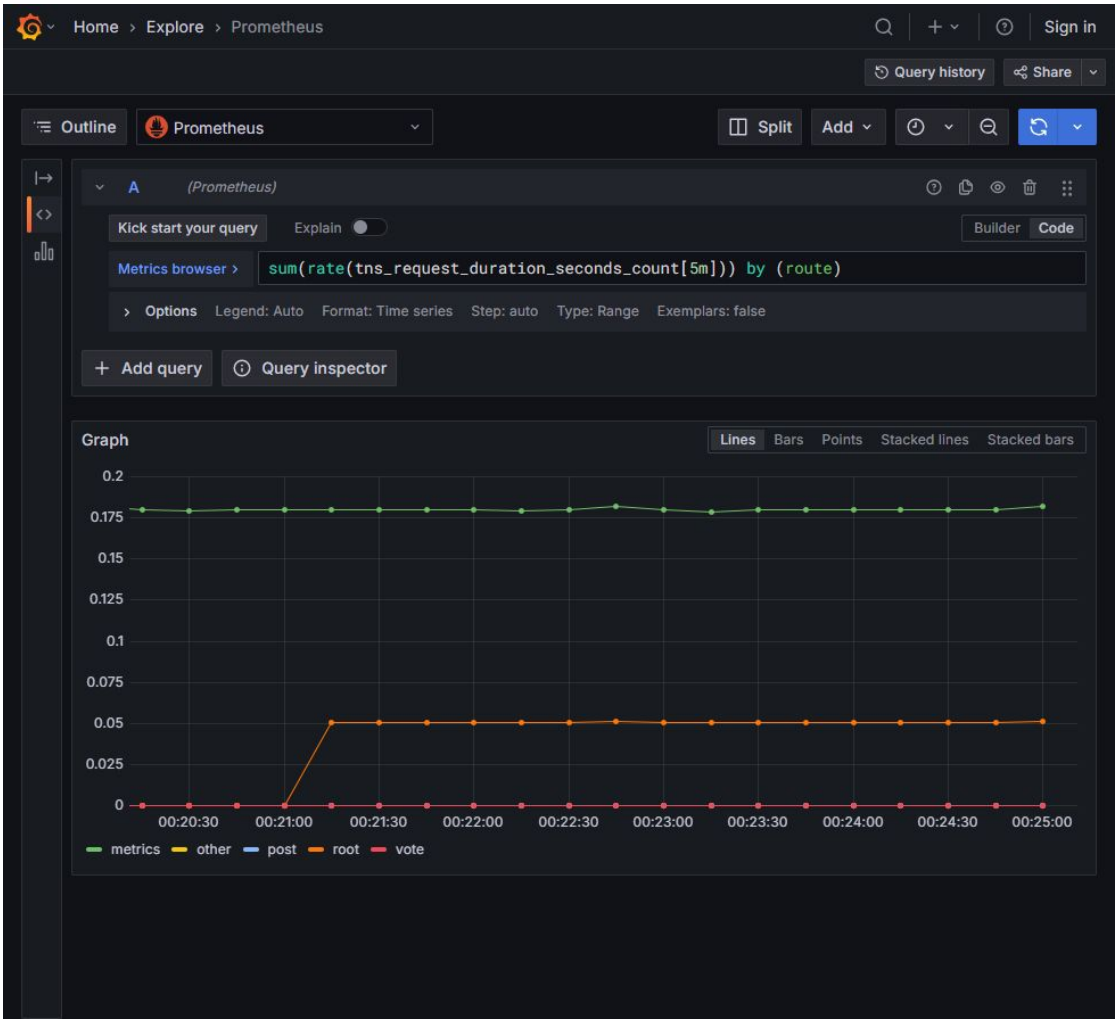
Calcular a taxa por rota

1. Substitua a consulta A pela expressão abaixo e clique em Run query.
2. Volte à aplicação e atualize a página 10 a 15 vezes. Espere cerca de 10 segundos para que o Prometheus colete novas amostras.
3. Volte ao Explore e clique em Run query novamente. Na legenda, localize root, post e vote, se estiverem presentes.
4. Leia a taxa na curva. A unidade é requisições por segundo. A função rate usa as amostras dos últimos 5 minutos.
5. Observe também a rota metrics: ela recebe as requisições de coleta do próprio Prometheus. Esse tráfego não significa que um aluno visitou a aplicação.
6. Anote uma rota, um valor aproximado e o período. Um valor 0,05 req/s equivale, em média, a uma requisição a cada 20 segundos.

```
sum(rate(tns_request_duration_seconds_count[5m])) by (route)
```

Resultado esperado: Você identifica taxas por rota e diferencia o scrape das ações do usuário.

Se não funcionar: Taxas pequenas são normais quando você clica poucas vezes. Não espere um número igual ao print. Zero com uma série presente é diferente de No data.



Tela real • 10-taxa-rota.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 2 • EXERCÍCIO 2.3 • ETAPA 5

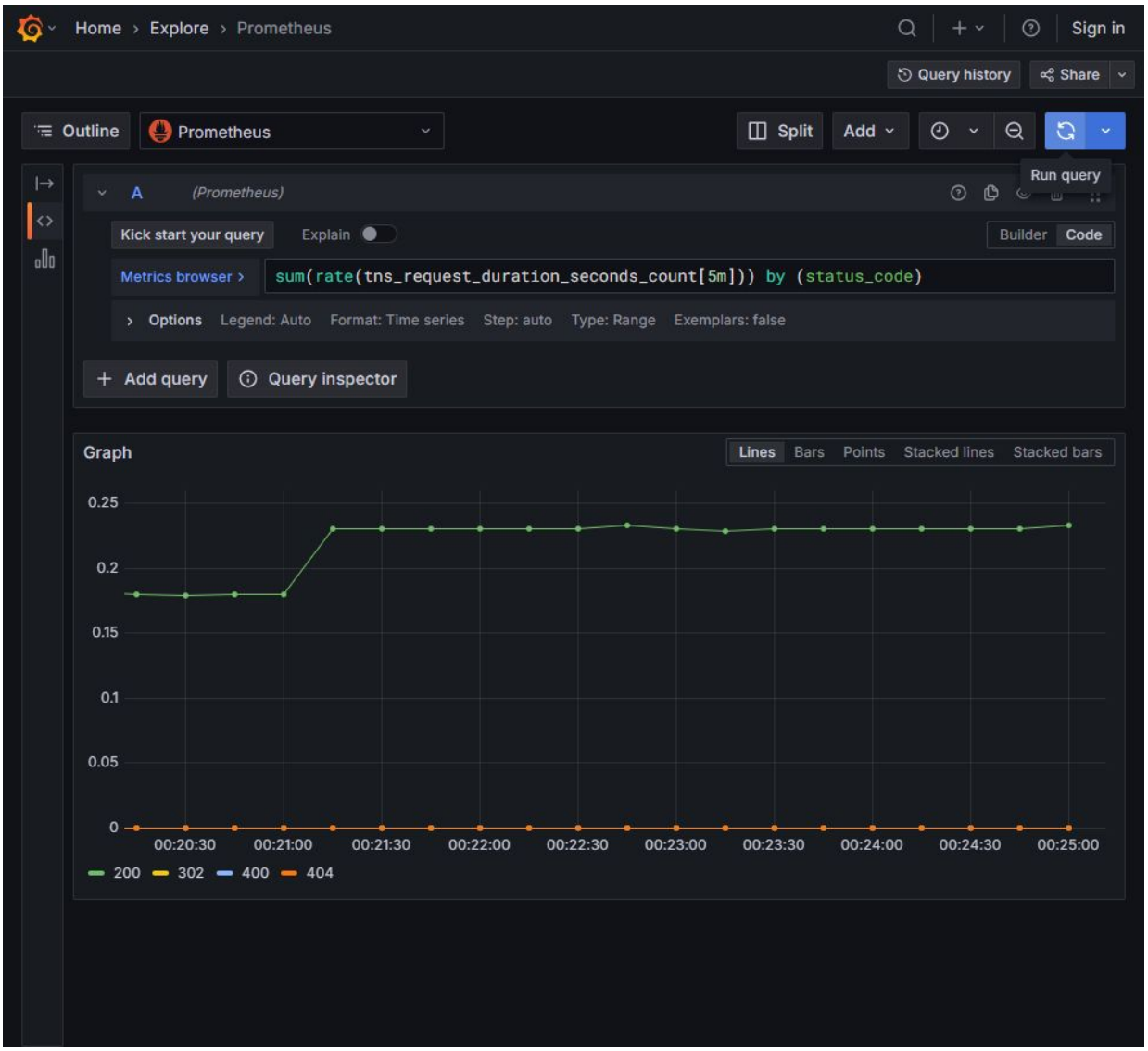
Comparar por status HTTP

1. Substitua by (route) por by (status_code), usando a consulta completa abaixo.
2. Execute Run query e leia a legenda: por exemplo 200, 302, 400 ou 404, conforme as ações já feitas.
3. Compare: route agrupa pelo caminho lógico; status_code agrupa pelo código de resposta. A unidade continua req/s.
4. Troque Last 5 minutes por Last 15 minutes e depois volte. A janela interna [5m] continua a mesma.
5. Registre as duas consultas e explique, em suas palavras, a diferença entre janela do rate e período exibido.
6. Entregue uma captura por agrupamento. A ausência de 400 é normal antes do erro que será produzido na Noite 3.

```
sum(rate(tns_request_duration_seconds_count[5m])) by (status_code)
```

Resultado esperado: Você compara as mesmas requisições usando dimensões diferentes.

Se não funcionar: Uma série nova aparece somente depois de a aplicação produzir aquela combinação de labels. Não invente erro para fazer a legenda coincidir com o print.



Tela real • 11-taxa-status.jpg • Abra a imagem original no guia HTML para ampliar.

Noite 2 — conferência da entrega

2.1 — Consolidação: contador e labels (15 minutos)

Concluído: ____ Evidência salva em: _____

2.2 — Consolidação: taxa por rota (20 minutos)

Concluído: ____ Evidência salva em: _____

2.3 — Consolidação: agrupamento e período (20 minutos)

Concluído: ____ Evidência salva em: _____

Registre para a correção

Fonte / consulta / caminho utilizado: _____

Horário, período e resultado observado: _____

O que esse resultado significa: _____

Se houve falha: mensagem exata e verificações já feitas: _____

Noite 3 — Guia de exercícios

Logs e LogQL

Fábio Silva • Arquiteto de Soluções • Green Tecnologia

Roteiro pedagógico atualizado em 20/10/2026. As capturas de referência foram obtidas em 19/10/2026 no laboratório local Grafana 12.0.1. Os valores e horários podem variar na sua instalação.

Como usar este guia

Siga os passos numerados e confira o resultado esperado. O bloco “Se não funcionar” ajuda a diagnosticar cada etapa. No HTML, clique na imagem para abrir o original e ampliar.

Durante a aula

Exercícios comentados: 20h45–21h00. Prática individual: 21h00–22h15 (75 minutos). Correção: 22h15–22h30. Intervalo: 20h20–20h45.

Atividade	Objetivo	Min
3.1	Conectar Loki	25
3.2	Gerar e localizar erro	25
3.3	Explicar investigação	25

Pré-requisitos

Pacote completo extraído, Docker Desktop ativo no Windows ou Docker + Compose no Ubuntu. Use o Grafana News do pacote; Node Exporter, e-commerce e Tempo do ambiente do professor não são requisitos destas atividades.

Abas do laboratório: Grafana http://localhost:3300 • aplicação http://localhost:38081 • receptor http://localhost:38090.

Mantenha o laboratório iniciado e os acessos da Noite 1 funcionando.

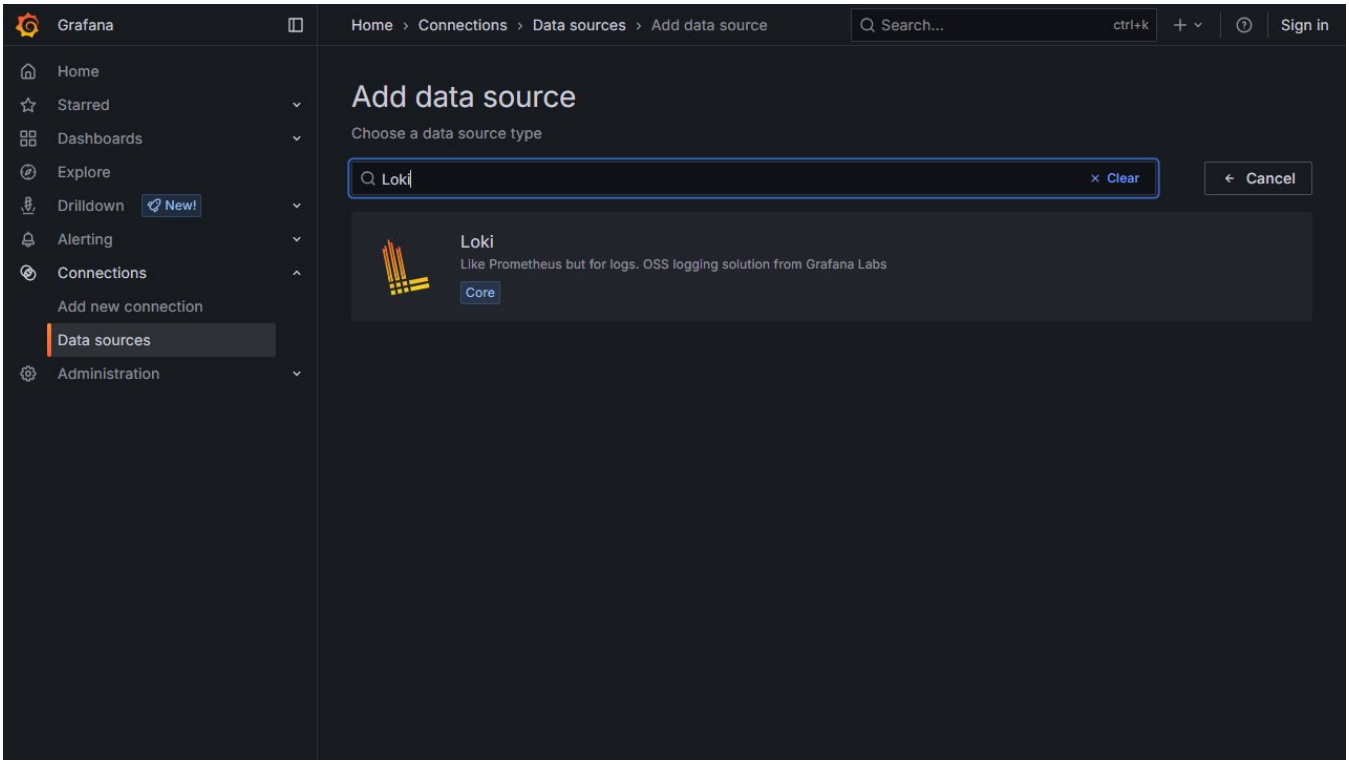
NOITE 3 • EXERCÍCIO 3.1 • ETAPA 1

Adicionar Loki sem criar duplicatas

1. Abra Connections > Data sources. Se Loki já existir, clique nela e siga para a configuração da próxima página.
2. Se ainda não existir, clique em Add new data source, no canto superior da lista.
3. Na caixa de busca, digite Loki. Na opção de mesmo nome, clique em Add new data source Loki.
4. Aguarde a página de configuração carregar. Não escolha Tempo ou Prometheus.
5. Não crie fontes loki-1, loki-2 a cada aula: reutilize a conexão correta.

Resultado esperado: A página de configuração de uma única fonte Loki está aberta.

Se não funcionar: Se a busca não apresentar Loki, limpe o termo e confira que está em Data sources, não na busca global do Grafana.



Tela real • 12-adicionar-loki.jpg • Abra a imagem original no guia HTML para ampliar.

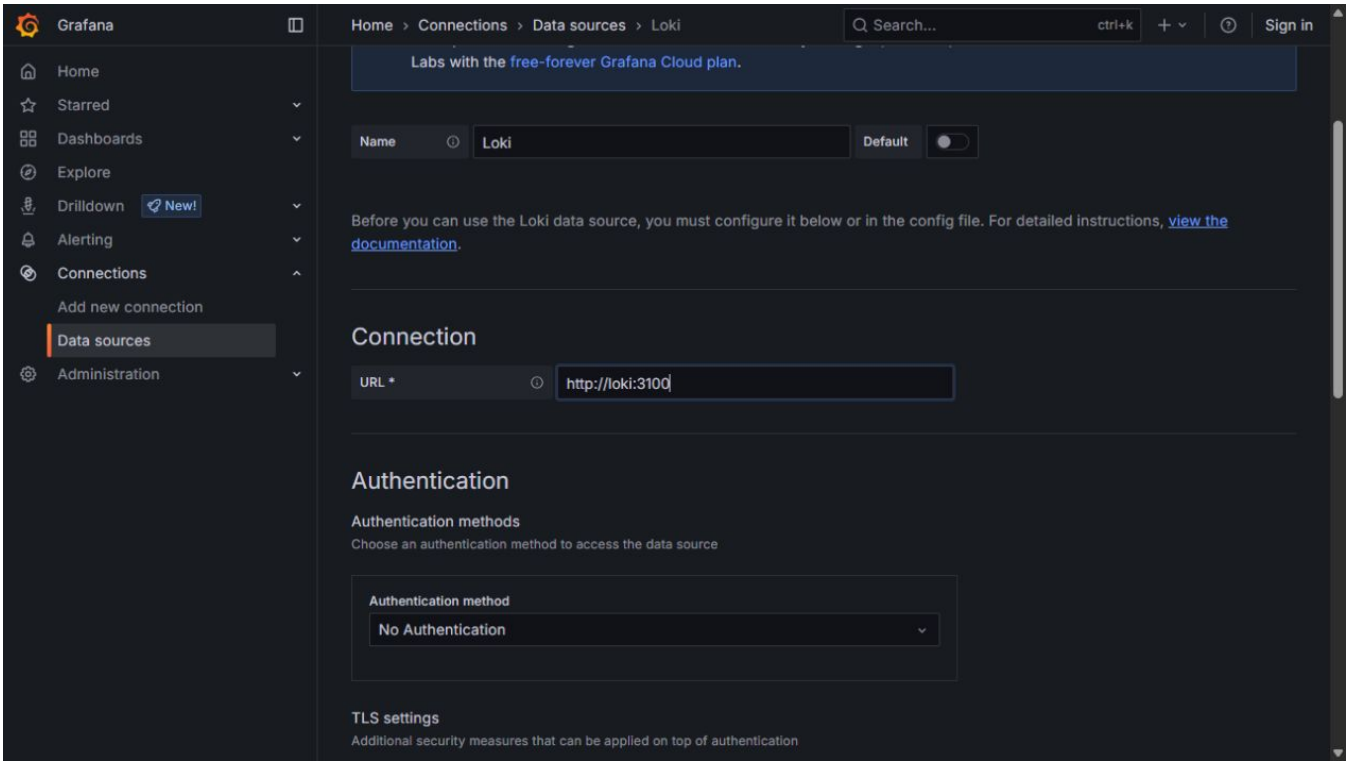
NOITE 3 • EXERCÍCIO 3.1 • ETAPA 2

Configurar a URL interna do Loki

1. No campo Name, defina Loki, com esse nome simples para reconhecer a fonte nos próximos exercícios.
2. Na seção Connection, preencha URL = http://loki:3100.
3. Mantenha No Authentication no laboratório local. Não preencha certificados, tokens nem campos de senha.
4. Role até o final da página e clique em Save & test.
5. Aguarde a confirmação Data source successfully connected. Não prossiga enquanto aparecer erro de conexão.

Resultado esperado: A conexão foi salva apontando para o serviço Loki da rede Docker.

Se não funcionar: Não use http://localhost:33100 nessa caixa: esse é o endereço do navegador, enquanto a fonte é acessada pelo container Grafana.



Tela real • 13-loki-url.jpg • Abra a imagem original no guia HTML para ampliar.

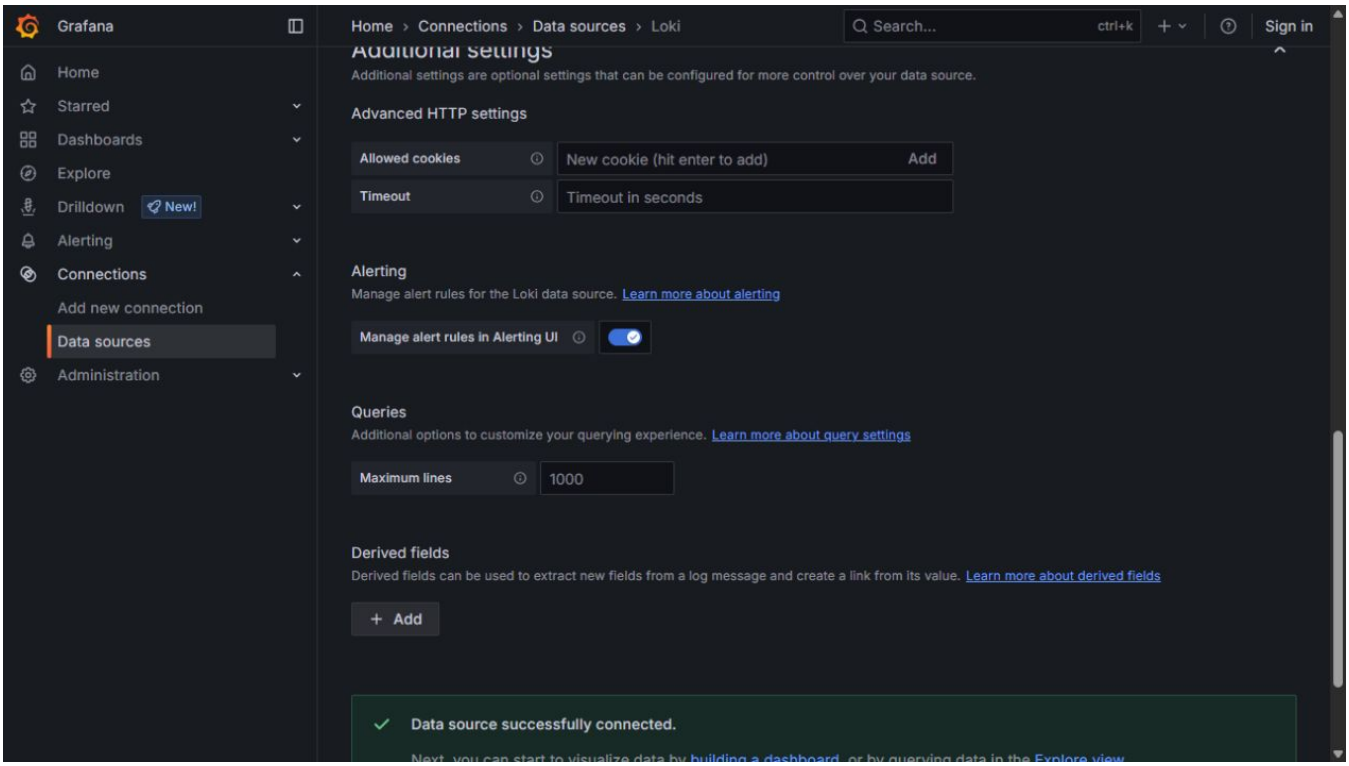
NOITE 3 • EXERCÍCIO 3.1 • ETAPA 3

Confirmar a conexão e entrar no Explore

1. Leia a mensagem Data source successfully connected.
2. Clique em Explore data na configuração da fonte, ou abra Explore pelo menu principal.
3. No seletor de fonte do Explore, confirme Loki. Selecione Code no editor A.
4. Use Last 15 minutes para começar. Se não aparecer seu primeiro evento, amplie para Last 1 hour.
5. Prepare a consulta do arquivo apresentada na próxima página.

Resultado esperado: Explore está usando Loki e não há erro de comunicação com a fonte.

Se não funcionar: A confirmação de conexão não cria logs. Se o container app ou promtail estiver parado, ainda poderá haver No data.



Tela real • 14-loki-teste.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 3 • EXERCÍCIO 3.1 • ETAPA 4

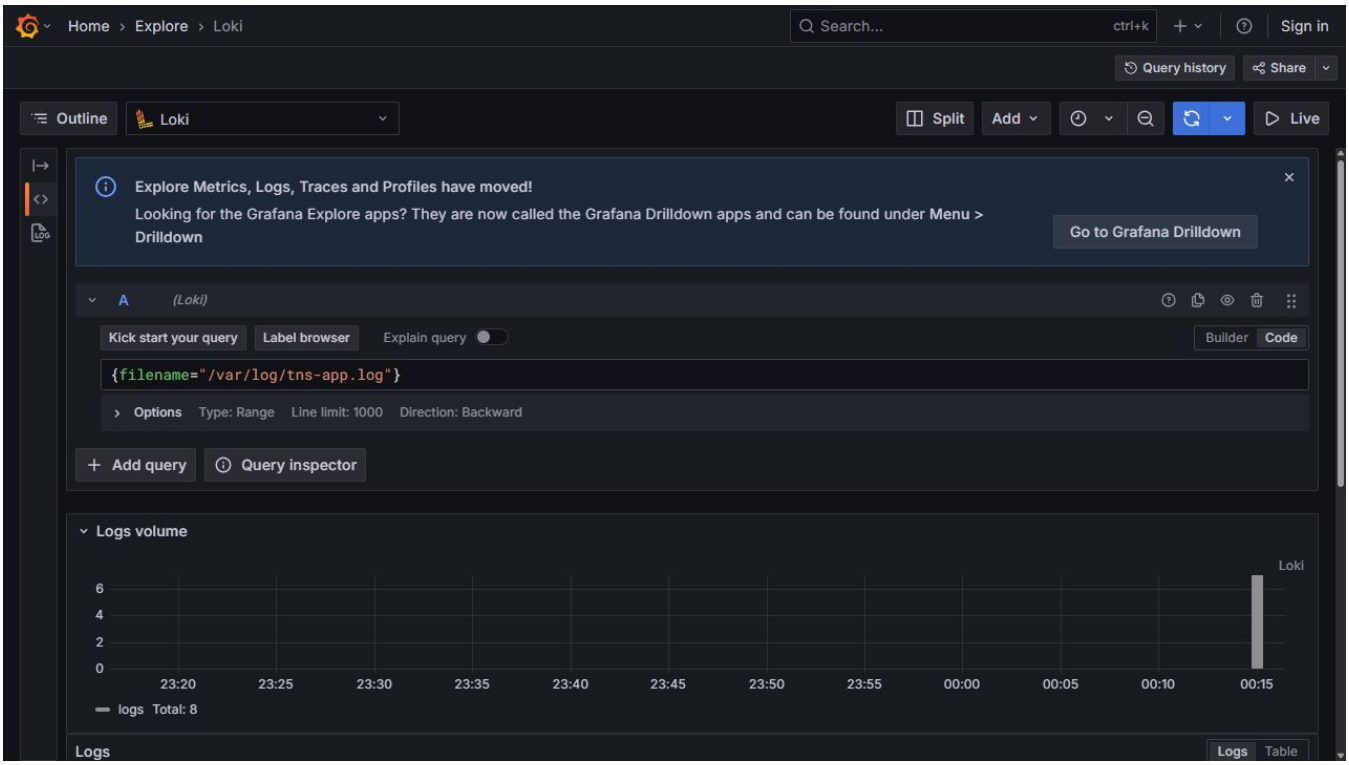
Consultar o arquivo inteiro

1. Cole a consulta abaixo no editor A e clique em Run query ou pressione Shift+Enter.
2. Aguarde o resultado. Você deve ver o volume de logs e uma lista de linhas da aplicação.
3. Se a lista estiver abaixo da área visível, role a página; você pode recolher Logs volume pelo título da seção.
4. Leia um horário e uma mensagem. filename identifica o arquivo; job identifica o fluxo de coleta.
5. Anote a quantidade de linhas exibida, sabendo que ela depende do período e do limite de resultados.

```
{filename="/var/log/tns-app.log"}
```

Resultado esperado: As linhas vêm de /var/log/tns-app.log e incluem eventos da aplicação.

Se não funcionar: Em No data: abra a aplicação, atualize-a, aguarde 10 a 20 segundos e execute novamente. Confira aspas, filename e período. Não use o seletor service do ambiente de e-commerce do professor neste laboratório.



Tela real • 15-loki-todos-logs.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 3 • EXERCÍCIO 3.2 • ETAPA 5

Produzir o erro controlado

1. Na aba da aplicação, preencha Title com Teste sem URL - seguido do seu nome.
2. Deixe o campo URL completamente vazio. Esta falha é intencional e limitada à aplicação de exemplo.
3. Anote o horário e clique em Submit uma única vez.
4. A aplicação deve mostrar empty url. Faça uma captura da mensagem.
5. Use Voltar do navegador para retornar ao formulário. Não continue reenviando o POST ao recarregar a página de erro.

Resultado esperado: A aplicação respondeu empty url e você registrou o horário.

Se não funcionar: Se o link foi publicado, havia uma URL no campo. Volte ao formulário, limpe-a e repita uma vez. Esse erro de validação não exige reinstalar o lab.



Tela real • 16-news-erro.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 3 • EXERCÍCIO 3.2 • ETAPA 6

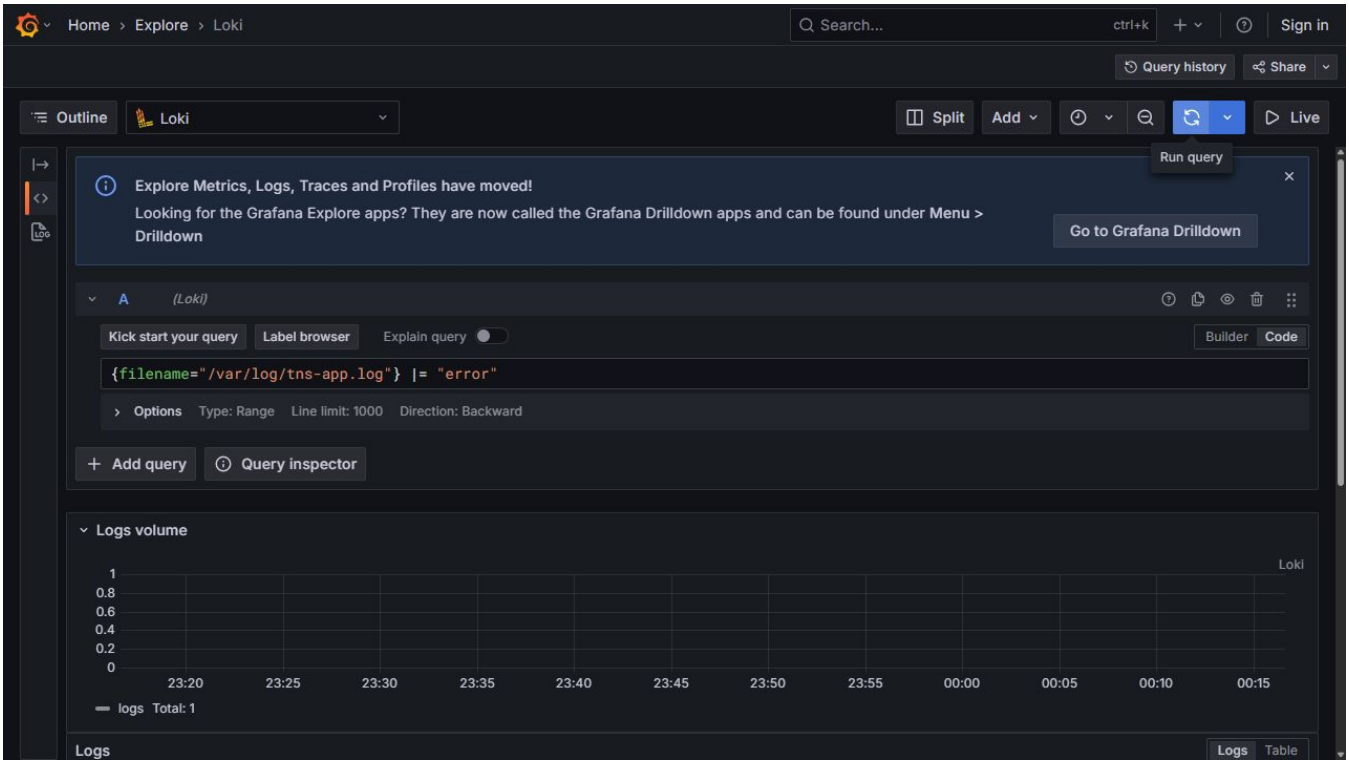
Localizar somente as linhas de erro

1. Volte ao Explore com Loki. Substitua a consulta anterior pelo filtro abaixo.
2. Selecione Last 15 minutes e clique em Run query. Espere a coleta se a ação acabou de ocorrer.
3. Na lista Logs, procure a linha level=error msg="empty url".
4. Compare o horário do log com o horário da ação no navegador. O fuso de exibição pode ser o do navegador.
5. O operador |= procura texto e diferencia maiúsculas de minúsculas. Neste aplicativo o texto é error, em minúsculas.

```
{filename="/var/log/tns-app.log"} |= "error"
```

Resultado esperado: Você relaciona o erro provocado à linha de log correspondente.

Se não funcionar: Se não houver resultado, remova temporariamente |= "error". Se os logs voltarem, revise o texto do filtro e o período. Se Live estiver ativo, pare-o antes de abrir os detalhes.



Tela real • 17-loki-erro.jpg • Abra a imagem original no guia HTML para ampliar.

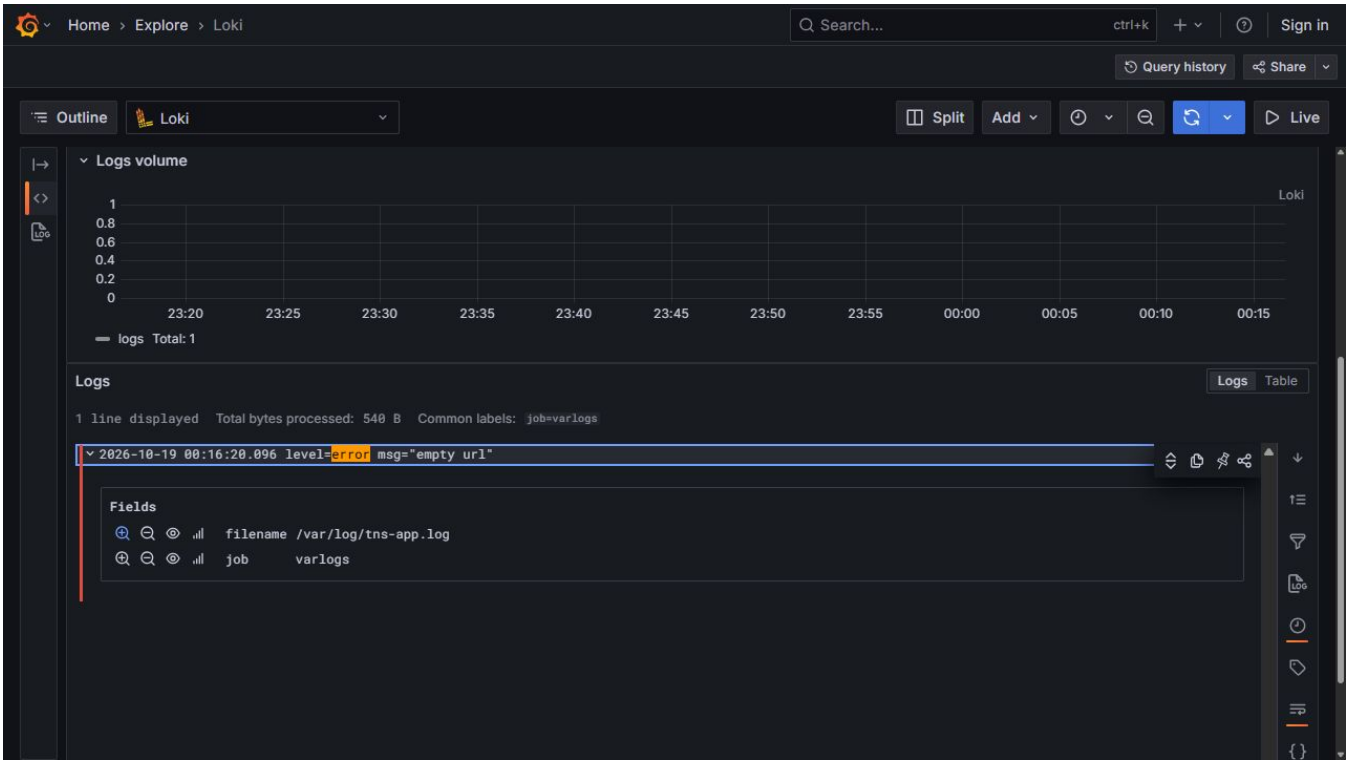
NOITE 3 • EXERCÍCIO 3.2 / 3.3 • ETAPA 7

Abrir o evento e descrever a investigação

1. Na linha do erro, clique na pequena seta ou no botão See log details, à esquerda da linha.
2. Leia a mensagem e os campos filename e job. Esses campos confirmam a origem do fluxo.
3. Compare a consulta sem filtro com a consulta contendo |= "error". A segunda deve mostrar um subconjunto da primeira no mesmo período.
4. Mude Last 15 minutes para Last 1 hour e observe se a quantidade de linhas aumenta. Depois volte ao período curto.
5. Entregue uma sequência reproduzível: ação feita, horário, fonte Loki, seletor, filtro e mensagem encontrada.
6. Escreva a conclusão: a aplicação rejeitou uma URL vazia e registrou o erro. Uma coincidência temporal isolada entre eventos diferentes não prova causa.

Resultado esperado: Sua evidência inclui o erro no navegador e a linha expandida nos logs.

Se não funcionar: As linhas logfmt deste exemplo não são JSON. Não acrescente | json automaticamente. A investigação de TraceID e Tempo pertence à demonstração complementar do professor.



Tela real • 18-loki-detalhe.jpg • Abra a imagem original no guia HTML para ampliar.

Noite 3 — conferência da entrega

3.1 — Conectar Loki (25 minutos)

Concluído: ____ Evidência salva em: _____

3.2 — Gerar e localizar erro (25 minutos)

Concluído: ____ Evidência salva em: _____

3.3 — Explicar investigação (25 minutos)

Concluído: ____ Evidência salva em: _____

Registre para a correção

Fonte / consulta / caminho utilizado: _____

Horário, período e resultado observado: _____

O que esse resultado significa: _____

Se houve falha: mensagem exata e verificações já feitas: _____

Noite 4 — Guia de exercícios

Dashboards e anotações

Fábio Silva • Arquiteto de Soluções • Green Tecnologia

Roteiro pedagógico atualizado em 20/10/2026. As capturas de referência foram obtidas em 19/10/2026 no laboratório local Grafana 12.0.1. Os valores e horários podem variar na sua instalação.

Como usar este guia

Siga os passos numerados e confira o resultado esperado. O bloco “Se não funcionar” ajuda a diagnosticar cada etapa. No HTML, clique na imagem para abrir o original e ampliar.

Durante a aula

Exercícios comentados: 20h45–21h00. Prática individual: 21h00–22h15 (75 minutos). Correção: 22h15–22h30. Intervalo: 20h20–20h45.

Atividade	Objetivo	Min
4.1	Criar e salvar um dashboard	30
4.2	Anotação manual	20
4.3	Anotação por logs e exportação	25

Pré-requisitos

Pacote completo extraído, Docker Desktop ativo no Windows ou Docker + Compose no Ubuntu. Use o Grafana News do pacote; Node Exporter, e-commerce e Tempo do ambiente do professor não são requisitos destas atividades.

Abas do laboratório: Grafana http://localhost:3300 • aplicação http://localhost:38081 • receptor http://localhost:38090.

Conclua a configuração de Loki da Noite 3 antes das anotações automáticas.

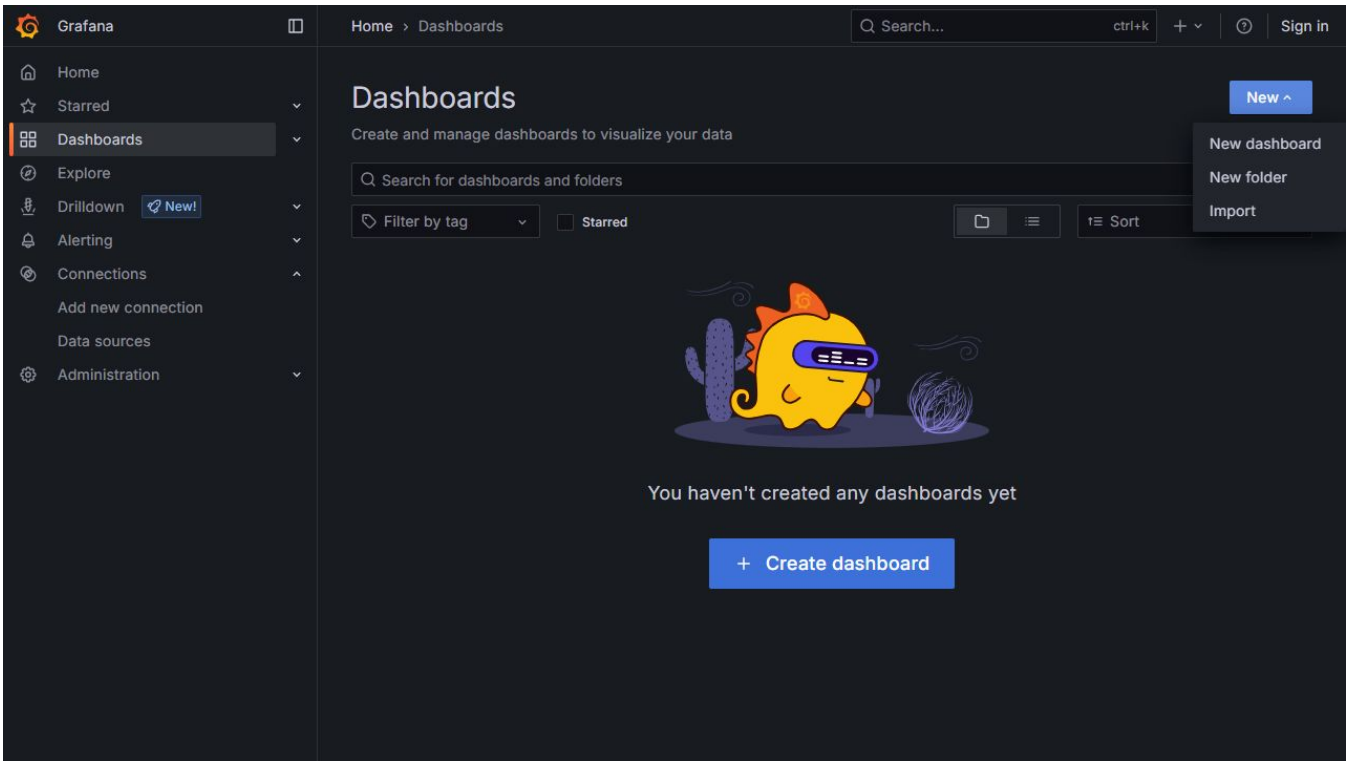
NOITE 4 • EXERCÍCIO 4.1 • ETAPA 1

Iniciar a criação manual do dashboard

1. No menu principal do Grafana, clique em Dashboards.
2. Na página Dashboards, abra o botão New e escolha New dashboard.
3. O exercício será feito manualmente para compreender consulta, visualização e salvamento.
4. Também há arquivos prontos em exemplos-importaveis, como material de apoio opcional.
5. Continue em Add visualization na próxima página e execute as etapas na ordem.

Resultado esperado: O dashboard vazio está aberto e pronto para receber seu primeiro painel.

Se não funcionar: O botão New da página de Dashboards é diferente do botão global + do cabeçalho. O menu da página mostra New dashboard e Import.



Tela real • 19-dashboard-new.jpg • Abra a imagem original no guia HTML para ampliar.

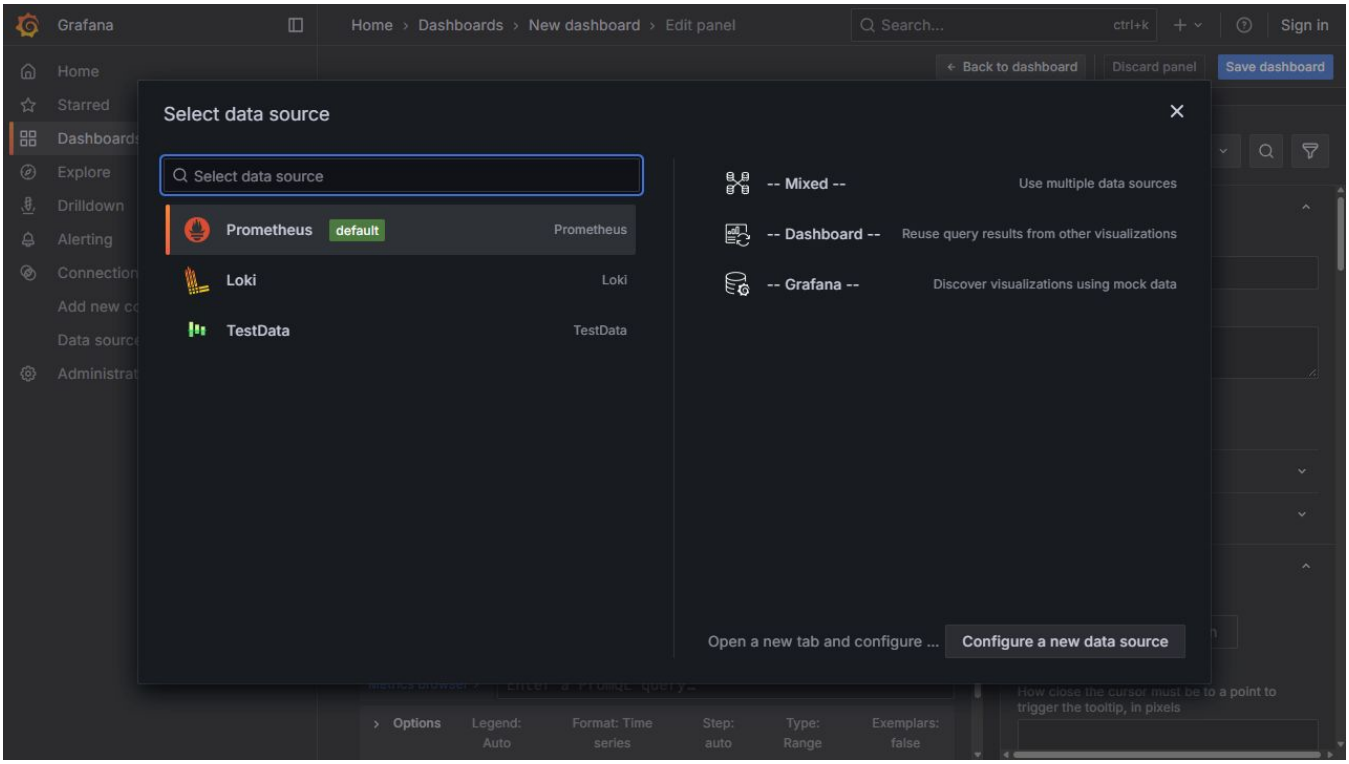
NOITE 4 • EXERCÍCIO 4.1 • ETAPA 2

Adicionar visualização Prometheus

1. No dashboard vazio, clique em Add visualization.
2. Na janela Select data source, escolha Prometheus.
3. Aguarde abrir o editor do painel. Na parte inferior, na aba Queries, confirme Data source = Prometheus.
4. Selecione Code no editor A, se ele estiver em Builder.
5. A visualização inicial deve ser Time series. Se houver outro tipo, clique no seletor Visualization e escolha Time series.

Resultado esperado: O editor mostra um painel Time series ligado a Prometheus.

Se não funcionar: Se a janela de fontes não abrir imediatamente, aguarde o carregamento. Escolher Loki aqui muda o editor para LogQL e não executa a consulta de métricas.



Tela real • 20-painel-fonte.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 4 • EXERCÍCIO 4.1 • ETAPA 3

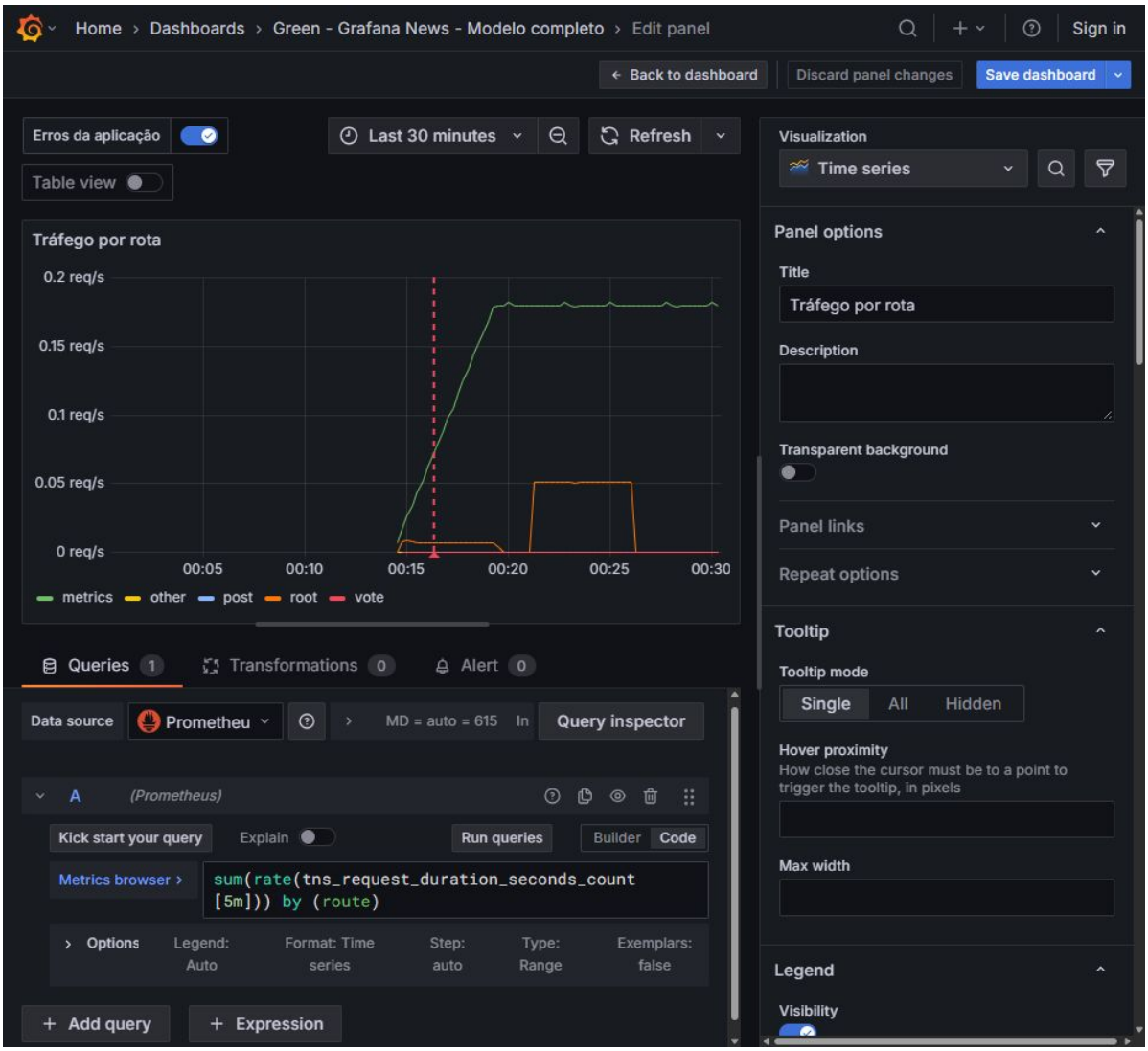
Preencher a consulta e o título

1. Na consulta A, cole a expressão abaixo e clique em Run queries.
2. À direita, em Panel options > Title, substitua New panel por Tráfego por rota.
3. Em Options, abaixo da consulta, confirme Type = Range e Format = Time series. Mantenha Legend = Auto; as rotas já identificam as séries.
4. Abra o seletor de período e escolha Last 15 minutes para concentrar o gráfico no laboratório recém-iniciado.
5. Se a consulta estiver escondida porque rolou a área inferior, role essa área para cima. O editor tem rolagem independente da coluna de opções.
6. Confira que o gráfico e a legenda apresentam as rotas. Os valores variam com o seu tráfego.

```
sum(rate(tns_request_duration_seconds_count[5m])) by (route)
```

Resultado esperado: Título, consulta, visualização e legenda correspondem ao exercício.

Se não funcionar: No Grafana 12, o retorno é Back to dashboard. Em algumas versões anteriores o botão equivalente é Apply. Não feche a aba do navegador.



Tela real • 21-painel-consulta.jpg • Abra a imagem original no guia HTML para ampliar.

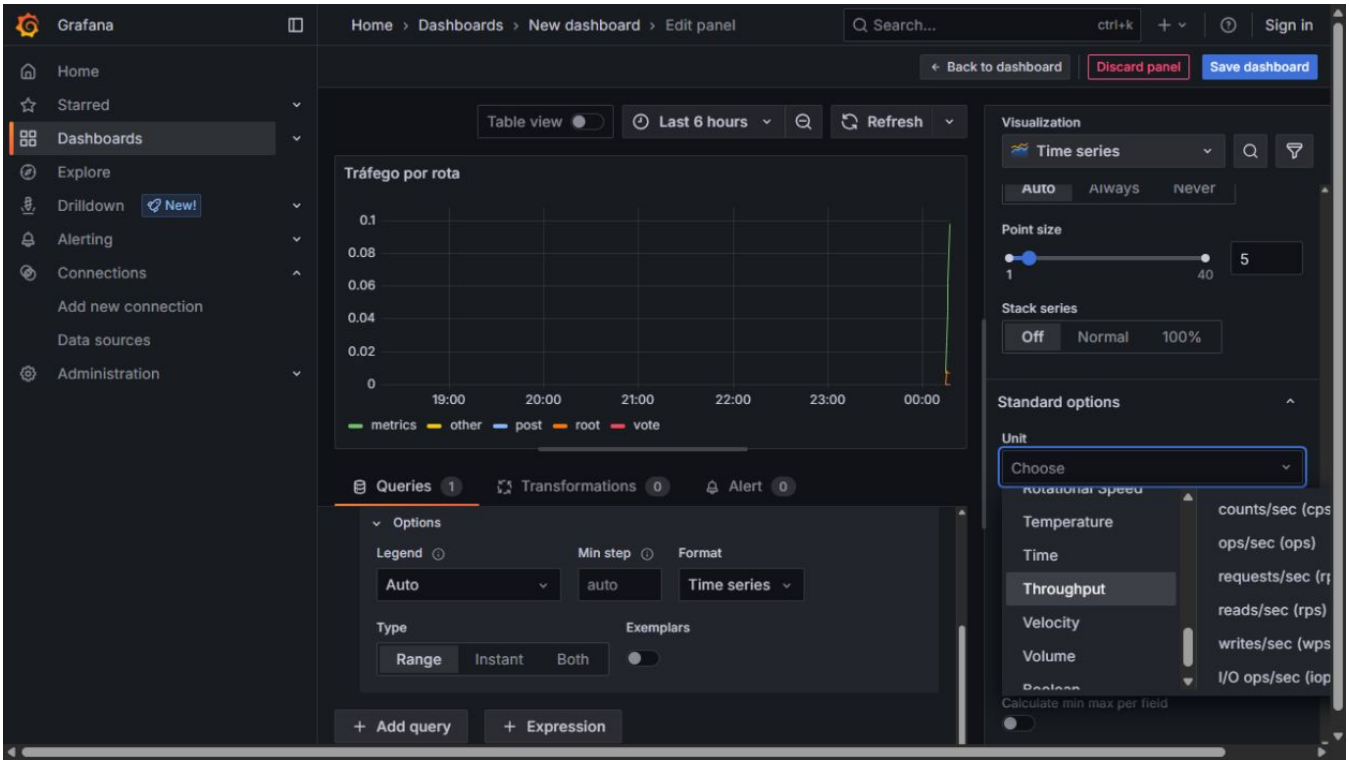
NOITE 4 • EXERCÍCIO 4.1 • ETAPA 4

Configurar a unidade correta

1. Na coluna de opções à direita, role até Standard options.
2. Clique no campo Unit, que inicialmente mostra Choose.
3. No menu de categorias, escolha Throughput. No submenu, clique em requests/sec (rps).
4. A unidade do eixo deve passar a req/s. Não escolha percentuais, bytes ou segundos.
5. Clique em Back to dashboard, no topo. Isso devolve o painel ao dashboard, mas você ainda precisa salvar o dashboard.

Resultado esperado: A curva é exibida em requisições por segundo.

Se não funcionar: Se o seletor não pesquisar ao digitar, use o caminho Throughput > requests/sec (rps). Não altere o nome da métrica para corrigir a unidade.



Tela real • 22-painel-unidade.jpg • Abra a imagem original no guia HTML para ampliar.

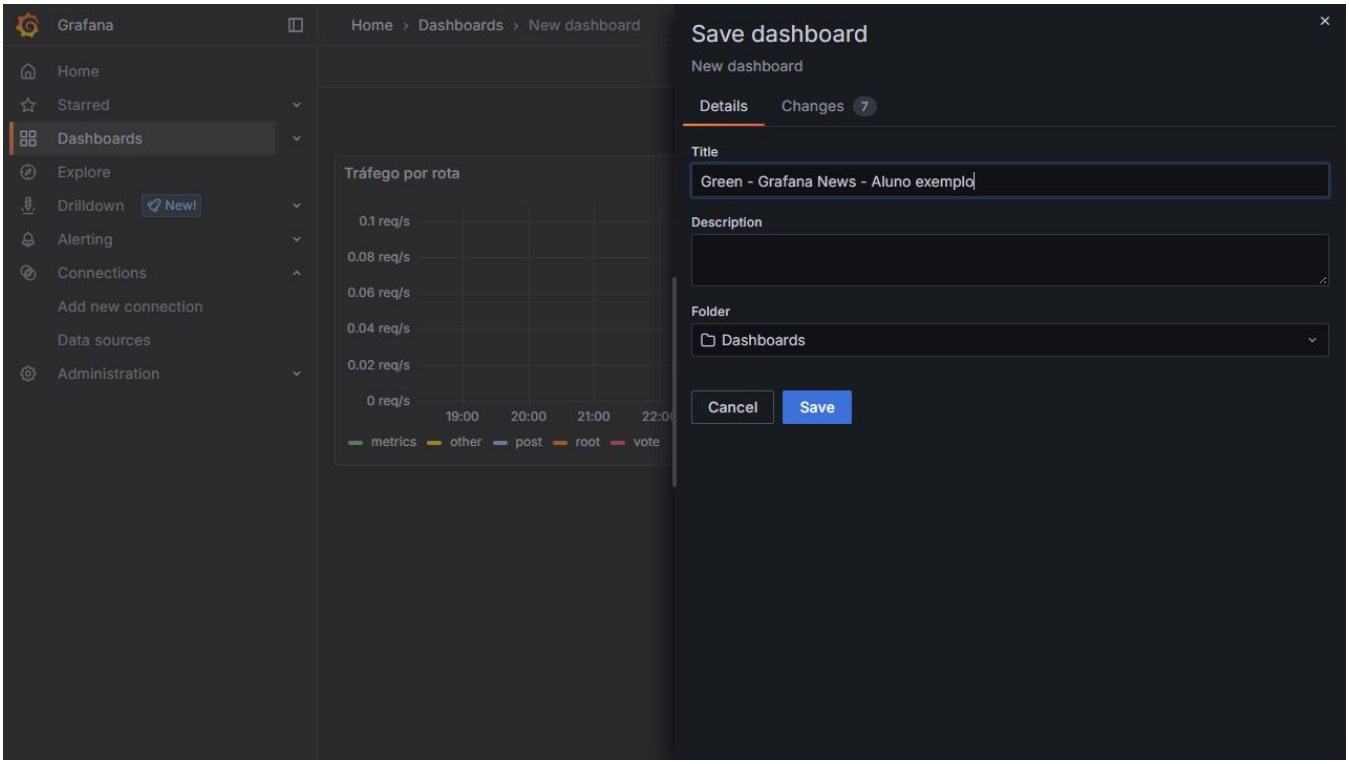
NOITE 4 • EXERCÍCIO 4.1 • ETAPA 5

Salvar com identificação do aluno

1. Clique em Save dashboard, no topo.
2. Em Title, digite Green - Grafana News - seguido do seu nome. Use um nome próprio para evitar conflitos.
3. Mantenha Folder = Dashboards, ou escolha a pasta pessoal da turma se ela já existir.
4. Clique em Save no rodapé da janela. Aguarde Dashboard saved.
5. Volte à página Dashboards e reabra pelo nome, ou recarregue a página atual, para conferir que o painel foi salvo.
6. Guarde o endereço do dashboard nos favoritos. Ele será usado na Noite 5.

Resultado esperado: O dashboard abre novamente e mantém o painel.

Se não funcionar: Back to dashboard aplica o painel; Save dashboard persiste a alteração. Se você só voltar e fechar sem salvar, poderá perder o trabalho.



Tela real • 23-dashboard-salvar.jpg • Abra a imagem original no guia HTML para ampliar.

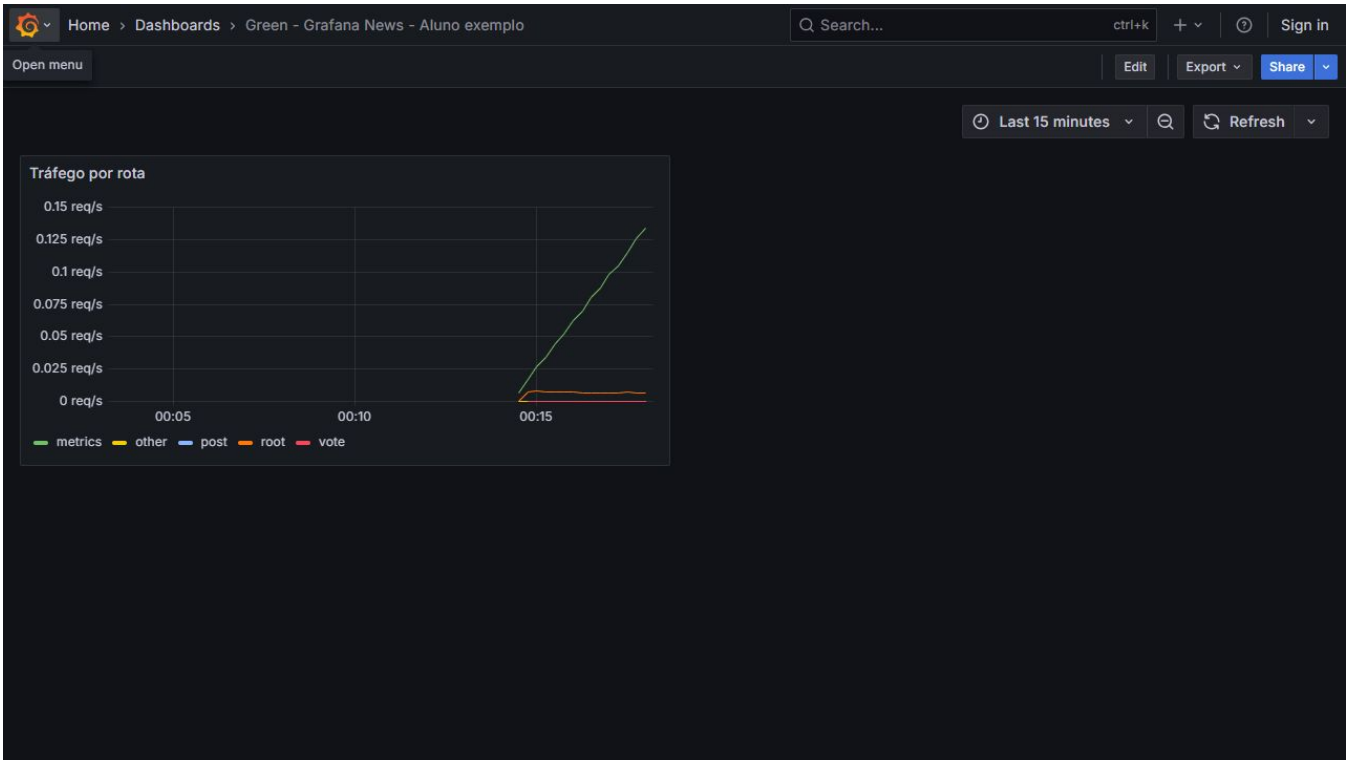
NOITE 4 • EXERCÍCIO 4.1 • ETAPA 6

Conferir o dashboard salvo

1. No dashboard salvo, confirme o título do painel Tráfego por rota e a unidade req/s.
2. No seletor de período, use Last 15 minutes. Clique em Refresh para atualizar.
3. Leia a legenda sem ocultar séries acidentalmente. Se clicar em uma série e as demais sumirem, clique novamente para restaurar.
4. Registre uma captura e o link do dashboard. Esta é a entrega do exercício 4.1.
5. Continue em 4.2. Os exemplos importáveis ficam em pasta separada e não são obrigatórios para concluir este exercício.

Resultado esperado: Você tem um painel salvo e sabe reabri-lo.

Se não funcionar: Um gráfico sem pontos antes da inicialização do lab é esperado. Ajuste o período para perto do horário em que começou a gerar atividade.



Tela real • 24-dashboard-manual.jpg • Abra a imagem original no guia HTML para ampliar.

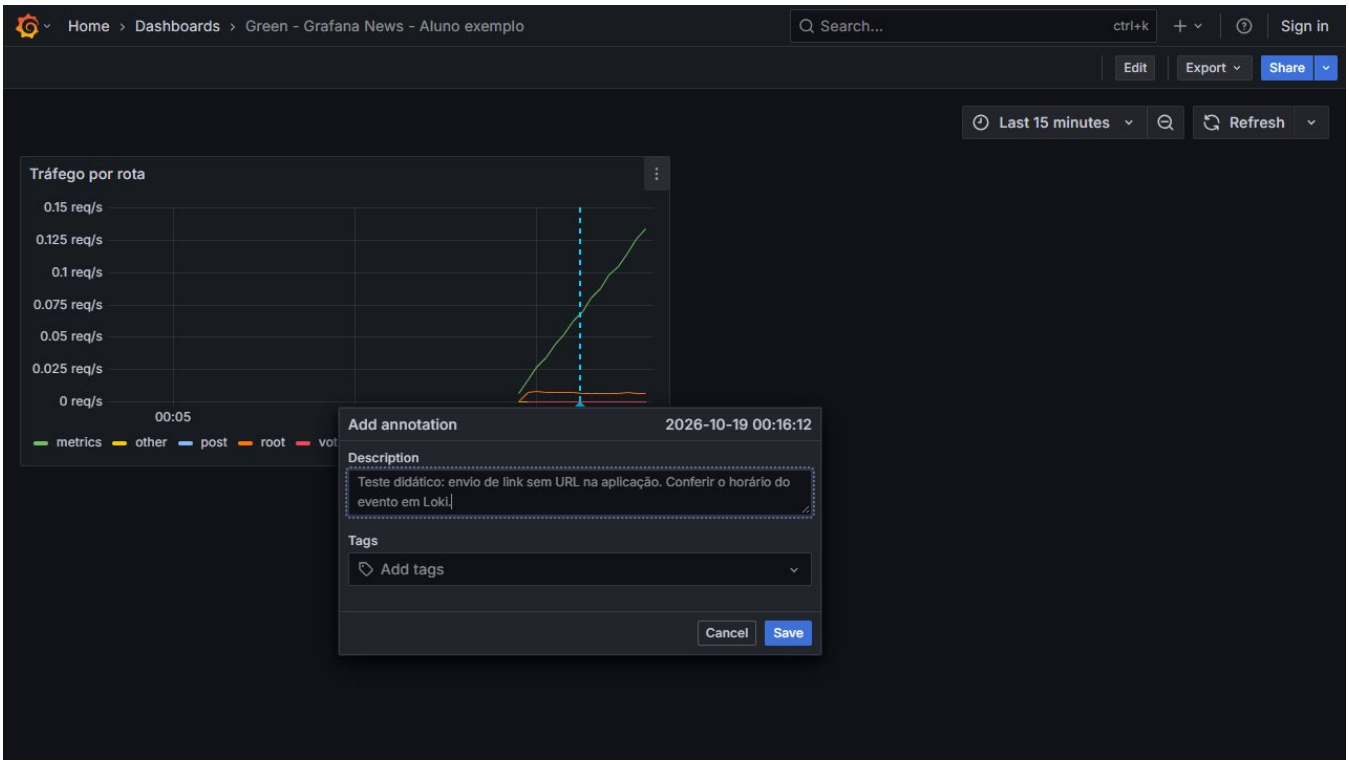
NOITE 4 • EXERCÍCIO 4.2 • ETAPA 7

Criar a anotação manual

1. Na aplicação, faça uma ação identificável, como publicar um novo link válido ou provocar empty url. Anote o horário exato.
2. Volte ao dashboard salvo e selecione Last 15 minutes. Localize no eixo do gráfico um ponto próximo ao horário da ação.
3. Segure Ctrl no Windows/Linux ou Command no macOS e clique dentro da área do gráfico. Deve abrir Add annotation.
4. Em Description, descreva a ação realmente feita. Exemplo: Teste didático: envio de link sem URL. Evite registrar um deploy ou uma migração que não ocorreu.
5. Clique em Save. Deve aparecer Annotation added e um marcador vertical no gráfico.
6. Para representar duração, use Ctrl/Command e arraste uma região; para um evento pontual, basta clicar. A anotação manual já é salva no banco do Grafana.

Resultado esperado: O gráfico contém um marcador com a descrição e o horário da sua ação.

Se não funcionar: Se Add annotation não abrir, confira se o dashboard já foi salvo e tente Ctrl+clique dentro do gráfico, longe da legenda. Passe o mouse sobre o marcador para ler o texto.



Tela real • 25-anotacao-manual.jpg • Abra a imagem original no guia HTML para ampliar.

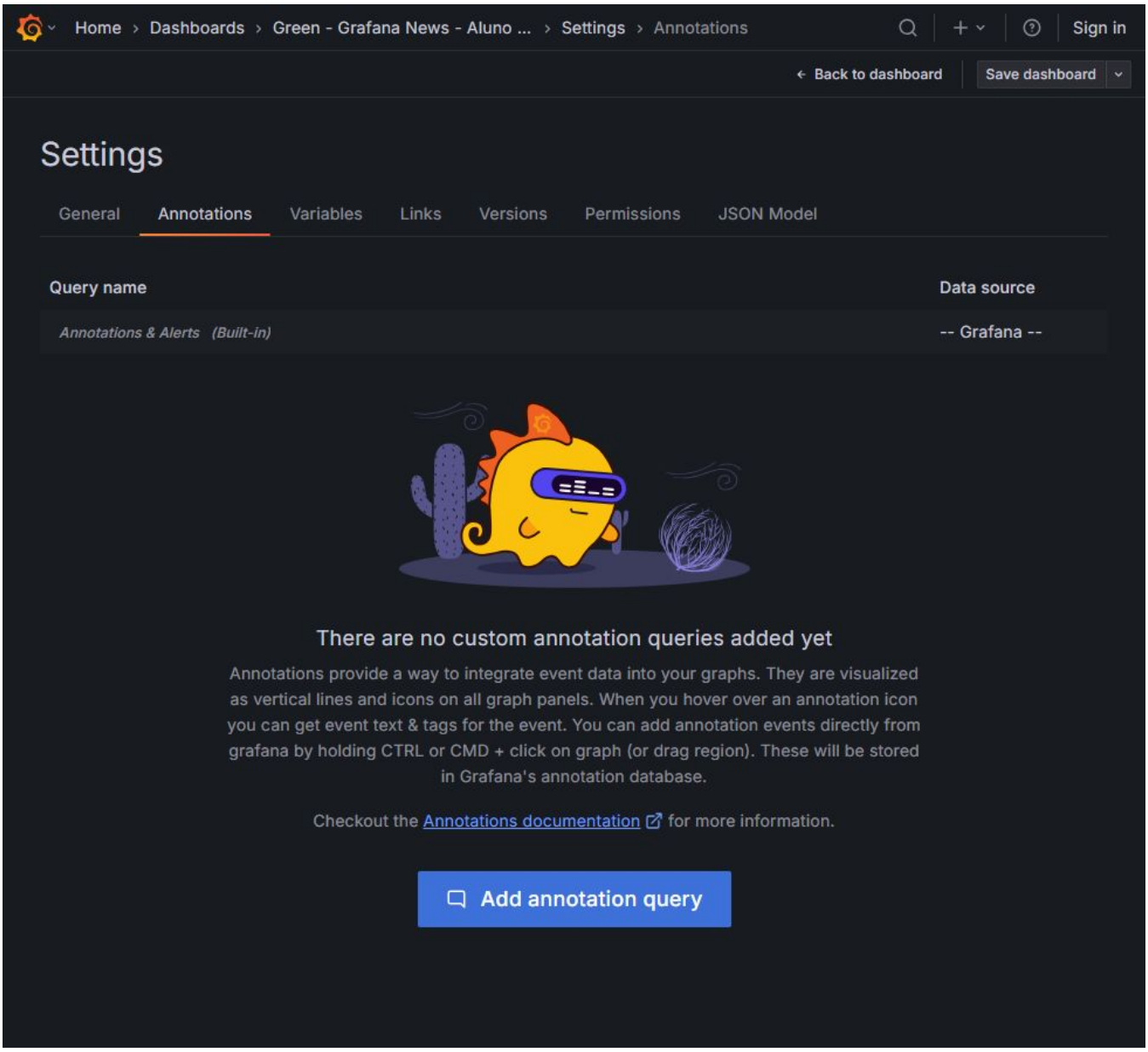
NOITE 4 • EXERCÍCIO 4.3 • ETAPA 8

Abrir consultas de anotação

1. No dashboard, clique em Edit. No topo, abra Settings. Em telas pequenas, procure em Show more items.
2. Na tela Settings, clique na aba Annotations.
3. Mantenha Annotations & Alerts (Built-in): ela exibe eventos salvos no Grafana.
4. Clique em Add annotation query. Será aberta uma consulta inicialmente ligada à fonte padrão.
5. Em Name, escreva Erros da aplicação. No seletor Data source, escolha Loki.

Resultado esperado: O formulário da nova anotação usa a fonte Loki.

Se não funcionar: Não remova a anotação integrada. Configurar Loki nesta tela não muda a fonte Prometheus do painel de métricas.



Tela real • 26-anotacoes-lista.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 4 • EXERCÍCIO 4.3 • ETAPA 9

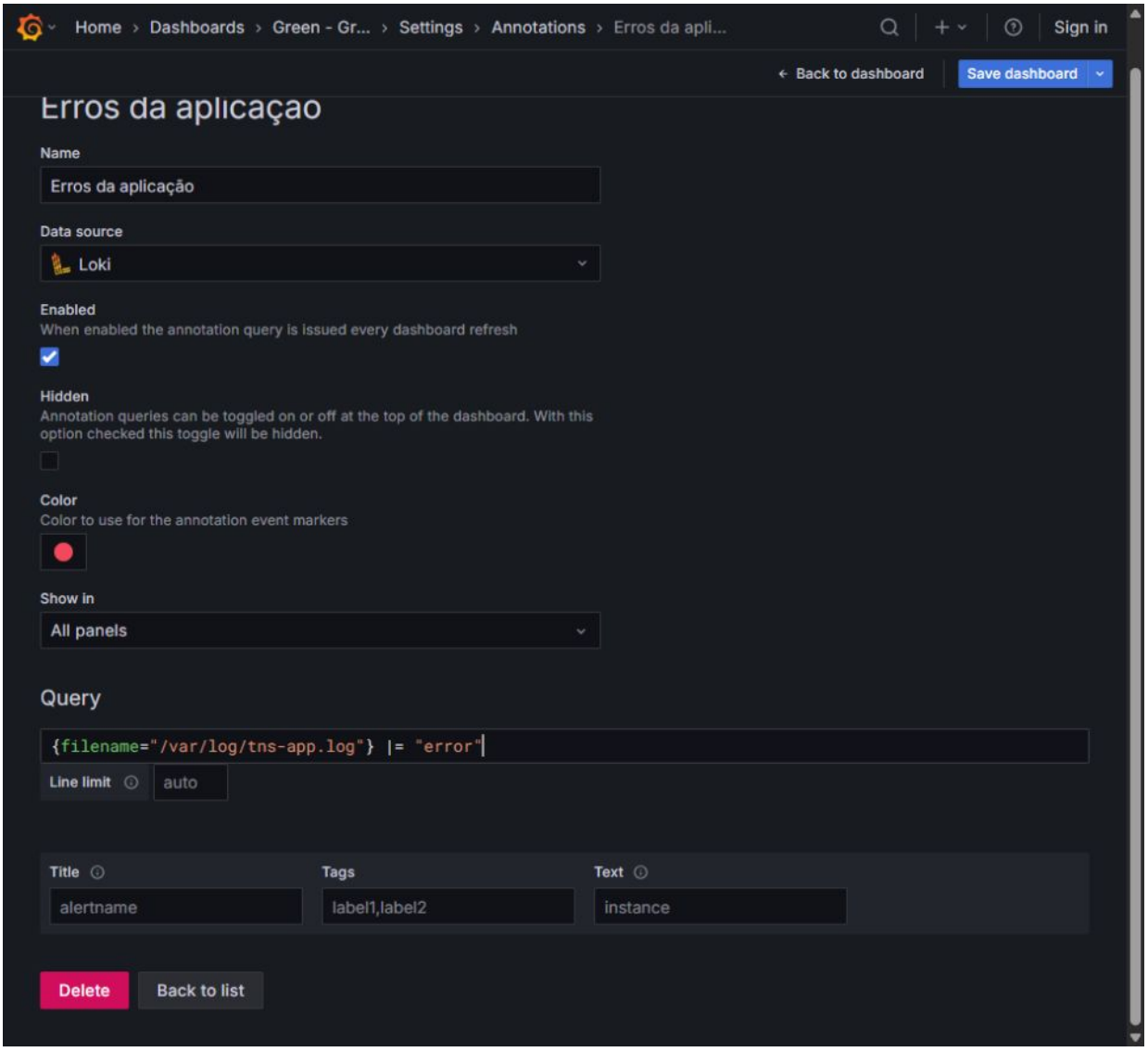
Configurar e salvar os marcadores de erro

1. Mantenha Enabled marcado. Deixe Hidden desmarcado para o controle aparecer no dashboard.
2. Em Query, cole a consulta abaixo. Mantenha Show in = All panels.
3. Deixe os campos opcionais Title, Text e Tags com os padrões. Não copie os valores alertname ou instance como se fossem obrigatórios para este aplicativo.
4. Clique em Back to dashboard e depois em Save dashboard. Confirme Save na janela.
5. Na aplicação, produza mais um erro de URL vazia. Espere 10 a 20 segundos e volte ao dashboard.
6. Clique em Refresh. Confirme que a chave Erros da aplicação está ligada e passe o mouse sobre o marcador do horário correspondente.

```
{filename="/var/log/tns-app.log"} |= "error"
```

Resultado esperado: Logs de erro aparecem como marcadores no painel de métricas.

Se não funcionar: Se não houver marcadores, execute a mesma consulta no Explore/Loki e confira o período. Primeiro valide o log; só depois investigue a anotação.



Tela real • 27-anotacao-loki.jpg • Abra a imagem original no guia HTML para ampliar.

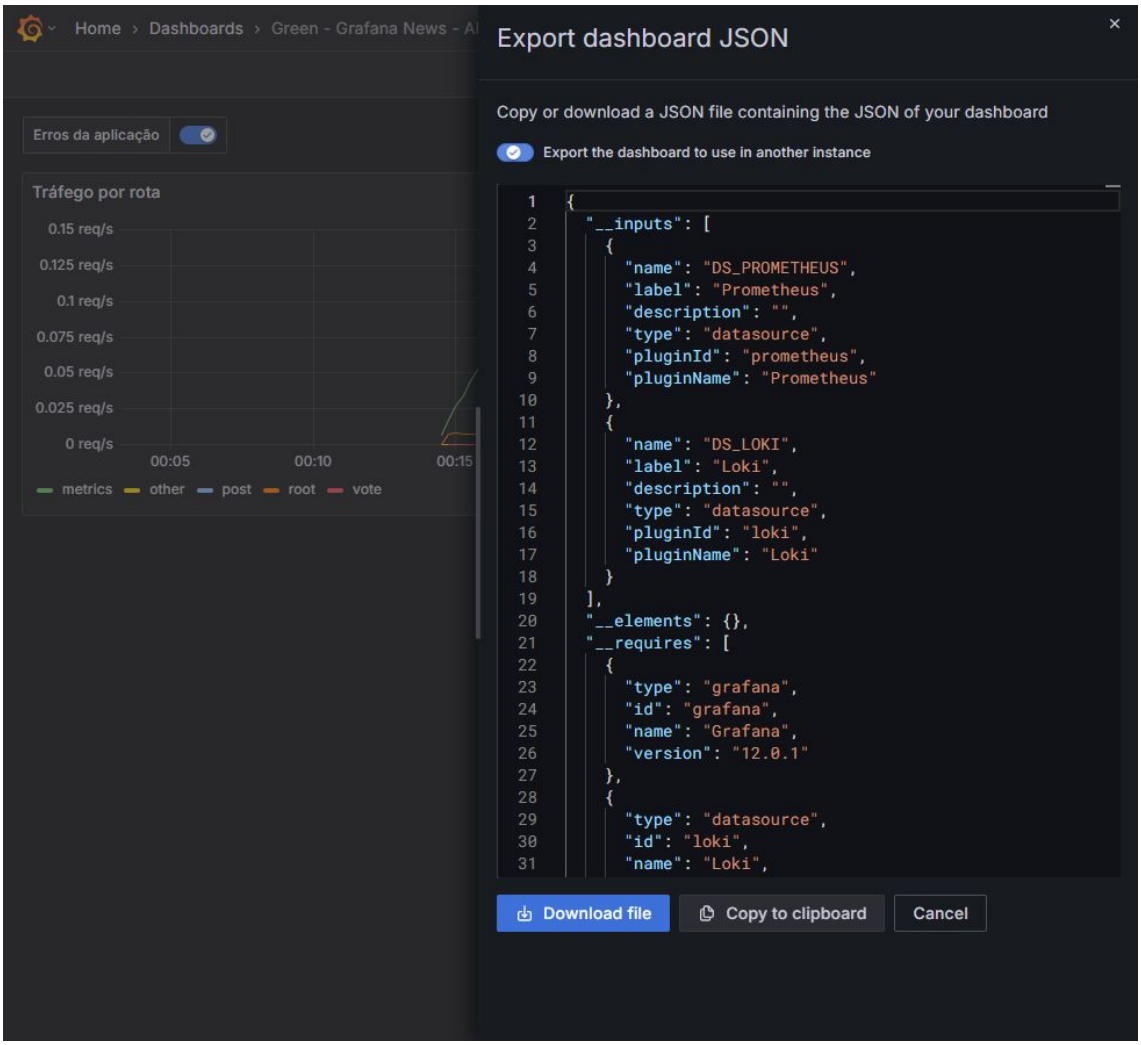
NOITE 4 • EXERCÍCIO 4.3 • ETAPA 10

Conferir e exportar para guardar o trabalho

1. Confira a anotação manual e a consulta de erros. Elas são mecanismos diferentes: o evento manual fica no banco; a consulta busca eventos no Loki.
2. Saia do modo Edit. No topo, abra Export > Export as JSON. Em versões anteriores o caminho pode estar em Share > Export.
3. Ative Export the dashboard to use in another instance para solicitar o mapeamento de fontes em outra instalação.
4. Clique em Download file e guarde o arquivo na pasta do curso. Você também pode usar Copy to clipboard para salvar o texto JSON.
5. O JSON preserva o painel e a definição da consulta de anotações. Ele não exporta os registros de logs nem os eventos manuais do banco.
6. Entregue captura do dashboard, anotação com texto e consulta de erros. Os exemplos importáveis para consulta opcional estão em exemplos-importaveis.

Resultado esperado: Você tem um arquivo JSON portátil e sabe o que ele contém.

Se não funcionar: Copiar o link do dashboard não cria um backup. Não remova containers antes de exportar. A existência do arquivo não substitui verificar se a importação abre corretamente.



Tela real • 29-exportar-json.jpg • Abra a imagem original no guia HTML para ampliar.

Noite 4 — conferência da entrega

4.1 — Criar e salvar um dashboard (30 minutos)

Concluído: ____ Evidência salva em: _____

4.2 — Anotação manual (20 minutos)

Concluído: ____ Evidência salva em: _____

4.3 — Anotação por logs e exportação (25 minutos)

Concluído: ____ Evidência salva em: _____

Registre para a correção

Fonte / consulta / caminho utilizado: _____

Horário, período e resultado observado: _____

O que esse resultado significa: _____

Se houve falha: mensagem exata e verificações já feitas: _____

Noite 5 — Guia de exercícios

Alertas e entrega do projeto

Fábio Silva • Arquiteto de Soluções • Green Tecnologia

Roteiro pedagógico atualizado em 20/10/2026. As capturas de referência foram obtidas em 19/10/2026 no laboratório local Grafana 12.0.1. Os valores e horários podem variar na sua instalação.

Como usar este guia

Siga os passos numerados e confira o resultado esperado. O bloco “Se não funcionar” ajuda a diagnosticar cada etapa. No HTML, clique na imagem para abrir o original e ampliar.

Durante a aula

Exercícios comentados: 20h45–21h00. Prática individual: 21h00–22h15 (75 minutos). Correção: 22h15–22h30. Intervalo: 20h20–20h45.

Atividade	Objetivo	Min
5.1	Contato e TestAlert	20
5.2	Regra, avaliação e disparo	30
5.3	Vínculo, apresentação e exportação	25

Pré-requisitos

Pacote completo extraído, Docker Desktop ativo no Windows ou Docker + Compose no Ubuntu. Use o Grafana News do pacote; Node Exporter, e-commerce e Tempo do ambiente do professor não são requisitos destas atividades.

Abas do laboratório: Grafana http://localhost:3300 • aplicação http://localhost:38081 • receptor http://localhost:38090.

Tenha um dashboard salvo e o painel Tráfego por rota da Noite 4.

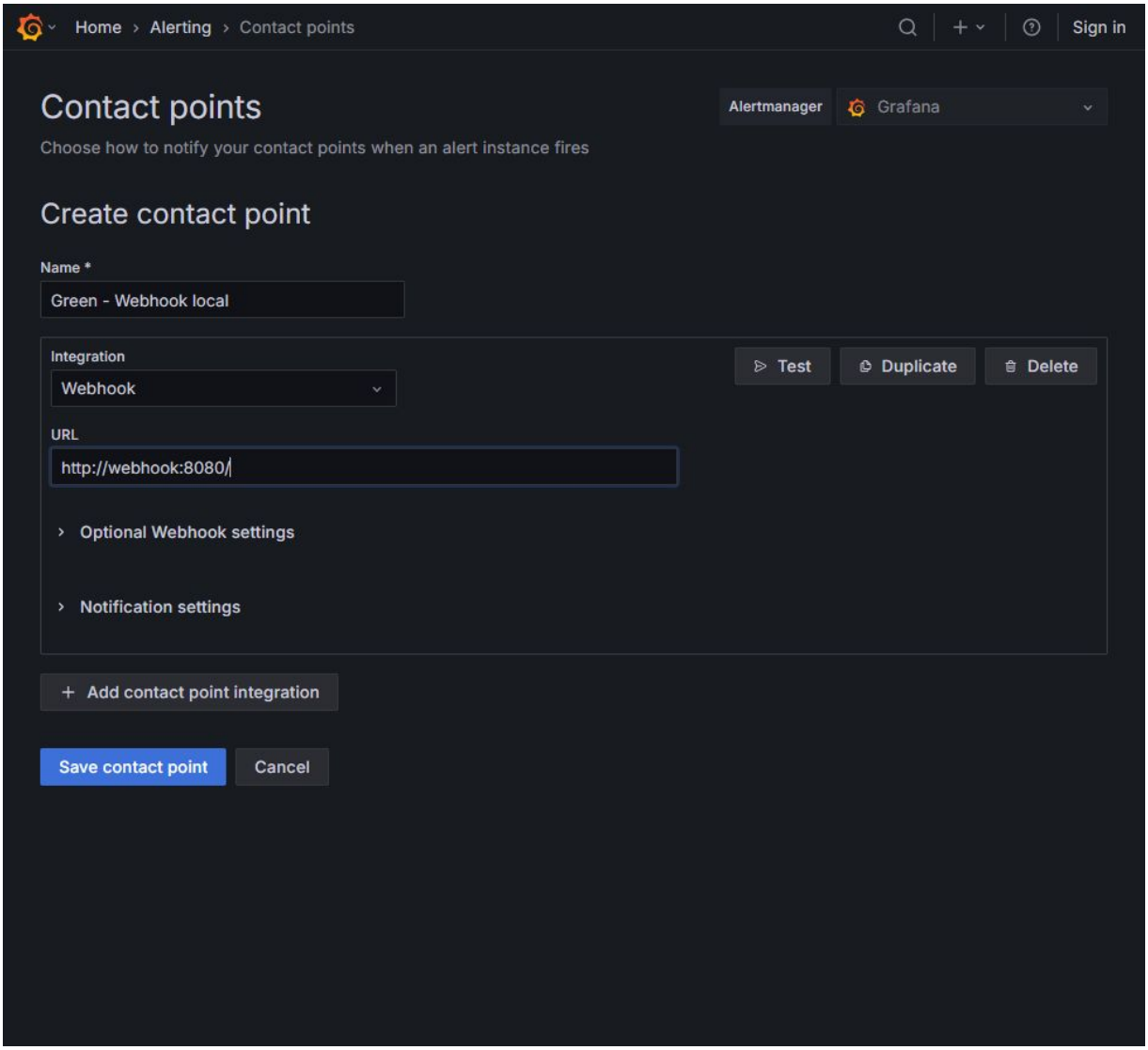
NOITE 5 • EXERCÍCIO 5.1 • ETAPA 1

Criar o contato Webhook local

1. Abra o receptor em http://localhost:38090 para confirmar que está acessível. Mantenha essa aba aberta.
2. No Grafana, abra Alerting > Contact points. Se estiver na página inicial de Alerting, clique em Manage contact points.
3. Clique em Create contact point. Em Name, escreva Green - Webhook local ou um nome que identifique você.
4. No seletor Integration, escolha Webhook. No campo URL, preencha exatamente http://webhook:8080/.
5. Use o endereço interno nessa configuração. localhost:38090 é o endereço externo usado pelo seu navegador.
6. Não configure Email para esta atividade. A localização do SMTP e dos destinatários está na teoria, sem envio de e-mail.

Resultado esperado: O formulário mostra Integration = Webhook e URL = http://webhook:8080/.

Se não funcionar: Se o receptor não abrir no navegador, confira o serviço webhook do pacote. Webhook.site não é necessário: o teste será todo local.



Tela real • 33-contato-webhook.jpg • Abra a imagem original no guia HTML para ampliar.

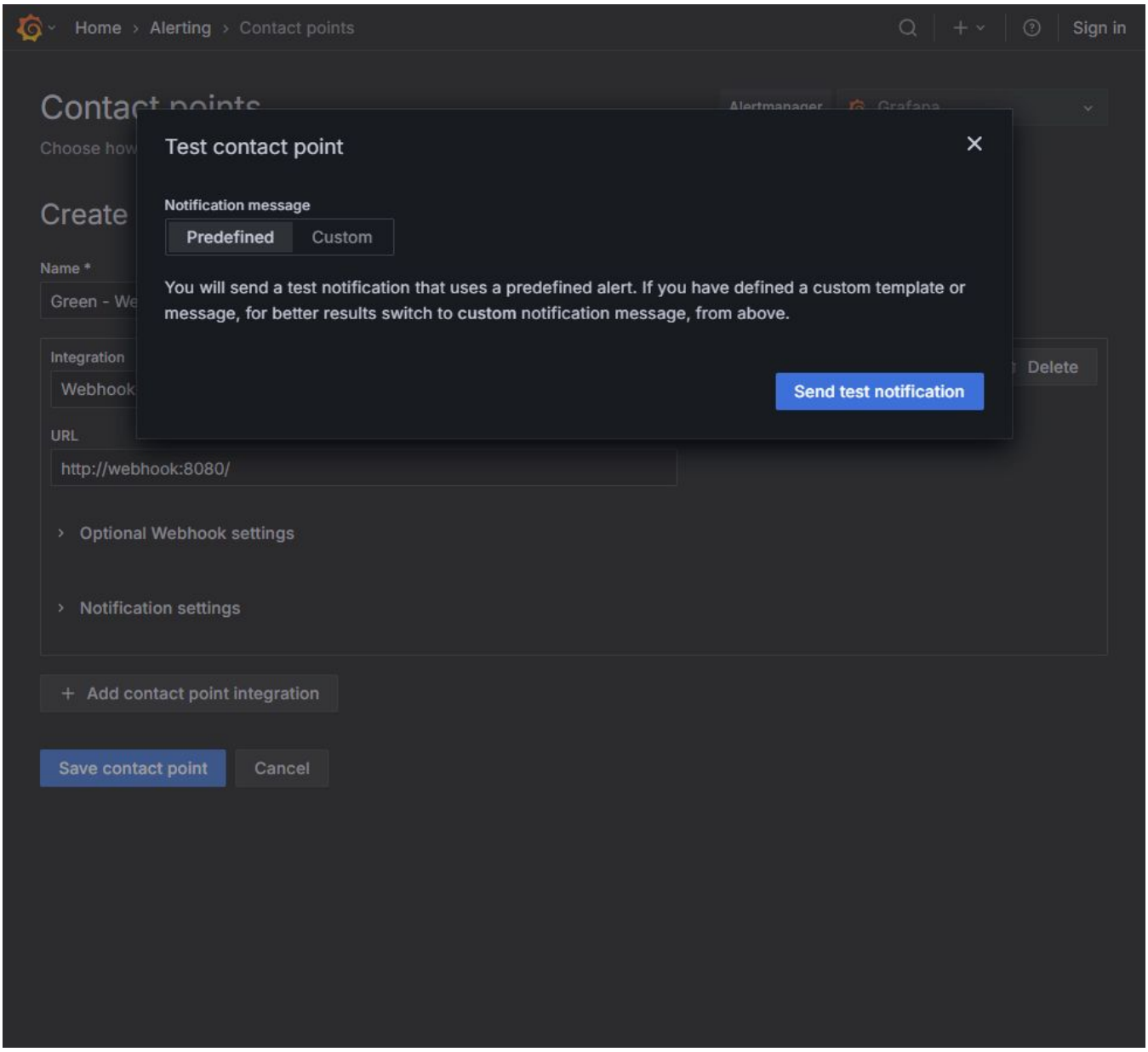
NOITE 5 • EXERCÍCIO 5.1 • ETAPA 2

Enviar o teste de comunicação

1. No formulário do contato, clique em Test.
2. Na janela Test contact point, mantenha Predefined.
3. Confira que a URL do contato é o receptor local e clique em Send test notification.
4. Espere Test alert sent. Feche a janela pelo X ou Close.
5. Clique em Save contact point para salvar o contato, mesmo depois de o teste ter dado certo.

Resultado esperado: O Grafana confirma o envio e o contato fica salvo.

Se não funcionar: Testar não salva o contato. Se houver erro de DNS ou conexão, confira a grafia webhook, a porta interna 8080 e se o serviço está na mesma rede Docker.



Tela real • 34-contato-teste.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 5 • EXERCÍCIO 5.1 • ETAPA 3

Confirmar a mensagem de teste no receptor

1. Volte à aba http://localhost:38090 e atualize a página pelo navegador.
2. Confira a contagem de mensagens. Ela deve aumentar depois do teste; não precisa ser exatamente 1 se você já testou antes.
3. No JSON, localize receiver = test e labels.alertname = TestAlert.
4. Essa mensagem comprova comunicação com o receptor, mas ainda não comprova que uma regra sua foi avaliada.
5. Guarde uma captura e anote TestAlert como origem do teste. O alerta real terá o nome da regra criada a seguir.

Resultado esperado: O receptor apresenta o JSON do TestAlert.

Se não funcionar: O receptor guarda as últimas 100 mensagens em memória. Reiniciar esse serviço apaga o histórico. Preserve suas evidências antes de reiniciá-lo.



Tela real • 35-webhook-recebido.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 5 • EXERCÍCIO 5.2 • ETAPA 4

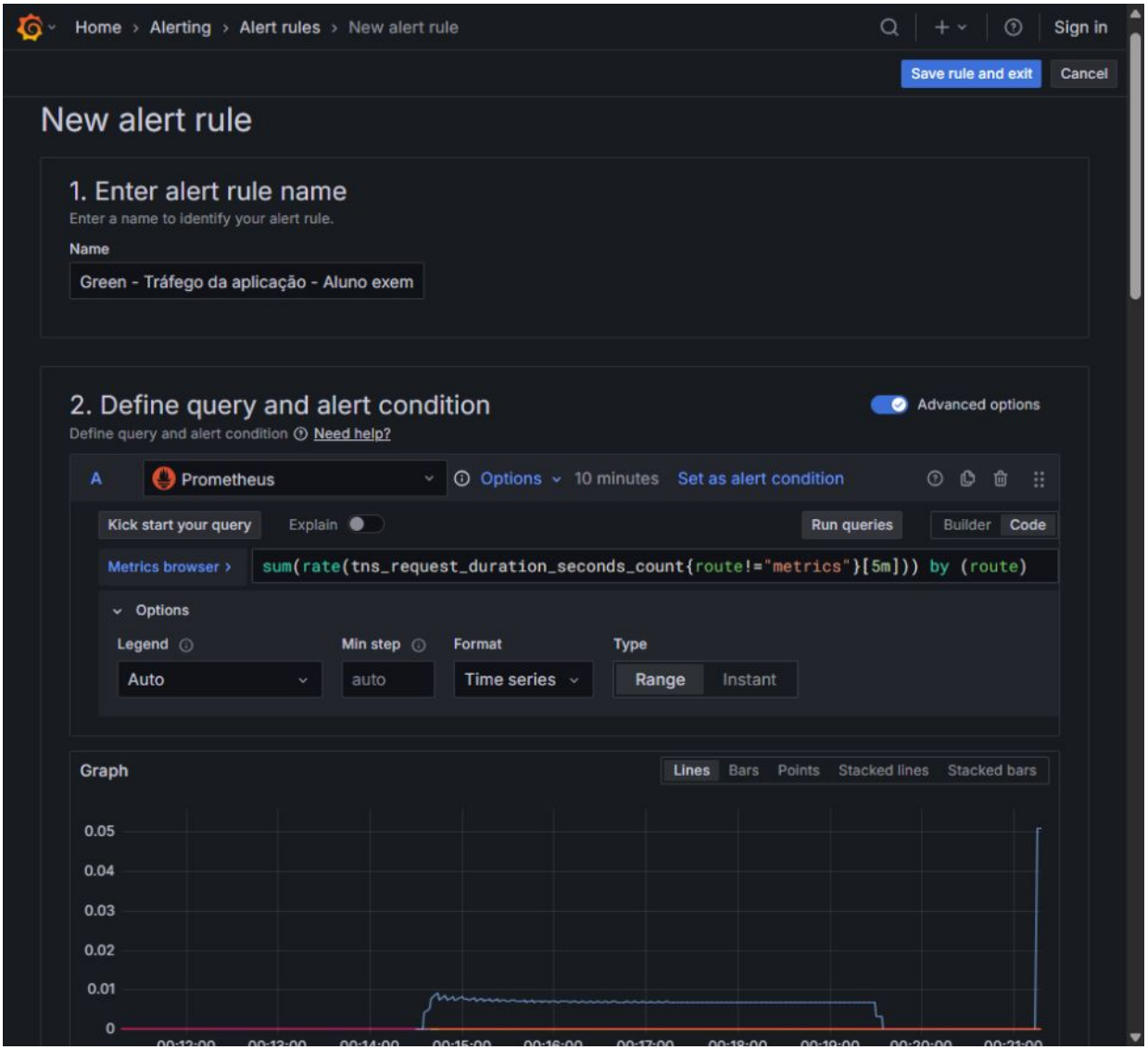
Criar a consulta A em modo avançado

1. Abra Alerting > Alert rules. Clique em New alert rule.
2. Em Name, escreva Green - Tráfego da aplicação - seguido do seu nome.
3. Na seção 2, ative Advanced options, se estiver usando a tela simplificada. Confirme Grafana-managed.
4. Selecione Prometheus e Code na consulta A. Cole a expressão abaixo.
5. Em Options da própria consulta, defina Type = Range. Mantenha o intervalo de consulta em 10 minutes e clique em Run queries.
6. A consulta exclui route="metrics", para que o scrape não dispare o alerta como se fosse navegação do aluno. Ela mantém uma série por rota.

```
sum(rate(tns_request_duration_seconds_count{route!="metrics"}[5m])) by (route)
```

Resultado esperado: A consulta A devolve séries temporais das rotas da aplicação.

Se não funcionar: A faixa de consulta 10m e a janela [5m] têm funções distintas. Se a UI criar uma expressão reducer automaticamente, reutilize-a no próximo passo em vez de adicionar outra igual.



Tela real • 36-alerta-query-range.jpg • Abra a imagem original no guia HTML para ampliar.

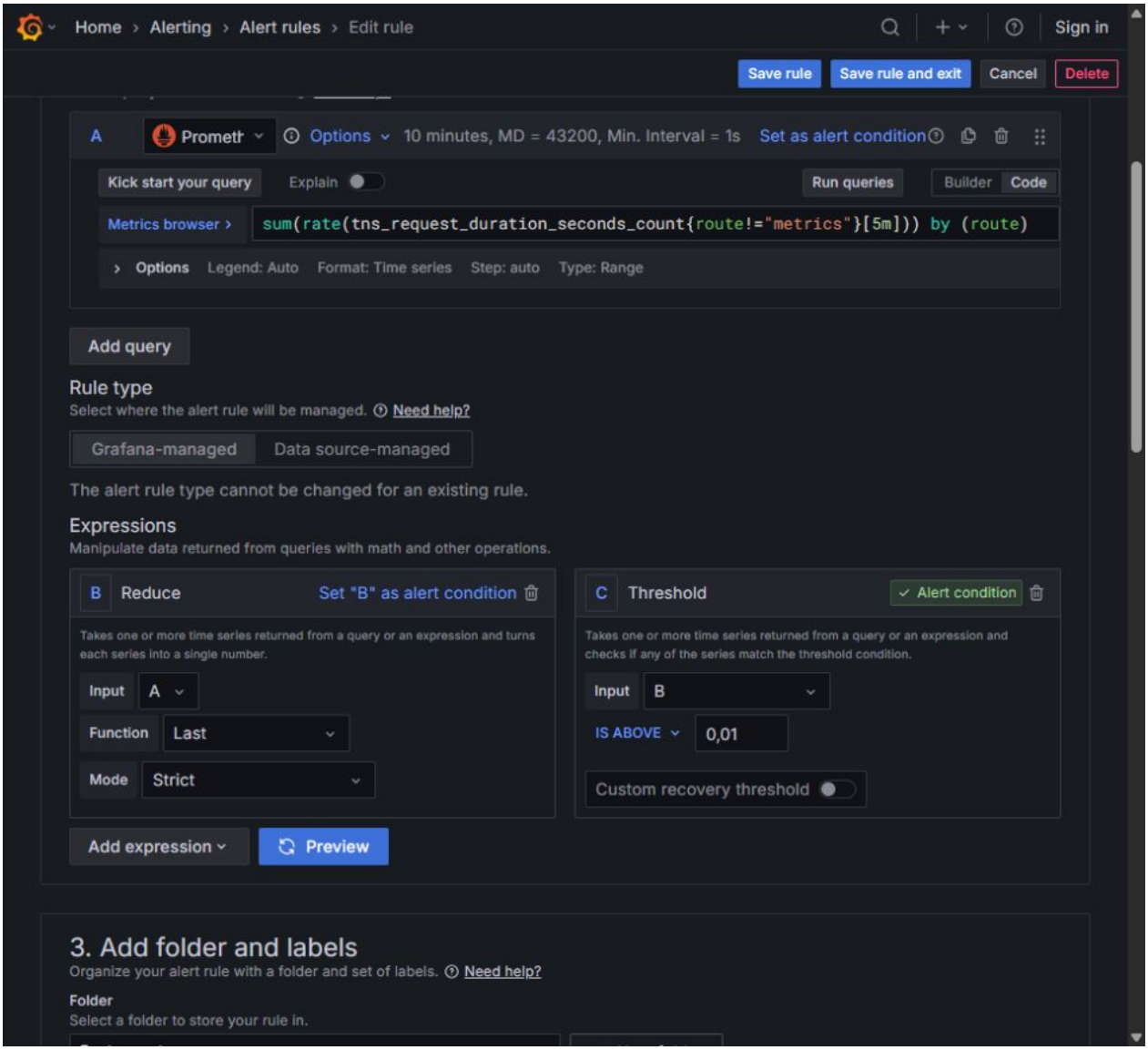
NOITE 5 • EXERCÍCIO 5.2 • ETAPA 5

Definir B e C sem ligar o limite à série bruta

1. Em Expressions, localize Reduce. Se ela já existir como reducer, clique no nome, troque por B e pressione Enter. Se não existir, use Add expression > Reduce e nomeie B.
2. Em B, configure Input = A, Function = Last e Mode = Strict. Cada série será reduzida a um valor.
3. Localize a expressão Threshold C. Configure Input = B, Is above e limite 0.01. A tela em português pode mostrar 0,01.
4. Depois de renomear reducer, confira novamente Input de C: algumas versões voltam para A. Reescolha B.
5. C deve estar marcada Alert condition. Caso não esteja, use Set as alert condition na expressão C.
6. Clique em Preview. A prévia pode mostrar Normal ou Firing, conforme o valor atual de B. Neste laboratório o limite é didático, não um limite recomendado para produção.

Resultado esperado: B tem um número por rota e C compara esse número com 0,01 req/s.

Se não funcionar: You cannot use time series data as an alert condition indica que C ainda usa A ou que B não é Reduce/Last. Corrija a referência antes de salvar. Não conclua entrega somente pela prévia.



Tela real • 37-alerta-reduce-threshold.jpg • Abra a imagem original no guia HTML para ampliar.

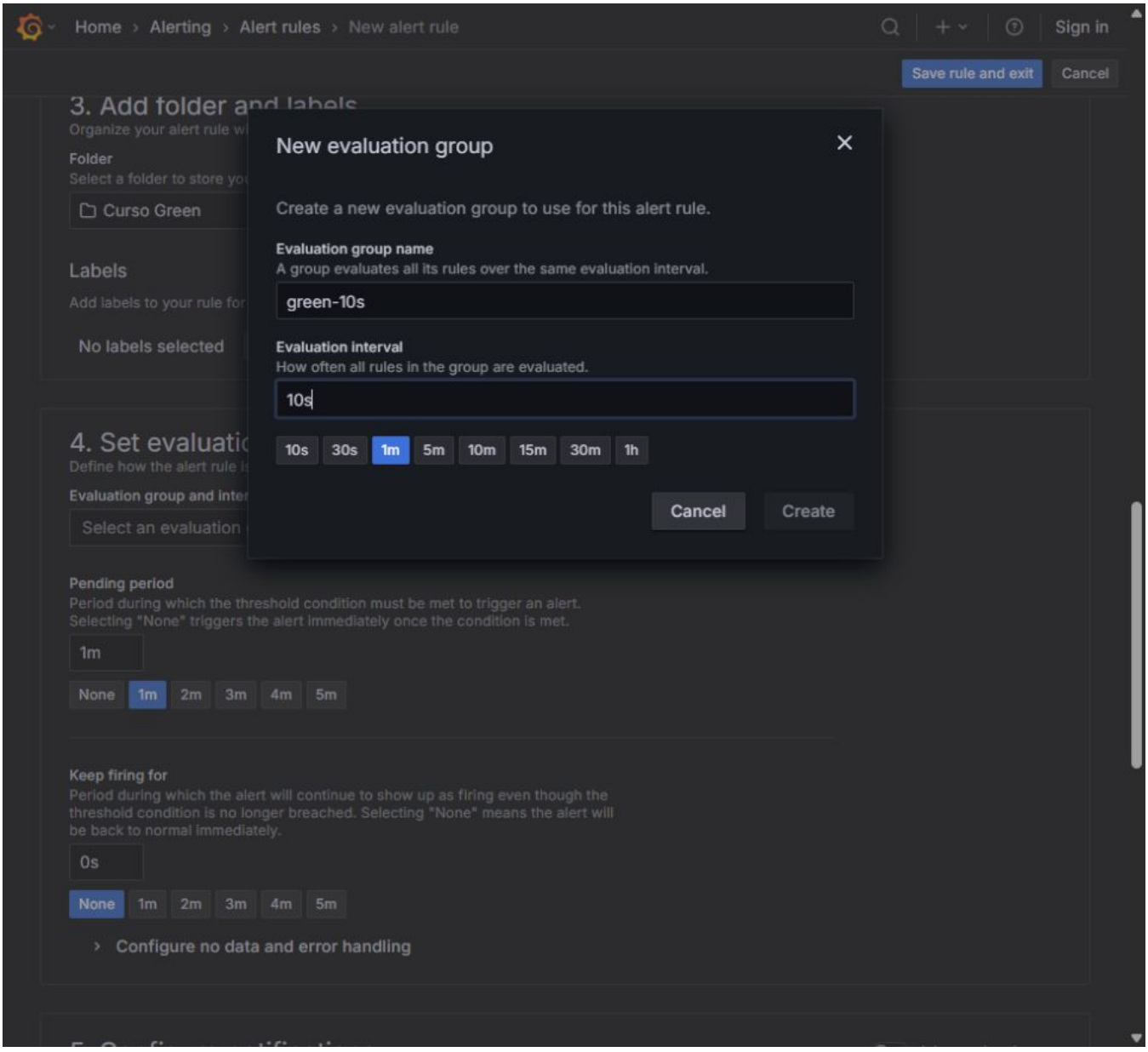
NOITE 5 • EXERCÍCIO 5.2 • ETAPA 6

Criar pasta e grupo de avaliação

1. Na seção Add folder and labels, clique em New folder, digite Curso Green e clique em Create. Se já existir, escolha essa pasta.
2. Na seção Set evaluation behavior, clique em New evaluation group.
3. Em Evaluation group name, digite green-10s. Em Evaluation interval, digite 10s.
4. Clique em Create. Se o grupo já existir na pasta, selecione-o em vez de criar uma duplicata.
5. O grupo define a frequência de avaliação da regra. Não é o intervalo do scrape do Prometheus nem o intervalo de atualização do dashboard.

Resultado esperado: A pasta Curso Green e o grupo green-10s estão selecionados.

Se não funcionar: New evaluation group só fica disponível depois de escolher uma pasta. Se o intervalo for rejeitado, use um múltiplo do intervalo mínimo aceito pela instalação.



Tela real • 38-alerta-grupo.jpg • Abra a imagem original no guia HTML para ampliar.

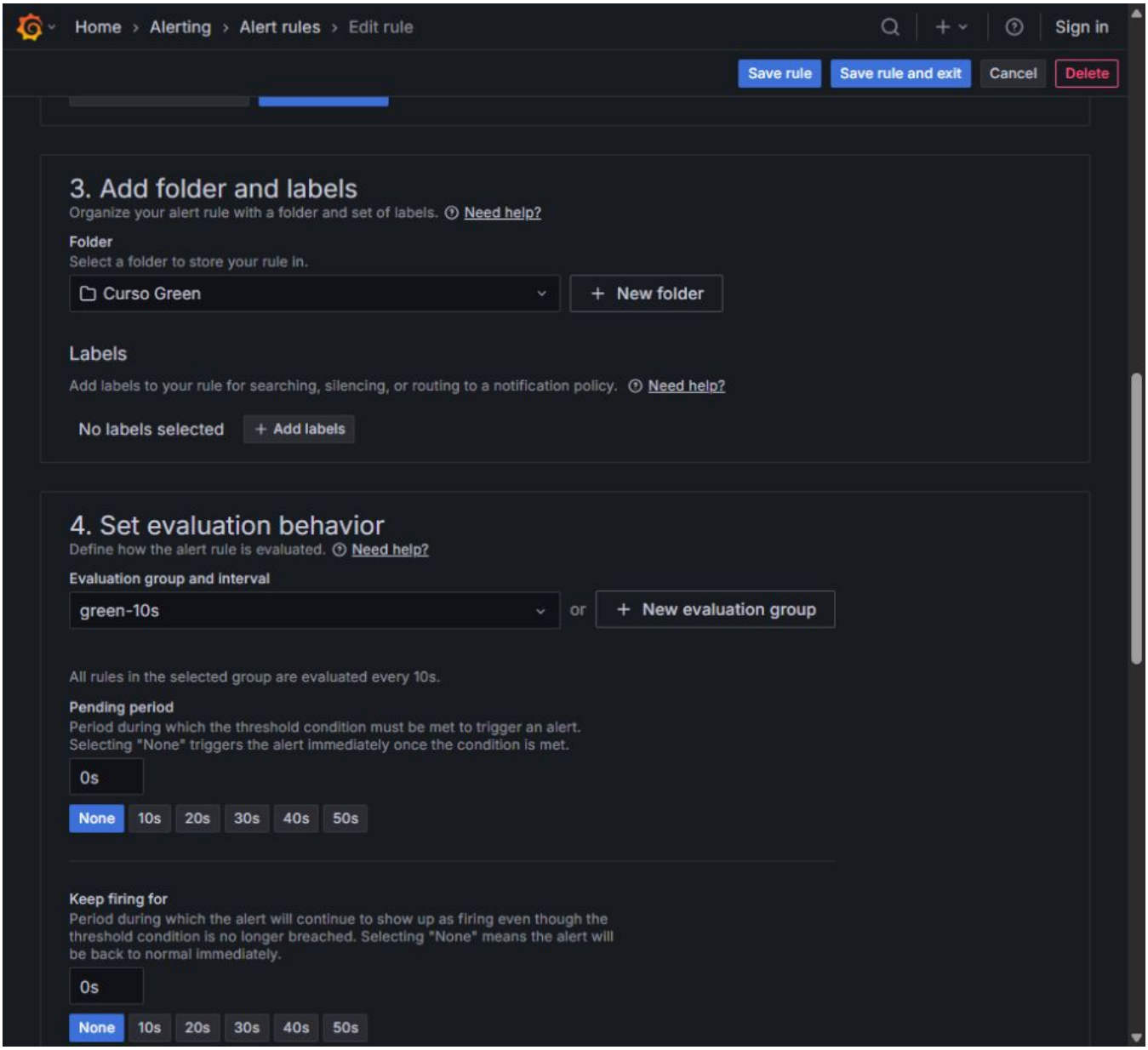
NOITE 5 • EXERCÍCIO 5.2 • ETAPA 7

Configurar pendência e recuperação

1. Confira a informação de que as regras do grupo são avaliadas a cada 10s.
2. Em Pending period, escolha None ou digite 0s. Assim o teste dispara na avaliação que encontrar a condição verdadeira.
3. Em Keep firing for, mantenha None ou 0s, para não prolongar artificialmente o estado do exercício.
4. Deixe Configure no data and error handling nos padrões do laboratório. Anote que No Data e Error são diferentes de uma comparação falsa.
5. Continue até Configure notifications. Não salve a regra ainda sem definir o contato local.

Resultado esperado: Avaliação = 10s, pendência = 0s e Keep firing for = 0s.

Se não funcionar: Um pending de 1m exige a condição verdadeira continuamente durante esse período. O objetivo aqui é observar rapidamente o ciclo; o valor 0s é específico da atividade.



Tela real • 50-alerta-avaliacao.jpg • Abra a imagem original no guia HTML para ampliar.

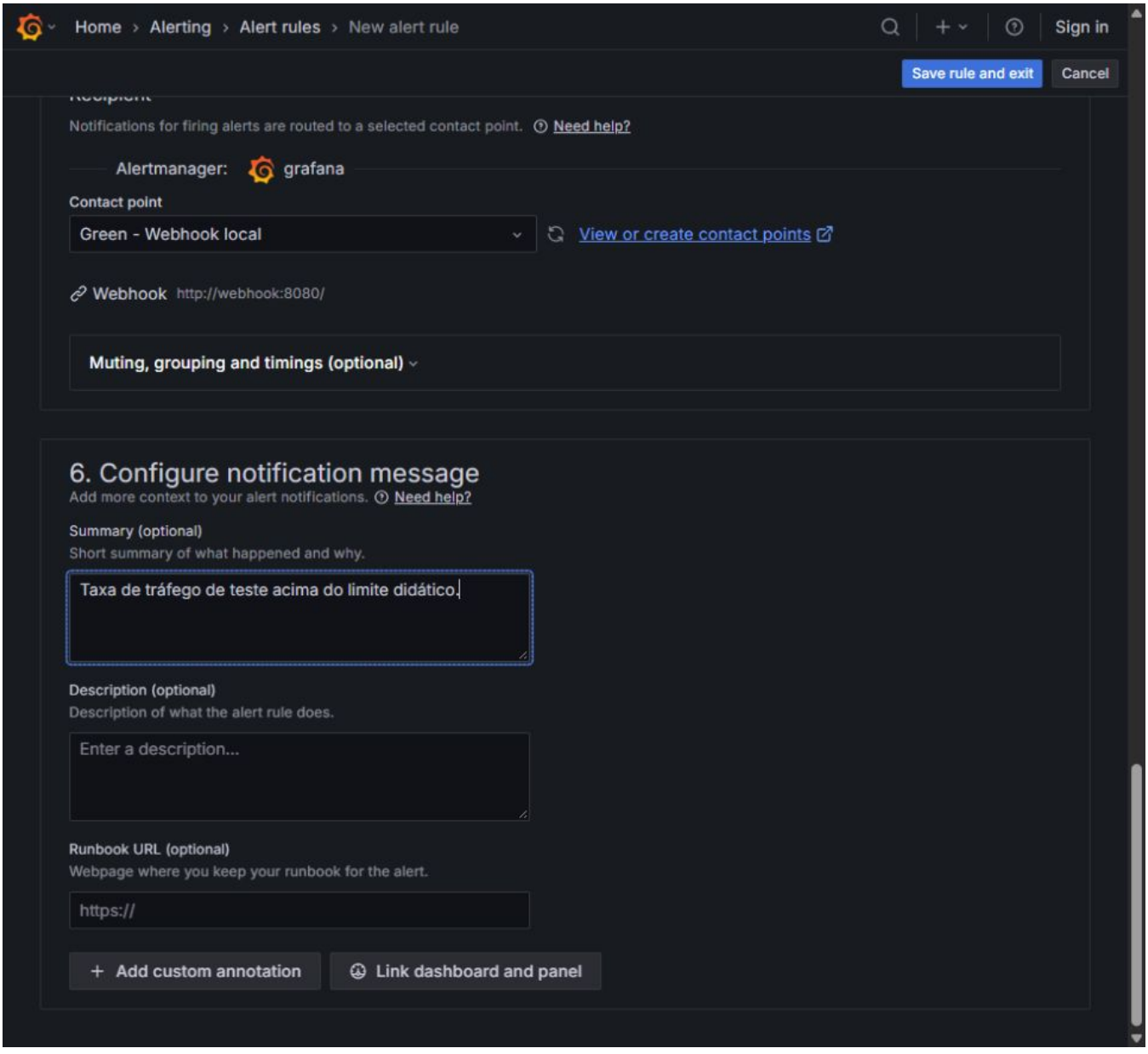
NOITE 5 • EXERCÍCIO 5.2 • ETAPA 8

Associar o contato e a mensagem

1. Em Configure notifications, deixe o Alertmanager grafana selecionado.
2. No campo Contact point, escolha o contato Green - Webhook local criado e testado em 5.1.
3. Confirme que o resumo da integração mostra Webhook e http://webhook:8080/.
4. Em Configure notification message > Summary, digite Taxa de tráfego de teste acima do limite didático.
5. Não escolha grafana-default-email. Esse contato padrão não tem SMTP configurado para o exercício.
6. Você fará o vínculo com o painel na próxima etapa. Depois, poderá salvar a regra.

Resultado esperado: A regra encaminhará as notificações ao receptor Docker local.

Se não funcionar: Se o contato não aparecer, use Refresh contact points list. Se continuar ausente, confira se clicou em Save contact point no exercício 5.1.



Tela real • 39-alerta-contato.jpg • Abra a imagem original no guia HTML para ampliar.

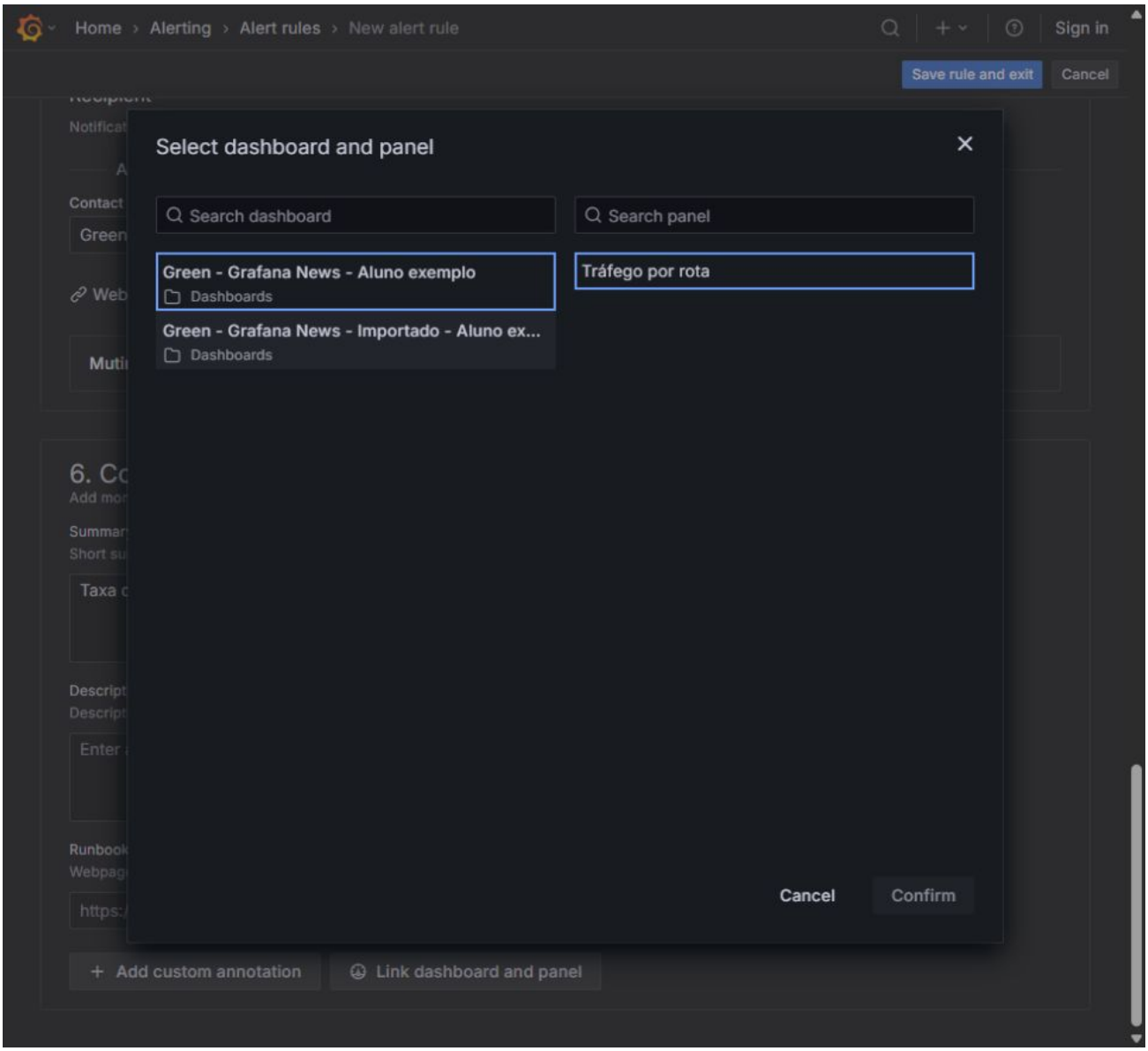
NOITE 5 • EXERCÍCIO 5.3 • ETAPA 9

Relacionar dashboard e painel antes de salvar

1. Na seção Configure notification message, clique em Link dashboard and panel.
2. Na coluna da esquerda, pesquise e selecione seu dashboard manual ou importado. Use o nome que você salvou, não o do exemplo.
3. Aguarde os painéis carregarem. Na coluna da direita, selecione Tráfego por rota, ou o título personalizado.
4. Clique em Confirm e confira os dois links mostrados na regra.
5. Clique em Save rule and exit, no topo, uma única vez. Aguarde retornar à lista de regras.
6. O vínculo acrescenta contexto e navegação. Ele não altera a consulta do painel nem transforma a prévia em mensagem recebida.

Resultado esperado: A regra foi salva, com contato local e referências ao seu dashboard e painel.

Se não funcionar: Se Confirm estiver desabilitado, ainda falta selecionar o painel. Se houver erro ao salvar, leia o campo indicado; revise pasta, grupo, condição C e contato.



Tela real • 40-alerta-vincular.jpg • Abra a imagem original no guia HTML para ampliar.

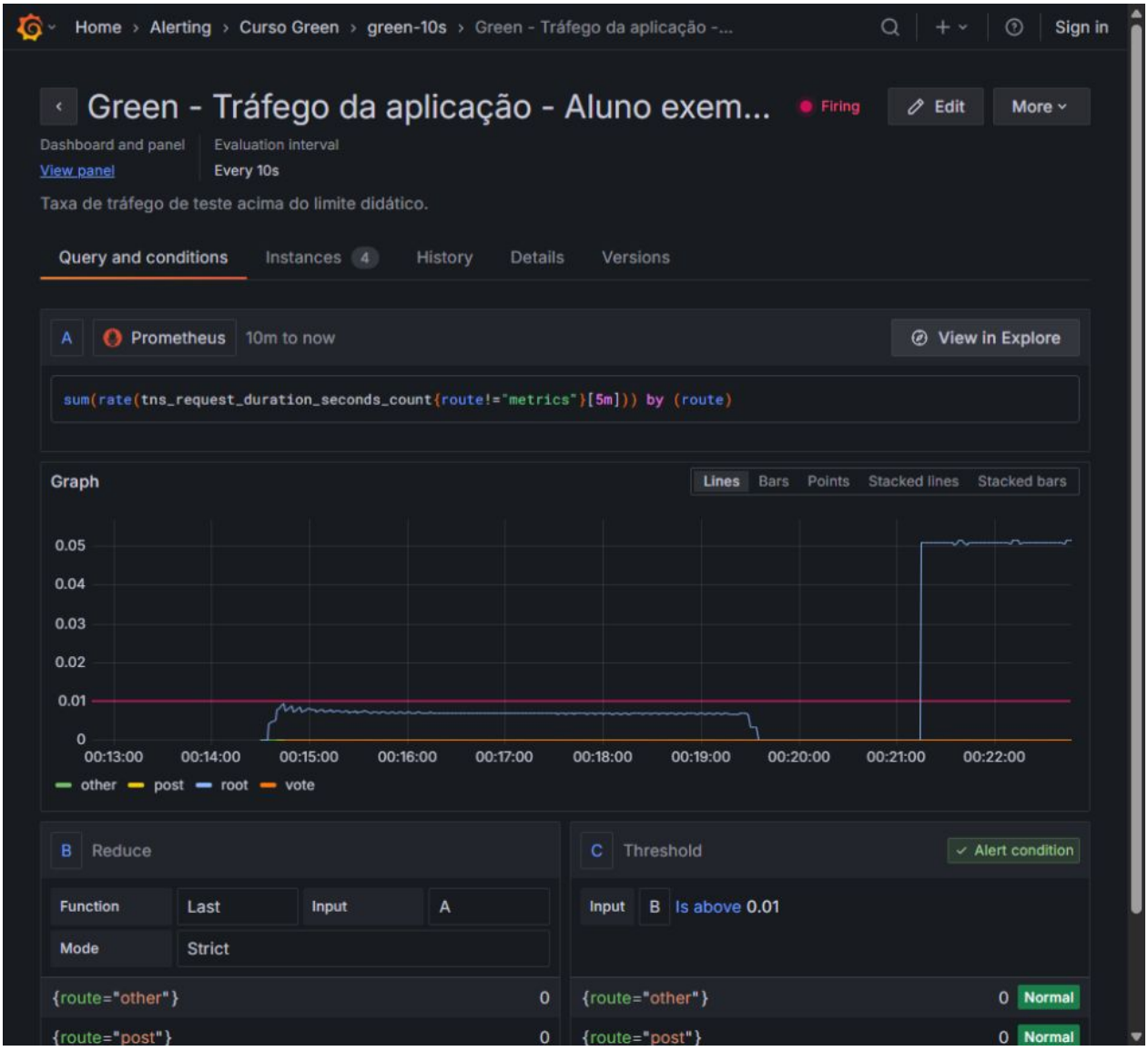
NOITE 5 • EXERCÍCIO 5.2 / 5.3 • ETAPA 10

Verificar a avaliação real

1. Na página Alert rules, abra a pasta Curso Green pela seta ao lado do nome. Clique em View na sua regra.
2. Confira o nome da regra e Every 10s. A aba Query and conditions mostra A, B e C.
3. Se estiver Normal, volte à aplicação, atualize-a 15 a 20 vezes, espere cerca de 10s e volte à regra.
4. Aguarde uma ou duas avaliações. Para a rota root, B deve ultrapassar 0.01; C passa a 1 e o estado fica Firing.
5. Se permanecer Normal, leia o valor de B e compare com o limite. A taxa usa 5 minutos: poucos cliques produzem taxas pequenas.
6. Aguarde também o agrupamento de notificações, normalmente cerca de 30s no lab. Não confunda a avaliação de 10s com o tempo de entrega.

Resultado esperado: A regra salva apresenta Firing para uma rota que ultrapassou o limite.

Se não funcionar: Se a condição foi ajustada para 0.2 em uma demonstração anterior, volte ao limite didático 0.01 deste roteiro. Evite diminuir para valores negativos apenas para forçar um resultado.



Tela real • 42-alerta-firing.jpg • Abra a imagem original no guia HTML para ampliar.

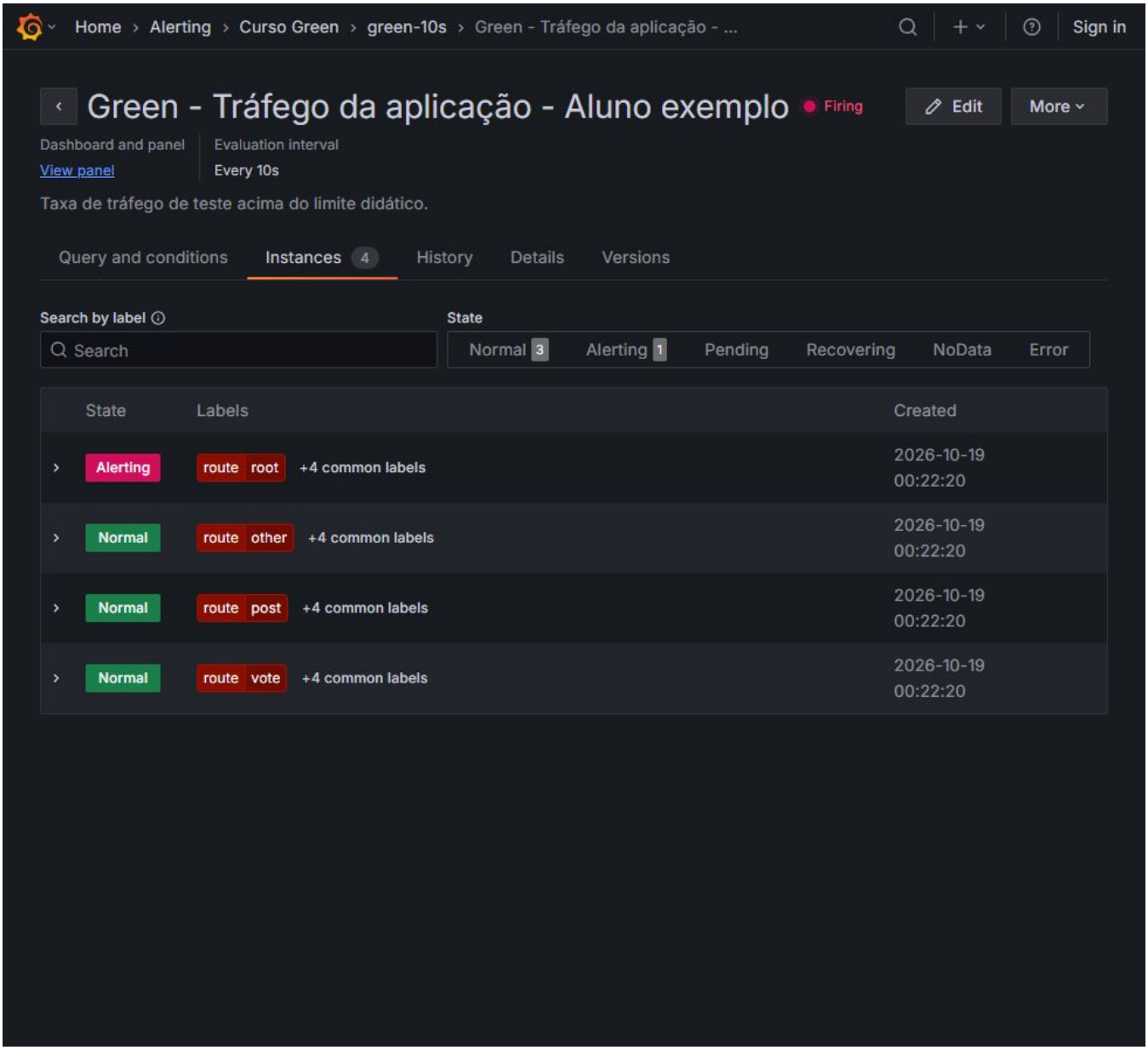
NOITE 5 • EXERCÍCIO 5.2 / 5.3 • ETAPA 11

Distinguir as instâncias de cada rota

1. Na regra, abra a aba Instances. O número ao lado do nome indica quantas instâncias existem naquele momento.
2. Localize route=root e leia o estado. Outras rotas podem continuar Normal.
3. Compare com by (route) da consulta A: cada conjunto de labels gera uma instância independente.
4. Registre a rota que disparou, seu valor B e o horário da observação.
5. Esse comportamento explica por que a regra pode mostrar Firing mesmo com outras rotas normais.

Resultado esperado: Você identifica qual rota realmente satisfez a condição.

Se não funcionar: O número de instâncias pode mudar conforme os labels disponíveis. Não trate o total de 4 do print como obrigatório.



Tela real • 44-alerta-instancias.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 5 • EXERCÍCIO 5.2 / 5.3 • ETAPA 12

Confirmar a notificação da sua regra

1. Atualize a aba http://localhost:38090 depois de a regra estar Firing e de aguardar o envio.
2. A contagem deve aumentar. Role até a mensagem mais recente; no receptor desta versão as mensagens ficam da mais antiga para a mais nova.
3. Procure receiver = Green - Webhook local e labels.alertname com o nome da SUA regra. TestAlert sozinho não satisfaz esta entrega.
4. Confira status = firing, a label route e values.B / values.C. Na captura real, B ficou aproximadamente 0,0508 e C = 1.
5. Faça uma captura incluindo o nome da regra e o status. Registre também o horário received_at; ele está em UTC, três horas à frente do horário de Brasília nesta data.
6. Os links absolutos do payload podem usar a porta padrão 3000 do servidor. Para navegar no pacote local, mantenha a mesma rota na porta 3300, ou use View panel pela interface do Grafana.

Resultado esperado: O receptor tem uma mensagem originada pela regra salva, além do TestAlert.

Se não funcionar: Firing sem mensagem: revise o contato, URL interna, serviço webhook, pausa/silêncio e o tempo de agrupamento. Clique Test no contato para isolar comunicação de avaliação; identifique as duas mensagens separadamente.



Tela real • 43-alerta-recebido.jpg • Abra a imagem original no guia HTML para ampliar.

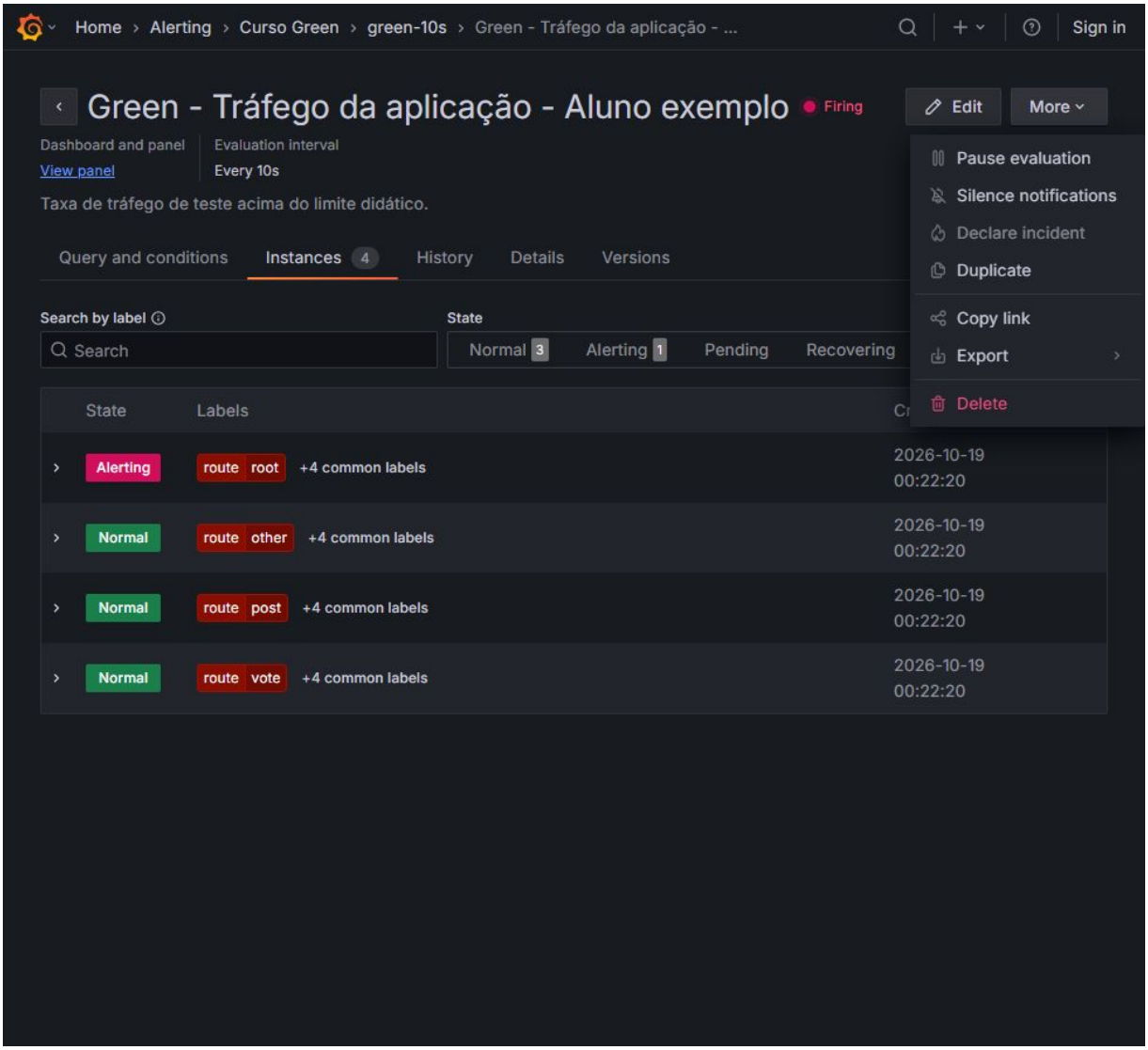
NOITE 5 • EXERCÍCIO 5.3 • ETAPA 13

Encerrar preservando o trabalho

1. Antes de encerrar, entregue: consulta do painel, um erro com horário nos logs, anotação manual, consulta de anotações e evidência de recebimento da regra.
2. Exporte o dashboard por Export > Export as JSON, com Export the dashboard to use in another instance ligado.
3. Na página da regra, abra More > Export e guarde também sua definição, se desejar preservar a regra. Regras não acompanham o JSON do dashboard.
4. Depois das evidências, use More > Pause evaluation na regra criada por você. Confira o texto Paused.
5. Não use Delete para encerrar. Para retomar outra aula, abra More > Resume evaluation.
6. Se for parar o lab, vá à pasta lab/tutorial-environment e use Compose stop com o mesmo projeto green-fundamentals. Preserve a pasta e os arquivos exportados; não use down -v.

Resultado esperado: Você concluiu as evidências e pausou somente sua regra didática.

Se não funcionar: O dashboard JSON não contém contatos, regras, logs, eventos manuais nem os dados históricos do Prometheus. Antes de remover containers, preserve cada item de que precisa.



Tela real • 45-alerta-pausar.jpg • Abra a imagem original no guia HTML para ampliar.

NOITE 5 • EXERCÍCIO Consulta opcional • ETAPA 14

Parar e retomar o laboratório com os comandos exatos

1. Antes de parar, confira que exportou seu dashboard e guardou as evidências. Os comandos stop preservam os containers; não use down -v.
2. No Windows, abra o PowerShell na raiz extraída do pacote. Execute os três primeiros comandos abaixo. Teste docker compose version: se funcionar, use a linha docker compose; se não existir, use apenas o bloco de alternativa Docker Desktop.
3. No Ubuntu, abra o terminal na raiz do pacote, entre em lab/tutorial-environment e execute a linha stop da rota Compose instalada.
4. Para retomar no Windows, abra um novo terminal na raiz e execute iniciar-lab.ps1 como na Noite 1. No Ubuntu, execute bash ./iniciar-lab.sh na raiz.
5. Depois de retomar, reabra os três endereços. Se precisar continuar a prática de alertas, use More > Resume evaluation somente na regra que criou.

```
# PowerShell: começar na raiz do pacote extraído
Set-Location .\lab\tutorial-environment
docker compose version
docker compose -p green-fundamentals stop

# Alternativa se docker compose não existir no Docker Desktop
$composeCurso = "$env:ProgramFiles/Docker/Docker/resources/cli-plugins/docker-compose.exe"
& $composeCurso -p green-fundamentals stop

# Ubuntu: começar na raiz do pacote extraído
cd lab/tutorial-environment
docker compose -p green-fundamentals stop
```

Resultado esperado: O laboratório foi parado sem remoção dos containers e você conhece o caminho de retomada.

Se não funcionar: Se estiver usando uma instalação diferente da pasta desta turma, confirme o nome do projeto antes de executar. Não remova volumes de outros projetos para corrigir uma falha de inicialização.

Noite 5 — conferência da entrega

5.1 — Contato e TestAlert (20 minutos)

Concluído: ____ Evidência salva em: _____

5.2 — Regra, avaliação e disparo (30 minutos)

Concluído: ____ Evidência salva em: _____

5.3 — Vínculo, apresentação e exportação (25 minutos)

Concluído: ____ Evidência salva em: _____

Registre para a correção

Fonte / consulta / caminho utilizado: _____

Horário, período e resultado observado: _____

O que esse resultado significa: _____

Se houve falha: mensagem exata e verificações já feitas: _____

Noite 2 — Desafios opcionais

Desafios opcionais (2.4 a 2.6)

Depois de concluir 2.1 a 2.3, quem terminar antes das 22h15 pode continuar. Os desafios são opcionais e usam as mesmas telas. As capturas de referência vieram do ambiente do professor (loja virtual); no seu laboratório use as consultas `tns_`.

Desafio 2.4 — Filtrar por label: tirar o scrape

Objetivo: usar um filtro de label dentro do seletor para responder uma pergunta mais específica.

1. No Explore, com Prometheus e Code ativos, clique no editor da consulta A e use Ctrl+A para substituir o texto.

2. Cole a consulta abaixo e clique em Run query (ou Shift+Enter). Ela é a taxa por rota do exercício 2.2, sem a rota `metrics`.

3. Compare a legenda com a do exercício 2.2: qual série desapareceu? Por que ela existia?

4. Troque o filtro por `{status_code="200"}` e execute de novo. Quais rotas continuam aparecendo?

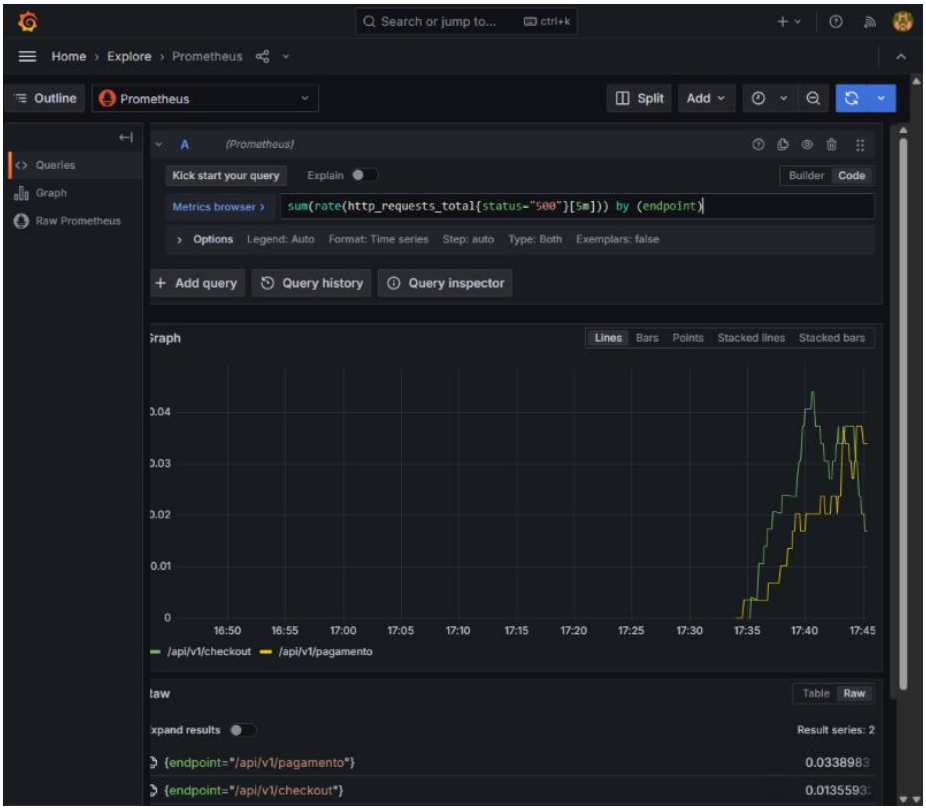
5. Registre a pergunta que cada consulta responde e a unidade (req/s).

```
sum(rate(tns_request_duration_seconds_count{route!="metrics"}[5m])) by (route)
```

Resultado esperado: a rota `metrics` (coleta do Prometheus) não aparece. Ficam apenas as rotas geradas por ações na aplicação, como root, post e vote.

Se não funcionar: em No data, confirme a fonte Prometheus e gere tráfego na aplicação. Um filtro que não corresponde a nenhuma série devolve No data, e isso é diferente de zero. Use `!=` (diferente de) e aspas retas, sem espaços dentro das chaves.

Captura de referência (ambiente do professor): `sum(rate(http_requests_total{status="500"}[5m])) by (endpoint)`. O filtro `{status="500"}` reduz a legenda a poucas séries.



Desafio 2.5 — Taxa ou total: rate e increase

Objetivo: distinguir "quantos por segundo" de "quantos na janela".

1. Execute a consulta abaixo. O resultado é **o aumento estimado de requisições na janela de 5 minutos**, e não req/s.

2. Anote o valor de uma rota na aba Raw.

3. Execute a consulta do exercício 2.2 (`rate`) e anote o valor da mesma rota.

4. Multiplique a taxa por 300 (segundos de 5 minutos). No mesmo instante e com a mesma janela, increase corresponde a rate multiplicado por 300; a comparação manual pode sofrer arredondamento ou diferença de horário.

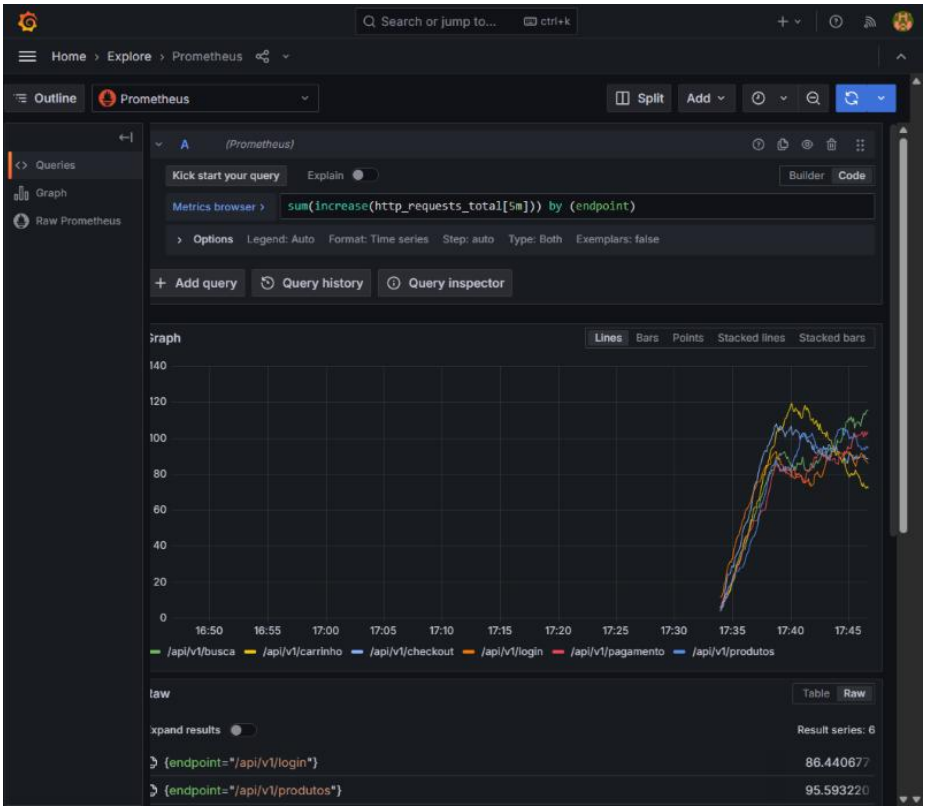
5. Explique por que a comparação exige o mesmo instante de avaliação, janela e seleção de séries.

```
sum(increase(tns_request_duration_seconds_count[5m])) by (route)
```

Resultado esperado: o gráfico mostra contagens em vez de frações. No ambiente do professor, uma rota teve `increase` ≈ 82 e `rate` ≈ 0,27 req/s; 0,27 × 300 ≈ 80.

Se não funcionar: compare as consultas no mesmo instante e com a mesma janela e filtros. increase é rate multiplicado pela duração da janela; os dois usam a mesma extrapolação. Instantes diferentes ou arredondamentos podem alterar a comparação manual. O resultado pode ser fracionário. Em um laboratório recém-iniciado, com poucas atualizações, os totais serão pequenos.

Captura de referência (ambiente do professor): `sum(increase(http_requests_total[5m])) by (endpoint)`.



Desafio 2.6 — Latência: percentil 95 com histograma

Objetivo: ler uma distribuição de duração e entender por que p95 não é média nem máximo.

1. Execute a consulta abaixo. O resultado está em **segundos**.

2. Leia a legenda e identifique qual rota tem o p95 mais alto.

3. Execute a segunda consulta (média de duração por rota) e compare com o p95 da mesma rota.

4. Escreva, em uma ou duas frases, o que significa "95% das requisições foram atendidas em até X segundos".

```
histogram_quantile(0.95, sum(rate(tns_request_duration_seconds_bucket[5m])) by (le,
route))
```

Média para comparação:

```
sum(rate(tns_request_duration_seconds_sum[5m])) by (route)
  / sum(rate(tns_request_duration_seconds_count[5m])) by (route)
```

Resultado esperado: compare p95 e média: são resumos diferentes da distribuição. O p95 não é obrigatoriamente maior que a média em toda distribuição. No ambiente do professor, duas rotas tinham p95 próximo de 0,45 s, e as demais ficavam perto de 0,23 s; as médias eram menores (0,09 s a 0,22 s).

Se não funcionar: mantenha `le` no `by (...)`. Sem ele, `histogram_quantile` não encontra os buckets. Com poucas requisições, o percentil pode oscilar ou aparecer como NaN: gere mais tráfego e aguarde novas coletas. O percentil é estimado a partir de faixas (buckets); por isso o valor é aproximado.

Captura de referência (ambiente do professor): `histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, endpoint))`.

